

THE SPRING BOOT PLAYBOOK

From First API to Hired Developer

by @JavaInterviewready

A NOTE BEFORE YOU BEGIN

Let me tell you something most Spring Boot guides won't.

Knowing Spring Boot is not the same as being ready for a Spring Boot interview. I have seen developers who built REST APIs, connected databases, and even added JWT security- and still froze when asked WHY auto-configuration works the way it does.

Interviewers are not testing whether you can add @RestController. They are testing whether you have actually thought about the framework. Whether you understand the decisions behind it. Whether you can handle a follow-up question without going blank.

Most candidates know what @Autowired does.

Very few can explain what happens when two beans of the same type exist in the context.

That gap is exactly why people with real project experience still get rejected.

This guide exists to close that gap. Not with definitions you can find on any blog. With the actual depth that gets you a nod in the room.

WHO BUILT THIS

I am a backend developer. Spring Boot is what I use every day at work.

JPA relationships, security filters, transaction rollbacks, N+1 problems at 2am. That is my world. This playbook is built from real interview experience, real production problems, and the exact questions I have seen trip people up time and again.

I did not write this to cover every Spring Boot topic. I wrote it to cover the RIGHT topics, at the RIGHT depth, with the context that actually matters when someone is sitting across from you in an interview.

WHAT THIS GUIDE IS NOT

Not a textbook. Not a list of annotations. Not another 'top 50 interview questions' post.

Every topic here is covered the way a senior developer would explain it to you the evening before your interview. Short where short is enough. Deep where depth is what gets you hired.

You will find interview traps, real follow-up questions, code examples that show what breaks and why, and the exact answers that separate a good candidate from the one who gets the offer.

WHO THIS IS FOR

Final year students heading into campus placements. Working developers preparing for a switch. Anyone who has used Spring Boot but still feels unsure walking into an interview room.

If you have built a Spring Boot project but cannot confidently explain how auto-configuration works, what happens when you miss mappedBy in a JPA relationship, or why constructor injection is recommended over field injection- this guide is for you.

Read it like a mentor wrote it for you.

Because that is exactly what this is.

- @JavaInterviewready

SPRING BOOT INTERVIEW SERIES

The Spring Boot Playbook

From First API to Hired Developer

CHAPTER 1

The Core Engine

A Senior Developer's Guide to Real Interview Preparation

PREMIUM EDITION • ₹149

© The Spring Boot Playbook. All rights reserved.

CHAPTER 1 - THE CORE ENGINE

Before you write your first REST endpoint or configure your first database, you need to understand what Spring Boot actually is. Not just at a surface level. The real question in interviews is whether you understand the **why** behind the things you use every day. This chapter builds that foundation- from how Spring and Spring Boot differ, to what happens in the first milliseconds when your application starts.

1. Spring Framework vs Spring Boot

Let me start with a question I see confuse a lot of freshers: Is Spring Boot a replacement for Spring? The answer is no. Spring Boot is **built on top of Spring**. They are not competing. They are complementary.

Spring Framework came first. It solved real enterprise problems- dependency injection, transaction management, data access- and it did it well. But setting it up was painful. You had to write XML configuration files. You had to manually tell Spring which beans to create, which server to use, which version of everything to wire together. A simple web project could have a dozen config files before you even wrote a single business logic line.

Spring Boot came along and asked: "**what if we could make all of this automatic?**"

Think of it like this. Spring Framework is like a fully equipped kitchen- you have every tool you need, but you have to set everything up, arrange it, and figure out what goes where. Spring Boot walks in and says "**here, I already set up your kitchen the way 90% of people want it.**" You can still rearrange things if you want to. But most of the time, you don't need to.

The difference, in plain words:

Spring Framework = The powerful foundation. Requires lots of configuration.

Spring Boot = Spring + sensible defaults + auto-configuration + embedded server.

Spring Boot doesn't replace Spring. It makes Spring easier to use.

When an interviewer asks "what is the difference between Spring and Spring Boot", they want you to show that you understand both layers- not just that "**Spring Boot is easier**". Anyone can say that. Say instead: Spring Boot auto-configures your application based on the dependencies on the classpath, provides an embedded

server, and removes the need for boilerplate XML configuration while still using the Spring Framework underneath.

2. Auto-Configuration- The Magic Behind the Curtain

This is one of the most important topics in this chapter. Interviewers love it because it separates people who have actually thought about Spring Boot from people who have just used it.

Here is the scenario. You add `spring-boot-starter-data-jpa` to your project. You add a datasource URL in `application.properties`. And suddenly your repository works. You never told Spring to create a `DataSource` bean. You never configured `EntityManagerFactory`. How did any of that happen?

Auto-configuration is the answer. Let's walk through how it actually works.

The classpath check

When your application starts, Spring Boot scans the classpath- that is, all the JARs and classes available to your application. It looks for specific classes. If it finds, say, `HikariDataSource` on the classpath (which comes in with the JPA starter), it says: **"Oh, the developer probably wants a connection pool. Let me configure one."**

This decision-making happens through something called **conditional annotations**. Specifically annotations like `@ConditionalOnClass`, `@ConditionalOnMissingBean`, and `@ConditionalOnProperty`. These are what power auto-configuration.

Where auto-configuration classes live

In older Spring Boot versions (before 2.7), there was a file called `META-INF/spring.factories` inside the `spring-boot-autoconfigure` JAR. It listed every auto-configuration class that Spring Boot ships with. There were hundreds of them.

In Spring Boot 3.x, this moved to

`META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports`.

Same concept, cleaner structure.

When your app starts, Spring Boot loads all these auto-configuration classes. But- and this is important- it does **not** activate all of them. Each one has conditions. `DataSourceAutoConfiguration` only fires if a `datasource` class is on the classpath **and** if no `DataSource` bean is already defined by you.

Auto-Configuration Flow (step by step):

1. `SpringApplication.run()` triggers context initialization
2. Spring Boot loads all auto-config class names from `AutoConfiguration.imports`
3. For each class, it checks the `@Conditional` conditions
4. If conditions pass → the configuration is applied
5. If you defined your own bean → Spring Boot steps aside (`ConditionalOnMissingBean`)
6. Result: your app has exactly what it needs, auto-wired and ready

There's a handy way to see what auto-configuration fired in your app. Add this to `application.properties`:

```
logging.level.org.springframework.boot.autoconfigure=DEBUG  
  
# Or use the actuator endpoint:  
management.endpoints.web.exposure.include=conditions
```

Run the app and check the logs. You'll see exactly which auto-configurations were applied and which were skipped and why. This is also a great debugging trick when something isn't getting configured the way you expect.

The key thing to say in an interview is: **"Auto-configuration is not magic. It is conditional class-based configuration that fires only when specific conditions are met on the classpath. It can always be overridden by defining your own beans."**

3. Starter Dependencies

You've probably used starters. `spring-boot-starter-web`, `spring-boot-starter-data-jpa`, `spring-boot-starter-security`. But do you know what they actually are?

A starter is just a **convenience dependency descriptor**. It's a Maven or Gradle dependency that pulls in a bunch of other dependencies that you typically need together. That's all.

Take `spring-boot-starter-web` as an example. When you add it, you automatically get:

- Spring MVC for building web and REST APIs

- Jackson for JSON serialization/deserialization
- Embedded Tomcat server (so you don't need to deploy to an external server)
- Spring Boot auto-configuration for all of the above

Without starters, you'd have to manually add all these dependencies, figure out compatible versions, and hope nothing conflicts. Starters solve this. They're version-tested and compatible by design.

A common mistake beginners make is thinking a starter **activates** features. It doesn't. A starter just brings dependencies into your classpath. It's **auto-configuration** that then reads those dependencies and configures beans. The two work together, but they're separate concepts.

4. @SpringBootApplication- The Three-in-One Annotation

You put `@SpringBootApplication` on your main class. It sits there looking simple. But this single annotation is doing three things at once, and every interviewer expects you to know what those three things are.

```
@SpringBootApplication
public class MyApp {
    public static void main(String[] args) {
        SpringApplication.run(MyApp.class, args);
    }
}
```

`@SpringBootApplication` is a composite annotation. Under the hood, it includes three others:

@Configuration

This tells Spring: **"this class can define beans"**. Any method annotated with `@Bean` inside a `@Configuration` class will have its return value registered as a bean in the Spring context. It's the modern Java-based replacement for XML configuration.

@EnableAutoConfiguration

This is what triggers the entire auto-configuration process we discussed. Without this, Spring Boot would not scan `AutoConfiguration.imports`, would not apply any defaults, and your app would be just a regular Spring app with zero setup.

@ComponentScan

This tells Spring to scan the current package and all sub-packages for classes annotated with `@Component`, `@Service`, `@Repository`, `@Controller`, and similar annotations- and register them as beans automatically.

Here's something that trips up a lot of beginners: if you put your main class in the wrong package, `@ComponentScan` might not find your other classes. Always make sure your main class is in the **root package**, one level above all your other packages. Like this:

```
com.myapp           ← MyApp.java lives here
com.myapp.controller ← UserController.java
com.myapp.service    ← UserService.java
com.myapp.repository ← UserRepository.java
```

If `MyApp.java` is inside `com.myapp.controller` by mistake, it won't scan the `com.myapp.service` package. Your services won't be found. You'll get a `NoSuchBeanDefinitionException` and spend 45 minutes wondering what went wrong.

5. @Configuration and @Bean- When to Use Each

This is where a lot of people get fuzzy. They know `@Component` is "automatic" and `@Bean` is "manual", but they can't explain why you'd choose one over the other. Let's clear this up.

When you annotate a class with `@Component` (or `@Service`, `@Repository`- they're all `@Component` variants), Spring finds it during component scanning and registers it. You're in control of the class. Simple.

But what about classes you **don't** control? Say you're using a third-party library and you need to register one of its objects as a Spring bean. You can't put `@Component` on it- it's not your code. This is exactly where `@Bean` inside a `@Configuration` class comes in.

```
@Configuration
public class AppConfig {

    @Bean
    public ObjectMapper objectMapper() {
        ObjectMapper mapper = new ObjectMapper();
        mapper.configure(DeserializationFeature.FAIL_ON_UNKNOWN_PROPERTIES,
false);
        return mapper;
    }
}
```


Here, `ObjectMapper` belongs to Jackson. We can't touch that class. But by putting it inside a `@Bean` method in a `@Configuration` class, we're telling Spring: **"create this object and manage it as a bean."** We also get to customize it before it's registered.

Quick Rule of Thumb:

Your own class → use `@Component` / `@Service` / `@Repository`

Third-party class OR needs custom construction → use `@Bean` in `@Configuration`

Complex setup, multiple related beans → group them in a `@Configuration` class

One thing worth knowing: a `@Configuration` class uses CGLIB proxying under the hood. This means if one `@Bean` method calls another `@Bean` method inside the same class, Spring intercepts that call and returns the existing bean from the context rather than creating a new instance. This keeps your beans as true singletons. If you use `@Component` instead of `@Configuration`, this proxying doesn't happen and you might accidentally create duplicate instances.

6. application.properties vs application.yml

Both files do the same job. They configure your Spring Boot application. The difference is format, readability, and personal preference. But there are practical things to know.

Here's the same configuration in both formats:

```
# application.properties
spring.datasource.url=jdbc:mysql://localhost:3306/mydb
spring.datasource.username=root
spring.datasource.password=secret
spring.jpa.hibernate.ddl-auto=update
server.port=8081
```

```
# application.yml
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/mydb
    username: root
    password: secret
  jpa:
    hibernate:
```

```
    ddl-auto: update
server:
  port: 8081
```

With properties files, every key is fully spelled out. You repeat `spring.datasource` over and over. YAML uses indentation to show hierarchy, so you type `spring` once and nest everything under it. For deeply nested configs, YAML is noticeably cleaner.

Where YAML can bite you: **indentation errors**. A single misplaced space breaks the whole file with a cryptic error. Properties files don't have this problem. Some teams prefer properties for this reason, especially for simpler projects.

One practical advantage of YAML is that it supports **multi-document files** using `---` as a separator. This lets you have multiple Spring profiles in a single YAML file, which is neat for small projects.

```
# Single yml file with multiple profiles
spring:
  application:
    name: my-app
---
spring:
  config:
    activate:
      on-profile: dev
  datasource:
    url: jdbc:h2:mem:devdb
---
spring:
  config:
    activate:
      on-profile: prod
  datasource:
    url: jdbc:mysql://prod-server/mydb
```

You can't do this with a `.properties` file. For multi-profile setups in properties, you'd need separate files: `application-dev.properties` and `application-prod.properties`.

In interviews, just be clear: both work, YAML is better for complex nested configs, properties is simpler and safer for beginners. Most real-world projects use one or the other consistently- just pick one and be consistent.

7. Embedded Server- Why It's a Bigger Deal Than You Think

Before Spring Boot, deploying a Java web application meant packaging it as a WAR file and deploying it to an external server- Tomcat, JBoss, WebLogic. The server had to be installed, configured, and running before your app could start. Developers wasted hours on environment setup before writing any code.

Spring Boot changed this by **embedding the server inside your application**. Tomcat (or Jetty or Undertow) is a dependency in your `pom.xml`. When you run your application, the server starts **inside** your JVM process. Your app is a standalone JAR that contains everything it needs.

```
# This is all you need to run a Spring Boot app
java -jar myapp.jar

# No external Tomcat needed.
# No WAR file. No deployment descriptor.
# The server is inside the JAR.
```

This might seem like a small thing until you think about deployment. In the world of cloud and containers, you want each service to be **self-contained**. A Docker container running a Spring Boot JAR is clean, portable, and doesn't need any pre-installed server. This is a huge reason Spring Boot became the standard for microservices.

The default embedded server is Tomcat. If you want Jetty instead, just exclude Tomcat from the web starter and add the Jetty starter:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
  <exclusions>
    <exclusion>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-tomcat</artifactId>
    </exclusion>
  </exclusions>
</dependency>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-jetty</artifactId>
</dependency>
```

The server port defaults to 8080. Change it with `server.port=9090` in `application.properties`. If you set it to 0, Spring Boot picks a random available port- useful in tests when you don't want port conflicts.

8. Project Structure- Where Everything Lives

When you generate a Spring Boot project from `start.spring.io`, you get a specific directory structure. Understanding it is basic but necessary- you'll be asked about it, and you'll use it every day.

```
myapp/
├── src/
│   ├── main/
│   │   ├── java/
│   │   │   ├── com/myapp/
│   │   │   │   ├── MyAppApplication.java    ← main class
│   │   │   │   ├── controller/
│   │   │   │   ├── service/
│   │   │   │   ├── repository/
│   │   │   │   └── model/
│   │   └── resources/
│   │       ├── application.properties      ← config
│   │       ├── static/                    ← CSS, JS, images
│   │       └── templates/                 ← Thymeleaf views
│   └── test/
│       ├── java/
│       │   ├── com/myapp/
│       │   └── MyAppApplicationTests.java
├── pom.xml                                ← Maven config
└── .mvn/
```

src/main/java

All your Java source code goes here. Organized by package, typically following the **layered architecture** pattern: controllers handle HTTP requests, services contain business logic, repositories talk to the database, and model/entity classes represent your data.

src/main/resources

Everything that's not Java code goes here. Your `application.properties` or `application.yml` lives here. The `static/` folder serves static web assets. The `templates/` folder holds template files if you're building server-side rendered views. Anything in this directory gets included in the final JAR.

The main class

This is your entry point. It has the `main()` method, `@SpringBootApplication` on the class, and a call to `SpringApplication.run()`. That's it. Keep this class clean. Don't put business logic in it.

And as mentioned earlier- keep this class in the **root package**. Not inside a sub-package. This ensures component scanning covers everything below it.

9. How a Spring Boot Application Actually Starts

This is the full picture. Walk an interviewer through this and they'll know you understand what's happening under the hood, not just how to write a controller.

Spring Boot Startup Flow:

- Step 1 → `main()` method is called
- Step 2 → `SpringApplication.run(MyApp.class, args)` is invoked
- Step 3 → `SpringApplication` creates a `BootstrapContext`, determines the app type (web/non-web)
- Step 4 → `ApplicationContext` is created (e.g., `AnnotationConfigServletWebServerApplicationContext` for web apps)
- Step 5 → `ApplicationListeners` and `ApplicationContextInitializers` are prepared
- Step 6 → Environment is set up- loads `application.properties` / `application.yml`
- Step 7 → `@ComponentScan` runs- beans from your classes are registered
- Step 8 → Auto-configuration classes are evaluated and applied
- Step 9 → All beans are instantiated and dependencies are injected
- Step 10 → Embedded server starts (if web application)
- Step 11 → `ApplicationReadyEvent` is published → your app is live

Each step matters. When your app fails to start, one of these steps went wrong. Understanding the flow helps you diagnose problems faster. For example, if a bean fails to autowire, that's step 9. If the server can't bind to port 8080, that's step 10.

One thing that trips up beginners is the **bean lifecycle** within step 9. Spring instantiates beans in order of their dependencies. If Bean A depends on Bean B, Spring creates B first, then A, then injects B into A. If there's a circular dependency (A needs B, B needs A), Spring will throw an error at startup- which is actually good, because it's better to fail fast than to silently fail at runtime.

```
// Basic startup- this is all it takes
@SpringBootApplication
public class MyAppApplication {

    public static void main(String[] args) {
        SpringApplication.run(MyAppApplication.class, args);
        // After this line returns, your app is fully started
        // Beans are ready. Server is up. You're live.
    }
}
```

One more thing. `SpringApplication.run()` actually returns the `ApplicationContext` object. Most people ignore this return value. But if you ever need to grab a bean programmatically after startup- for a quick test or a one-time migration- you can use it:

```
ApplicationContext ctx = SpringApplication.run(MyAppApplication.class, args);
UserService userService = ctx.getBean(UserService.class);
// Not common in production, but useful to know
```

Chapter 1 Summary

You now have a solid foundation. Let's do a quick recap before you move to Chapter 2.

1. **Spring vs Spring Boot:** Spring Boot is built on Spring, not a replacement. It adds auto-configuration, starters, and an embedded server.
2. **Auto-Configuration:** Not magic. It uses conditional annotations and classpath detection to apply sensible defaults. Always overridable.
3. **Starters:** Curated dependency bundles. They bring code to the classpath; auto-configuration does the rest.
4. **@SpringBootApplication:** Three annotations in one- `@Configuration`, `@EnableAutoConfiguration`, `@ComponentScan`. Know what each does.
5. **@Configuration vs @Bean:** Use `@Component` for your own classes. Use `@Bean` in `@Configuration` for third-party or complex objects.
6. **Properties vs YAML:** Same purpose, different format. YAML is cleaner for nesting; properties is simpler to manage.
7. **Embedded Server:** Tomcat runs inside your JAR. No external setup needed. Perfect for containers and microservices.

8. **Project Structure:** Main class in root package. Java code in src/main/java. Config in src/main/resources.
9. **Startup Flow:** main() → SpringApplication.run() → context creation → component scan → auto-config → beans → server starts.

Before Your Interview:

Be able to explain the startup flow without reading from notes.

Know the difference between @Component and @Bean by heart.

Understand that auto-configuration is conditional, not unconditional.

If asked "how does Spring Boot work internally", walk through the startup steps.

Up next: Chapter 2- Building REST APIs That Interviewers Actually Want to See

@JavaInterviewready

SPRING BOOT INTERVIEW SERIES

The Spring Boot Playbook

From First API to Hired Developer

CHAPTER 2

The Annotation Arsenal

TOP 1% TEACHING EDITION

Beans · Injection · Scopes · Lifecycle

© The Spring Boot Playbook. All rights reserved.

CHAPTER 2 - THE ANNOTATION ARSENAL

This is where most freshers stall. Not because the concepts are complex, but because they learn the **syntax** without learning the **thinking**. They can write `@Autowired`. They cannot explain what happens if you don't use it. They know `@Service` exists. They cannot tell you what would break if you swapped it for `@Component`.

This chapter fixes that. We're not covering annotations. We're covering **why Spring works the way it does**, and what happens when you get it wrong.

1. What is a Bean?

Here's the actual problem Spring is solving. In a large application, you have hundreds of objects. A `UserService` needs a `UserRepository`. An `OrderService` needs both `UserService` and `PaymentService`. Every object needs other objects. If each class creates its own dependencies with `new`, you end up with deeply tangled code where swapping one class means touching twenty others.

Spring's answer to this is the **IoC container**- a centralized registry that creates objects, manages their connections, and hands them out on demand. The objects it manages are called **beans**.

A Spring Bean is not a special type of class. It's not something that extends a base class or implements an interface. It's simply **a Java object whose entire lifecycle Spring owns**. Spring decides when it's created, what it receives, and when it's destroyed.

```
// This object is NOT a bean. Spring has no idea it exists.
public class UserService {
    public void createUser() { ... }
}

// This IS a bean. Spring owns it from creation to destruction.
@Service
public class UserService {
    public void createUser() { ... }
}
```

The moment you add `@Service` (or any `@Component` variant), Spring's component scanner picks it up at startup, creates one instance, stores it in the **ApplicationContext**, and makes it available anywhere in your app. No `new UserService()`. No manual wiring. Spring handles all of it.

Two ways to register a bean:

1. Component scanning: put `@Component` (or `@Service`, `@Repository`, `@Controller`) on the class
2. Manual `@Bean` declaration: return an instance from a method inside a `@Configuration` class

Component scanning is for your own classes.

`@Bean` is for third-party classes you can't annotate directly.

And that distinction between the two registration methods is not just trivia. If you need to register a **Jackson ObjectMapper** or a **RestTemplate**- classes you don't own- you can't put `@Component` on them. You write a `@Bean` method inside a `@Configuration` class, and Spring registers the return value.

INTERVIEW TRAP

What most freshers say:

"A bean is an object managed by Spring."

What the interviewer expects:

That answer isn't wrong, but it's shallow.

A stronger answer: "A Spring bean is a Java object whose lifecycle is fully managed by the Spring IoC container. Spring creates it, resolves and injects its dependencies, manages its initialization hooks, and destroys it when the context shuts down. Beans are registered either via component scanning or `@Bean` declarations in a `@Configuration` class, and they live in the `ApplicationContext`."

→ IF YOU REMEMBER ONE THING

A bean isn't special because of what it IS. It's special because of who OWNS it. Spring owns it. That's the whole idea.

2. @Component vs @Service vs @Repository vs @Controller

This is where most people give a technically correct but professionally weak answer. They say "they all do the same thing". An interviewer at a product company will follow up with: "then why does Spring have four?" Most people can't answer that cleanly.

Here's the complete picture.

Annotation	Layer	What Spring Actually Does
@Component	Generic / any layer	Registers the bean. Nothing else.
@Service	Business logic	Registers the bean. Signals intent. No extra Spring behaviour today.
@Repository	Data access	Registers the bean + enables persistence exception translation via a proxy.
@Controller	Web (MVC / HTML)	Registers the bean + enables Spring MVC request mapping.
@RestController	Web (REST / JSON)	@Controller + @ResponseBody. Returns serialized JSON, not view names.

The Only One With Real Runtime Behaviour: @Repository

When you annotate a class with `@Repository`, Spring wraps it in a proxy. This proxy catches low-level database exceptions- `SQLExceptions` from JDBC, Hibernate-specific exceptions, JPA exceptions- and translates them into Spring's `DataAccessException` hierarchy.

Why does this matter? Because `DataAccessException` is database-agnostic. Whether you're using MySQL, PostgreSQL, or MongoDB, your service layer gets the same exception type. You can write error-handling code in your service without worrying about which database threw the original exception. That's a meaningful feature, not just a name.

Here's the failure scenario: you annotate a DAO class with `@Component` instead of `@Repository`. The bean gets registered. Component scanning finds it. Everything looks fine. But if a database query fails, the raw Hibernate or JDBC exception propagates up to your service layer instead of a clean `DataAccessException`. Your error handling breaks. You won't notice until something goes wrong in production.

@Service: Semantic Today, Potentially Functional Tomorrow

Right now, `@Service` has no extra runtime behaviour beyond registering the bean. But two things still matter. First, it communicates to every developer on your team: **business logic lives here**. Second, the Spring team has left it as a distinct annotation precisely because they **can** add behaviour to it in future versions without breaking backward compatibility. Use it. It costs nothing and communicates clearly.

@RestController Is Not Just Syntax Sugar

A lot of freshers think `@RestController` is just a "shorter" `@Controller`. That misses the point. The `@ResponseBody` part changes what Spring does with your method's return value entirely.

With `@Controller` alone, Spring assumes you're returning a view name- a string that maps to a Thymeleaf template or JSP file. With `@RestController`, every return value is serialized directly to the HTTP response body as JSON (via Jackson). If you use `@Controller` by mistake on a REST endpoint, Spring tries to resolve the returned object as a view name and throws a `ViewNotFoundException`. This is a real mistake that wastes real debugging time.

INTERVIEW TRAP

What most freshers say:

*"@Service is for the service layer, @Repository for database layer."
(When asked: what's the actual difference between them?)*

What the interviewer expects:

"All four are specializations of `@Component` and trigger component scanning. The only one with distinct runtime behaviour is `@Repository`, which enables persistence exception translation through a Spring-created proxy. `@Service` is currently semantic, but using it correctly signals architectural intent and allows Spring to add layer-specific behaviour in future versions. `@RestController` is `@Controller` + `@ResponseBody`- it's for REST APIs and differs fundamentally from `@Controller` in how return values are handled."

→ IF YOU REMEMBER ONE THING

They all register beans. Only @Repository does something extra at runtime. Use the right annotation for the right layer anyway- it communicates intent, and @Repository's proxy only fires when the annotation is correct.

3. Dependency Injection

Let's start with why this exists, because most explanations skip that part.

Without DI, objects create their own dependencies. `OrderService` calls `new UserService()` inside its constructor. This means `OrderService` is now tightly coupled to `UserService`- it knows about its constructor, its arguments, its implementation. If you want to swap `UserService` for a mock in a test, you

can't. If you want to change how `UserService` is constructed, you have to change `OrderService` too. In a large codebase, this cascades.

DI inverts this. `OrderService` doesn't create `UserService`. It **declares that it needs one**. Spring reads this declaration and provides it. This is **Inversion of Control**- the control over object creation moves from the class itself to the container.

@Autowired- What It Actually Does

When Spring sees `@Autowired`, it performs a lookup in the `ApplicationContext` for a bean that matches the required type. This is called **type-based dependency resolution**. If exactly one bean of that type exists, it gets injected. If zero beans exist, Spring throws `NoSuchBeanDefinitionException` at startup. If multiple beans exist, Spring throws `NoUniqueBeanDefinitionException`. Both failures happen at application startup, not at runtime- which is actually a good thing.

```
@Service
public class OrderService {

    @Autowired
    private UserService userService; // Spring resolves this by type at
    startup

    public void placeOrder(Long userId) {
        String name = userService.getUser(userId);
    }
}
```

This is field injection. It works. But it has three specific problems that you need to be able to explain, not just recite.

Constructor Injection vs Field Injection

This is the question that separates developers who understand Spring from developers who just use it.

```
// Field injection- the problematic way
@Service
public class OrderService {
    @Autowired
    private UserService userService;
}

// Constructor injection- the right way
@Service
```

```
public class OrderService {  
  
    private final UserService userService;  
  
    // @Autowired not needed since Spring 4.3- single constructor is  
    auto-detected  
    public OrderService(UserService userService) {  
        this.userService = userService;  
    }  
}
```

The difference looks cosmetic. It isn't. Here are the three concrete problems with field injection:

Why Field Injection Fails in Production

PROBLEM 1- UNTESTABLE WITHOUT SPRING:

With field injection, you cannot create `OrderService` in a unit test by calling `new OrderService()`. The field is private and Spring-injected. To test it, you need to either spin up a Spring context (slow) or use reflection hacks to set the field. With constructor injection: `new OrderService(mockUserService)`. Done. Fast. Clean.

PROBLEM 2- HIDES DESIGN PROBLEMS:

With field injection, you can add 10 `@Autowired` fields and the class still looks clean. With constructor injection, a constructor with 7 parameters is a loud signal: this class has too many responsibilities. Fix the design, don't hide it.

PROBLEM 3- NO IMMUTABILITY:

Field injection prevents final fields. Final fields mean thread-safety guarantees and protection against accidental reassignment. Without final, nothing stops someone from setting `userService = null` later in the code. It's a subtle, dangerous gap.

With constructor injection, the field is **final**. It's set once. It cannot be changed. And because the dependency is passed through the constructor, Spring 4.3+ detects it automatically- no `@Autowired` annotation needed on the constructor at all.

In production code, always use Lombok's `@RequiredArgsConstructor` to avoid the boilerplate:

```
@Service
```

```
@RequiredArgsConstructor // generates constructor for all final fields
public class OrderService {

    private final UserService userService;
    private final PaymentService paymentService;

    // No @Autowired anywhere. Spring finds the Lombok-generated constructor.
    // Both fields are final. This class is safe, testable, and clean.
}
```

Senior-Level Note

There is a third injection type: setter injection, using @Autowired on a setter method.

It exists for a specific reason: optional dependencies. When a dependency is not always required- for example, a cache that may or may not be present- setter injection lets Spring skip it if the bean isn't found. Constructor injection fails if any dependency is missing. Setter injection does not. For genuinely optional dependencies, setter injection is the correct choice. For everything else, use constructor injection.

INTERVIEW TRAP

What most freshers say:

"Constructor injection is better because it's the recommended way."

What the interviewer expects:

"I prefer constructor injection for three concrete reasons. First, it enables final fields- true immutability. Second, it makes unit testing straightforward: I can instantiate the class directly with a mock without needing a Spring context. Third, it surfaces design problems- a constructor with 7 parameters tells me the class has too many responsibilities, which field injection would hide. Field injection also prevents circular dependency detection at startup, which can let broken wiring reach production."

→ IF YOU REMEMBER ONE THING

Constructor injection is not a style preference. It's a design decision that makes your code testable, immutable, and honest about its complexity.

4. @Primary vs @Qualifier

Real scenario. You're building a payment system. You have two implementations of **PaymentGateway**: **StripeGatewayService** and **RazorpayGatewayService**. Both are Spring beans. Your **OrderService** injects **PaymentGateway**. Spring throws **NoUniqueBeanDefinitionException** at startup. Your entire application fails to boot before a single request is served.

This is not a corner case. Multiple implementations of the same interface is one of the most common patterns in production Spring code. You need to know both solutions cold.

```
public interface PaymentGateway {
    void charge(BigDecimal amount);
}

@Service public class StripeGatewayService implements PaymentGateway { ... }
@Service public class RazorpayGatewayService implements PaymentGateway { ... }

// Spring finds TWO beans of type PaymentGateway.
// It does not guess. It throws NoUniqueBeanDefinitionException.
@Autowired
private PaymentGateway paymentGateway; // STARTUP FAILURE
```

@Primary- Set the Default

Mark one implementation with **@Primary**. Spring uses it everywhere **PaymentGateway** is injected, unless explicitly overridden. Use this when one implementation is the correct default 90% of the time.

```
@Service
@Primary // Use Stripe unless told otherwise
public class StripeGatewayService implements PaymentGateway { ... }

// Now this works- gets StripeGatewayService
@Autowired
private PaymentGateway paymentGateway;
```

@Qualifier- Be Explicit

When you need a specific non-primary bean, or when there's genuinely no "default", use **@Qualifier**. By default, Spring uses the class name in camelCase as the qualifier name. **StripeGatewayService** becomes **"stripeGatewayService"**. You can also define a custom name using **@Qualifier** on the class itself.

```
// Injecting a specific bean by its default name (camelCase class name)
@Service
```



```

@RequiredArgsConstructor
public class RefundService {

    // We need Razorpay specifically for refunds
    @Qualifier("razorpayGatewayService")
    private final PaymentGateway refundGateway;
}

// Or use @Qualifier on the implementation for a cleaner name
@Service
@Qualifier("razorpay")
public class RazorpayGatewayService implements PaymentGateway { ... }

// Then inject by that cleaner name
@Qualifier("razorpay")
private final PaymentGateway refundGateway;

```

One thing tutorials almost never show: `@Qualifier` works perfectly with constructor injection. Put it directly on the constructor parameter. You don't need field injection for this:

```

@Service
public class RefundService {

    private final PaymentGateway refundGateway;

    public RefundService(
        @Qualifier("razorpay") PaymentGateway refundGateway) {
        this.refundGateway = refundGateway;
    }
}

```

Spring also does name-based fallback matching without any qualifier. If your field name matches a bean name exactly, Spring resolves it. But this is fragile- rename the class and it breaks silently. Don't rely on this. Be explicit.

Senior-Level Note

In large codebases, string-based `@Qualifier` is brittle. Rename the bean class and the qualifier string doesn't update. The compiler won't catch it. You get a startup failure at best, or worse, the wrong bean silently injected if bean name matching finds something.

The type-safe solution: create a custom qualifier annotation.

```
@Qualifier
@Retention(RetentionPolicy.RUNTIME)
public @interface RazorpayGateway {}
```

Then use `@RazorpayGateway` on both the implementation and the injection point.

If the class is renamed, the annotation remains correct. IDEs navigate through it.

This is the production-grade approach for teams with multiple implementations.

INTERVIEW TRAP

What most freshers say:

*"Use @Primary for the default and @Qualifier for specific beans."
(Correct but shallow- no understanding of the failure scenario or production implications)*

What the interviewer expects:

"When Spring finds multiple beans of the same type, it throws `NoUniqueBeanDefinitionException` at startup. `@Primary` marks one implementation as the default for type-based injection. `@Qualifier` resolves a specific bean by name when the default isn't what you need. For large codebases, I'd prefer custom qualifier annotations over string-based `@Qualifier` because they're type-safe, refactor-friendly, and navigable in IDEs."

→ IF YOU REMEMBER ONE THING

@Primary and @Qualifier don't compete. @Primary sets the default. @Qualifier overrides it at a specific injection point. Knowing when to use which is what matters.

5. Bean Lifecycle

Most developers know that beans have a lifecycle. Very few know it well enough to use it correctly or explain it under pressure. Here's where people fail: they memorise "`@PostConstruct` runs after creation, `@PreDestroy` runs before destruction" without understanding **when, why, or what can go wrong**

The Full Lifecycle- Accurately

Bean Lifecycle: What Actually Happens:

Step 1 Spring calls the constructor (bean is instantiated)

Step 2 Dependencies are injected (constructor params or @Autowired fields)

Step 3 BeanPostProcessors run their beforeInitialization hooks

Step 4 @PostConstruct method is called (YOUR initialization logic)

Step 5 InitializingBean.afterPropertiesSet() called (if implemented)

Step 6 @Bean(initMethod=...) called (if specified)

Step 7 BeanPostProcessors run their afterInitialization hooks

>>> BEAN IS FULLY READY. SERVING REQUESTS. <<<

Step 8 @PreDestroy method is called (YOUR cleanup logic)

Step 9 DisposableBean.destroy() called (if implemented)

Step 10 Bean is removed from context and eligible for GC

For interviews, you need steps 1, 2, 4, and 8 cold. The others are real and worth knowing, but steps 4 and 8 are what you'll actually use.

@PostConstruct- Not Just Initialization

Here's the mistake people make most often with lifecycle. They put setup logic in the constructor, like opening a connection or loading a config. With field injection, this fails silently: at the time the constructor runs, the injected fields are still `null`. You call `this.dataSource.getConnection()` and get a `NullPointerException`. Not at the right time. Not with a helpful message.

@PostConstruct runs **after** all injection is complete. It's the guaranteed safe place to run any logic that depends on your dependencies.

```
@Service
public class CacheService {

    @Autowired
    private DataSource dataSource;    // null in constructor, available by
    @PostConstruct

    private Map<String, Object> cache;

    @PostConstruct
    public void warmUp() {
        // dataSource is fully injected here. Safe to use.
        this.cache = loadCacheFromDatabase(dataSource);
        System.out.println("Cache warmed. " + cache.size() + " entries
loaded.");
    }
}
```

```
@PreDestroy
public void shutdown() {
    // Flush, close, release- anything that needs cleanup
    cache.clear();
    System.out.println("Cache cleared. Clean shutdown.");
}
```

Important technical note: `@PostConstruct` and `@PreDestroy` are not Spring annotations. They come from `jakarta.annotation` (formerly `javax.annotation`). They're part of the Jakarta EE standard. Spring honours them, but they work in non-Spring environments too. If an interviewer asks "are these Spring annotations?", the correct answer is no.

Watch Out

`@PreDestroy` does NOT fire for prototype-scoped beans.

Spring creates prototype beans on demand and hands them to whoever asks. After that, Spring holds no reference to them. It cannot call `@PreDestroy` on something it doesn't track.

This means: if your prototype bean opens a database connection, starts a thread, or acquires any resource- YOU are responsible for cleaning it up. Spring won't do it.

This catches people in production. The resource leak is real.

InitializingBean and DisposableBean: Don't Use Them

Spring provides these interfaces as an older alternative to `@PostConstruct` and `@PreDestroy`.

The problem: implementing them couples your class to the Spring API. If you ever want to use that class outside Spring, you're dragging Spring with you as a dependency.

`@PostConstruct` and `@PreDestroy` are standard Java annotations. Non-invasive. Portable.

Always use them instead.

INTERVIEW TRAP

What most freshers say:

"@PostConstruct runs after the bean is created and @PreDestroy runs before it's destroyed."

What the interviewer expects:

"@PostConstruct runs after the bean is instantiated AND all its dependencies are injected. This is important- using injected fields in a constructor is unsafe because they're null at that point. @PostConstruct is the correct place for setup logic that depends on injected beans. @PreDestroy runs before the context shuts down and is meant for releasing resources. Worth noting: @PreDestroy does NOT fire for prototype-scoped beans because Spring stops tracking them after handing them off."

→ IF YOU REMEMBER ONE THING

@PostConstruct is not just 'runs after creation'. It runs after INJECTION IS COMPLETE. That's the only reason it exists.

6. Bean Scopes

Scope defines **who gets which instance and when**. Get this wrong and you're shipping race conditions into production. This is one of those areas where a single misunderstanding can cause bugs that only appear under load, never in your unit tests, never on your local machine.

Singleton- One Instance, Every Thread

Singleton is the default. One instance per ApplicationContext, created at startup, shared by every part of your application, forever. Most developers know this. What they don't think about is the implication: **every thread in your application is using the same object simultaneously**.

```
// DANGEROUS: mutable state in a singleton
@Service
public class OrderService {
    private List<String> processingLogs = new ArrayList<>(); // shared
    across ALL threads

    public void process(String orderId) {
        processingLogs.add(orderId); // Thread A and Thread B both write
        here.
        // ArrayList is not thread-safe. This corrupts data under concurrent
        load.
    }
}

// SAFE: stateless singleton
@Service
```

```
public class OrderService {
    public String process(String orderId) {
        return "Processed: " + orderId;    // No shared state. Thread-safe by
        design.
    }
}
```

The rule for singletons is simple but non-negotiable: **singletons must be stateless**. They process inputs and return outputs. They don't hold data that varies by request, user, or thread. Every field in a singleton should either be a constant or another injected bean (which follows the same rule).

Senior-Level Note

One subtle but important singleton behaviour. In a `@Configuration` class, if one `@Bean` method calls another `@Bean` method, Spring intercepts that call via a CGLIB proxy and returns the existing singleton from the context rather than creating a new instance.

This preserves singleton semantics when beans have cross-dependencies. If you use `@Component` instead of `@Configuration`, CGLIB proxying does NOT apply. Calling one `@Bean` method from another creates a new instance each time. This is a real source of subtle duplicate-instance bugs that are difficult to trace.

Prototype- New Instance Every Time

Prototype creates a fresh instance every time the bean is requested from the container. Use it for **stateful objects**- things that accumulate data, hold state per-operation, or are not safe to share across calls.

```
@Component
@Scope("prototype")
public class ReportBuilder {
    private final List<String> lines = new ArrayList<>();

    public ReportBuilder addLine(String line) {
        lines.add(line);
        return this;    // fluent
    }

    public String build() {
        return String.join("\n", lines);
    }
}
```

The Singleton-Prototype Injection Trap: This is Where Most People Fail

This is the most asked senior-level scope question. It catches people who memorised the definition of prototype scope without thinking about how injection actually works.

The Scenario That Breaks:

You have a prototype-scoped ReportBuilder.

You inject it into a singleton-scoped OrderService.

What do you get?

A SINGLE ReportBuilder instance. The same one. Forever.

Here's why: the singleton is created once at startup. Spring injects ReportBuilder into it once at that moment. After startup, the singleton holds a fixed reference to that one instance. No new instances are created on subsequent method calls.

Prototype scope has been completely bypassed.

```
// THE BROKEN PATTERN
@Service // singleton- created once
public class OrderService {

    @Autowired
    private ReportBuilder builder; // prototype injected ONCE at startup

    public void generateReport() {
        builder.addLine("Line"); // Same builder every time. State
        // After 100 calls: 100 lines in one builder. Totally wrong.
    }
}
```

The fix: don't inject the prototype bean. Inject a `ObjectProvider<T>`, which is Spring's lazy-resolution wrapper. Every call to `.getObject()` goes back to the container and gets a genuinely fresh instance:

```
// THE CORRECT PATTERN
@Service
@RequiredArgsConstructor
public class OrderService {
```

```

private final ObjectProvider<ReportBuilder> builderProvider;

public void generateReport() {
    ReportBuilder builder = builderProvider.getObject(); // fresh every
time
    builder.addLine("Line 1");
    builder.addLine("Line 2");
    System.out.println(builder.build());
    // This builder is then garbage collected. Clean state next call.
}
}

```

Web Scopes: Request and Session

Scope	New Instance Per	Use Case
singleton	ApplicationContext (once)	Stateless services, repos, config- the vast majority of beans
prototype	Every getBean() / injection	Stateful builders, per-operation processors
request	HTTP request	Request-specific data holders
session	HTTP session	User session state (use carefully in distributed systems)
application	ServletContext (once)	App-wide shared web data (rare)

A word on session scope in distributed systems: if you're running multiple instances of your service behind a load balancer, session-scoped beans are a problem. Session data lives in one JVM. When the load balancer routes the next request to a different instance, the session bean isn't there. This is why modern distributed architectures often push session state into a shared store like Redis rather than relying on in-JVM session scope.

Senior-Level Note

Lazy initialization is worth knowing as a related concept.

@Lazy on a bean defers its creation until the first time it's requested.

By default, all singletons are eagerly created at startup.

spring.main.lazy-initialization=true makes ALL beans lazy globally.

This reduces startup time- useful for local development and Lambda/serverless deployments where cold start matters. In production services with steady traffic, prefer eager initialization: it surfaces wiring errors at startup instead of

serving the first user a slow response while beans spin up.

INTERVIEW TRAP

What most freshers say:

*"Prototype creates a new bean every time it's injected."
(Interviewer follows up: What if I inject a prototype into a singleton?)*

What the interviewer expects:

"Prototype scope creates a new instance every time the bean is requested from the container. But there's a critical trap: injecting a prototype into a singleton only happens once, at the singleton's creation time. After that, the singleton holds a fixed reference to a single prototype instance- which defeats the purpose of prototype scope entirely.

The correct solution is to use `ObjectProvider<T>`, which lazily resolves the bean on each call, ensuring a fresh prototype instance every time."

→ IF YOU REMEMBER ONE THING

Singletons must be stateless. Prototype scope in a singleton injection point is a trap- use `ObjectProvider<T>` to get genuinely fresh instances.

Chapter 2 Summary

Here's what this chapter was actually about: not annotations, but **decisions**. Every annotation in Spring is answering a question. `@Service` answers "what does this class do?". Constructor injection answers "how do I keep this testable and honest?". `@Primary` answers "which bean should win when there's ambiguity?". `@PostConstruct` answers "when is it safe to use my dependencies?".

If you understand the question each annotation answers, you can explain it confidently in any interview- not by reciting definitions, but by **explaining the problem it solves**.

1. **Bean:** Spring-owned object. `ApplicationContext` holds it. Two registration paths: component scan or `@Bean` in `@Configuration`.
2. **@Repository:** The only stereotype with real runtime behaviour- persistence exception translation via proxy. Others are semantic.
3. **@RestController:** Not just shorter syntax. Adds `@ResponseBody`. Without it, Spring tries to resolve your return value as a view name.

4. **Constructor injection:** Enables final fields, clean unit tests, design feedback, and early circular dependency detection. Not a preference- a practice.
5. **@Primary / @Qualifier:** @Primary sets the default. @Qualifier overrides it. Use custom qualifier annotations in large codebases for type safety.
6. **@PostConstruct:** Runs after injection is complete. The only safe place to use injected dependencies in initialization logic.
7. **@PreDestroy:** Does NOT fire for prototype beans. If prototype holds resources, you manage cleanup manually.
8. **Singleton:** One instance, shared by all threads. Must be stateless. Mutable fields in singletons create race conditions under load.
9. **Prototype in singleton:** Injected once at startup. Use `ObjectProvider<T>` to get a fresh prototype instance on each method call.
10. **@Configuration vs @Component:** @Configuration uses CGLIB proxy- @Bean method calls return existing beans from context. @Component does not.

Before Your Interview:

Say 'constructor injection' and explain 3 reasons without pausing.

Walk through the singleton-prototype injection trap- the problem, why it happens, `ObjectProvider` fix.

Explain why `@Repository` behaves differently from `@Service` at runtime.

Know that `@PreDestroy` does not fire for prototype beans. That is asked.

Know that `@PostConstruct` and `@PreDestroy` are Jakarta EE annotations, not Spring-specific.

Be ready: 'What happens if two beans of the same type exist and you `@Autowire` by type?'

Up next: Chapter 3- Building REST APIs That Interviewers Actually Want to See

EXTRA VALUE SECTION

Before You Walk In

Your last-minute companion- cheat sheet, 15 rapid-fire Q&As, and the mindset to close the deal.

Quick Revision Cheat Sheet

Don't read this section slowly. Scan it. If something looks unfamiliar, flip back to that chapter. This is your 10-minute refresh before the interview, not a replacement for actually reading the book.

Core Spring Boot Annotations

Annotation	What It Does	Where It Lives
<code>@SpringBootApplication</code>	Auto-configures everything. Entry point of any Spring Boot app.	Main class
<code>@Component</code>	Registers a class as a Spring-managed bean.	Any utility/helper class
<code>@Service</code>	Same as <code>@Component</code> - signals this class holds business logic.	Service layer classes
<code>@Repository</code>	Same as <code>@Component</code> - also enables DB exception translation.	Data access / DAO classes
<code>@Controller</code>	Marks a class as an MVC controller that returns views (HTML).	MVC web apps (Thymeleaf etc.)
<code>@RestController</code>	= <code>@Controller</code> + <code>@ResponseBody</code> . Returns JSON directly.	REST API controllers
<code>@Configuration</code>	Declares a class that defines Spring beans manually.	Config / bean setup classes
<code>@Bean</code>	Tells Spring to manage the return value of this method as a bean.	<code>@Configuration</code> classes

Dependency Injection

Annotation	What It Does	Where It Lives
<code>@Autowired</code>	Injects a Spring bean automatically. Works on field, constructor, setter.	Service, Controller, etc.

<code>@Qualifier</code>	Tells Spring which bean to inject when multiple options exist.	With <code>@Autowired</code>
<code>@Primary</code>	Makes one bean the default when multiple of the same type exist.	On the preferred <code>@Bean</code> / <code>@Component</code>

Web Layer (REST)

Annotation	What It Does	Where It Lives
<code>@RequestMapping</code>	Base path for a controller class, or a catch-all mapping method.	Class or method level
<code>@GetMapping</code>	Maps HTTP GET requests to a method.	Fetch / read operations
<code>@PostMapping</code>	Maps HTTP POST requests to a method.	Create new resource
<code>@PutMapping</code>	Maps HTTP PUT requests to a method.	Full update of a resource
<code>@DeleteMapping</code>	Maps HTTP DELETE requests to a method.	Remove a resource
<code>@PathVariable</code>	Extracts a value from the URL path: <code>/users/{id}</code>	Method parameter
<code>@RequestParam</code>	Reads a query parameter from the URL: <code>?name=rahul</code>	Method parameter
<code>@RequestBody</code>	Converts the JSON request body into a Java object automatically.	POST / PUT method parameters
<code>ResponseBody<T></code>	Wraps the response- lets you set status code, headers, and body.	Controller method return type

JPA / Data Layer

Annotation	What It Does	Where It Lives
<code>@Entity</code>	Marks a class as a JPA entity (maps to a DB table).	Model/domain classes
<code>@Table</code>	Specifies the exact DB table name if different from class name.	On <code>@Entity</code> classes
<code>@Id</code>	Marks the primary key field.	Entity field
<code>@GeneratedValue</code>	Auto-generates the primary key value (e.g. auto-increment).	On the <code>@Id</code> field
<code>@Column</code>	Maps a field to a specific DB column. Set name, nullable, length.	Entity field

<code>@OneToMany</code>	Maps a one-to-many relationship between entities.	Entity field (collection side)
<code>@ManyToOne</code>	Maps the many side of a relationship (foreign key side).	Entity field
<code>@JoinColumn</code>	Specifies the foreign key column for a relationship.	With <code>@ManyToOne</code> / <code>@OneToOne</code>

Properties & Profiles

Annotation	What It Does	Where It Lives
<code>@Value</code>	Injects a value from application.properties into a field.	Any Spring-managed bean
<code>@ConfigurationProperties</code>	Binds a whole block of properties to a class at once.	Config / POJO classes
<code>@Profile</code>	Activates a bean only for a specific environment (dev, prod).	<code>@Component</code> , <code>@Bean</code> , <code>@Configuration</code>

One thing worth repeating: `@Service`, `@Repository`, and `@Component` are functionally identical at the Spring container level. The difference is semantic- they tell the next developer (and the interviewer) what role that class plays. Always pick the right one for the right layer.

Top 15 Spring Boot Interview Questions

These aren't exam questions. They're the actual things interviewers say out loud when they want to see if you know your stuff. Some are direct, some are sneaky. Read the answers, then practice saying them in your own words out loud. Seriously- out loud.

From Chapter 1- Spring Boot Basics

Q1. What is Spring Boot and how is it different from the Spring Framework?

A: Spring Framework is a vast set of modules- you wire everything manually. Spring Boot sits on top of it and adds auto-configuration, embedded servers, and starter dependencies. You write less config code and focus on the actual application. Think of Spring as the engine, Spring Boot as the car that comes pre-assembled.

Q2. What does `@SpringBootApplication` do internally?

A: It combines three annotations: `@SpringBootConfiguration` (marks it as a config class), `@EnableAutoConfiguration` (triggers auto-config based on classpath), and `@ComponentScan` (scans the package for beans). Remove any one of those and your app behaves differently.

Q3. What is auto-configuration in Spring Boot?

A: Spring Boot looks at the libraries present on your classpath and automatically configures beans you'd otherwise have to set up manually. Add `spring-boot-starter-data-jpa` and it auto-configures a `DataSource`, `EntityManagerFactory`, and transaction manager for you. You can override any of it in `application.properties`.

Q4. What is the difference between `@Component`, `@Service`, and `@Repository`?

A: All three register a class as a Spring bean. The difference is semantic. `@Component` is generic. `@Service` signals business logic. `@Repository` signals data access and additionally enables Spring's persistence exception translation- it converts low-level DB exceptions into Spring's `DataAccessException` hierarchy. Use the right annotation for the right layer.

Q5. What is the difference between `@Autowired` on a field vs. a constructor?

A: Field injection is convenient but makes testing harder- you can't inject a mock without Spring. Constructor injection makes dependencies explicit, works without Spring in unit tests, and is recommended by the Spring team. If you see `@RequiredArgsConstructor` from Lombok in a codebase, that's constructor injection with less boilerplate.

From Chapter 2- Spring Core Internals

Q6. What is the Spring IoC container?

A: IoC stands for Inversion of Control. Instead of your code creating objects, you hand that responsibility to the Spring container. The container creates, wires, and manages the lifecycle of beans. `ApplicationContext` is the interface that represents this container. Your job is just to define what you need- Spring figures out how to give it to you.

Q7. What is a Spring Bean? What are the different bean scopes?

A: A Spring Bean is any object managed by the Spring container. By default, beans are Singleton- one instance per container, shared everywhere. Prototype creates a new instance every time it's requested. There are also Request and Session scopes for web apps. Singleton is 99% of what you'll use in real projects.

Q8. What is the difference between `@Primary` and `@Qualifier`?

A: `@Primary` makes one bean the default when Spring finds multiple candidates for injection. `@Qualifier` lets you specify exactly which bean you want by name at the injection point. Use `@Primary` when one implementation should be the default, and `@Qualifier` when you need precise control at each injection site.

Q9. What does `@Value` do and when would you use `@ConfigurationProperties` instead?

A: `@Value` injects a single property: `@Value("${app.name}")`. It's fine for one or two values. `@ConfigurationProperties` maps a whole group of related properties into a POJO at once- cleaner, type-safe, and easier to validate. If you have more than 3 related settings, use `@ConfigurationProperties`.

Q10. What are Spring Profiles and why are they useful?

A: Profiles let you define different bean configurations or property files for different environments. You might have `application-dev.properties` for local development and `application-prod.properties` for production. Activate a profile with `spring.profiles.active=dev`. Beans annotated with `@Profile("dev")` only load in that environment. This keeps environment-specific config out of your main code.

From Chapter 3- Building the Web

Q11. What is REST and what does stateless mean?

A: REST is an architectural style for building APIs over HTTP. It uses standard HTTP methods, URLs as resource identifiers, and usually JSON for data exchange. Stateless means the server stores no client state between requests- every request must carry all the information needed to process it. This makes REST APIs scalable and easy to cache.

Q12. What is the difference between `@PathVariable` and `@RequestParam`?

A: `@PathVariable` extracts data from the URL path itself- like the 42 in `/users/42`. `@RequestParam` reads from the query string- like the page in `/products?page=2`. Use `@PathVariable` to identify a specific resource, `@RequestParam` for optional filters or pagination. They can be used together in the same method.

Q13. When would you use `ResponseEntity` instead of returning the object directly?

A: When you need control over the HTTP status code or headers. Returning an object directly always gives 200 OK. With `ResponseEntity` you can return 201 for creation, 204 for deletion, 404 when something doesn't exist, or attach a Location header. It's cleaner API design and makes your intent clear to whoever consumes your API.

Q14. What is the difference between PUT and PATCH?

A: PUT replaces the entire resource. If you send only the name field in a PUT, the email and phone fields may get overwritten with null. PATCH does a partial update- only the fields you include in the request are changed. In practice, PATCH is safer for update operations on large objects.

Q15. Walk me through what happens when a POST request hits your Spring Boot application.

A: The request hits the embedded Tomcat server, which passes it to Spring's DispatcherServlet. DispatcherServlet finds the right controller method based on the URL and HTTP method. @RequestBody deserializes the JSON body into a Java object. The controller calls the service layer, which runs the business logic and calls the repository to persist data. The repository talks to the database via JPA. The result travels back up the chain, gets serialized into JSON by Jackson, and is returned as an HTTP response with the appropriate status code.

One Last Thing

Interviewers aren't just checking whether you know these answers. They're watching how you think. If you don't know something, say "I haven't used that in a project yet, but here's how I understand it..." and then explain what you do know. Confidence under uncertainty is a skill too.

You've read the book. You've done the prep. Now go get that offer.

The Spring Boot Playbook: From First API to Hired Developer | Extra Value Section

CHAPTER 3

Building the Web

APIs, HTTP, and everything you need to build your first real Spring Boot web layer

Let's be honest. The reason most of us got into backend development is this- someone hits a URL, our code runs, and something useful comes back. That loop is everything. And Spring Boot makes it stupidly easy.

But "easy to use" doesn't mean "easy to explain in an interview." Interviewers love this chapter. REST, HTTP methods, controllers, request params- they ask all of it. So let's build it up properly, from the ground.

1. What Is a REST API?

Think of a waiter at a restaurant. You don't go into the kitchen yourself. You tell the waiter what you want, the kitchen prepares it, and the waiter brings it back. The waiter is the API- a middleman that follows certain rules about how to take your request and return a response.

REST stands for Representational State Transfer. Don't worry about the full form- nobody asks you to spell it in interviews. What matters is the idea.

REST is just a set of rules for how two systems talk to each other over HTTP. The client (a browser, mobile app, Postman) sends a request. The server receives it, does something, and sends back a response. Simple.

What Makes an API "RESTful"?

- It uses HTTP as the protocol
- Resources are represented as URLs- /users, /orders/5, /products
- It uses HTTP methods to indicate the action (GET to fetch, POST to create, etc.)
- Responses are usually in JSON format
- Each request is independent- the server doesn't remember your previous request

That last point is called statelessness. It means every request must carry all the information the server needs. The server doesn't maintain a session for you by default. This is one of those things interviewers slip in casually- "What does stateless mean in REST?"- and a lot of freshers freeze.

Your answer: "In REST, the server does not store any client state between requests. Each request must be self-contained and carry all the information needed to process it."



Interview Question

What is REST? How is it different from SOAP?

How to answer:

REST is an architectural style for designing networked applications using HTTP. It's stateless, uses standard HTTP methods, and returns data in JSON or XML. SOAP is a protocol with strict standards, XML-only, and more overhead. REST is lightweight and preferred for modern web APIs. Most companies use REST today.

2. The Flow: Client → Controller → Service → Repository

Before we write a single annotation, you need this picture in your head. When a request hits your Spring Boot app, here's what happens:



Each layer has a job. The Controller's job is to receive requests and return responses- nothing else. The moment you start writing business logic inside a controller method, you're doing it wrong.

The Service layer holds all the actual logic. Calculation, validation, calling external APIs- that all goes here. The Repository is basically your database access layer. In Spring, we usually use Spring Data JPA repositories which give us methods like `save()`, `findById()`, `deleteById()` out of the box.

This separation isn't just "good practice." It's what makes your code testable. You can test the Service layer without a database. You can test the Controller layer without real business logic. Interviewers absolutely love when you mention testability.



Tip

If an interviewer asks 'why do we separate Controller and Service?', don't just say 'clean code'.

Say: 'It makes each layer independently testable, reduces coupling, and follows the Single Responsibility Principle.'

That answer will land well every single time.

3. HTTP Methods- GET, POST, PUT, DELETE

HTTP methods are how you tell the server what you want to do. Think of a library. GET is browsing books. POST is donating a new book. PUT is replacing a damaged book with a new copy. DELETE is removing a book permanently.

Method	Purpose
GET	Fetch data. Should never change anything on the server.
POST	Create a new resource. Sends data in the request body.
PUT	Update an existing resource. Replaces the whole thing.
PATCH	Partially update a resource. Just change what you specify.
DELETE	Remove a resource from the server.

One common confusion: PUT vs PATCH. PUT replaces the entire resource. If you PUT a user object with only the name field, everything else (email, phone) gets wiped. PATCH is gentler- it only updates what you send. Spring Boot interviews often test this.

GET is safe and idempotent. Safe means it doesn't modify data. Idempotent means calling it 10 times gives the same result as calling it once. POST is neither. DELETE is idempotent (deleting something twice gives the same end state- it's gone). These are the kinds of details that separate a prepared candidate from the rest.



Interview Question

What is the difference between PUT and PATCH?

How to answer:

PUT replaces the entire resource. If you omit fields, they may get overwritten with nulls. PATCH does a partial update- only the fields you send are changed. In Spring Boot, both are valid endpoints, but PATCH is preferred when updating specific fields of a large object.

4. HTTP Status Codes- What Your API Says Back

Every HTTP response has a status code. It tells the client whether things went well or what went wrong. There's a whole list of them, but you need to know the important ones cold.

Code	Meaning
200 OK	Everything worked. You're returning data.
201 Created	A new resource was successfully created (after POST).
204 No Content	Success, but nothing to return (like after DELETE).

400 Bad Request	Client sent invalid data. Their fault, not yours.
401 Unauthorized	User is not authenticated (not logged in).
403 Forbidden	User is authenticated but doesn't have permission.
404 Not Found	The resource doesn't exist.
500 Internal Server Error	Something broke on the server side. Your fault.

A mistake I see all the time: returning 200 OK for everything, even when there's an error. "Resource not found? 200. User not logged in? 200 with an error message in the body." Please don't do this. HTTP status codes exist for a reason.

Also worth knowing: 401 vs 403. 401 means the user is not logged in- we don't know who they are. 403 means we know who they are, but they're not allowed to do this. Different problems, different codes.

⚠ Common Mistake

Don't return a 200 status code with a body like { "error": "User not found" }.

That's confusing to frontend developers and breaks API contracts.

Return 404 with an error message in the body instead. The status code should always reflect the actual outcome.

5. @RestController vs @Controller

Here's a question that comes up in almost every Spring Boot interview: what's the difference between @Controller and @RestController?

@Controller is the older annotation. It tells Spring this class handles web requests. But by default, it expects to return a view name- like an HTML page. So if you return "home" from a method, Spring looks for a template called home.html.

@RestController = @Controller + @ResponseBody. That @ResponseBody part is the key. It tells Spring to write the return value directly into the HTTP response body as JSON- not to look for a view.

```
// This tries to find a view called 'home.html'
@Controller
public class PageController {
    @GetMapping("/home")
    public String home() {
        return "home"; // looks for home.html template
    }
}

// This returns JSON directly
@RestController
public class ApiController {
```

```
@GetMapping("/user")
public User getUser() {
    return new User("Rahul", "rahul@email.com"); // → JSON response
}
```

For REST APIs- which is what you'll be building 99% of the time- use `@RestController`. If you're building a server-side rendered web app with Thymeleaf or JSP, use `@Controller`. Chances are you'll mostly deal with REST.

Interview Question

What is the difference between `@Controller` and `@RestController`?

How to answer:

`@Controller` is used for MVC applications where methods return view names. `@RestController` is a shortcut for `@Controller` + `@ResponseBody`. It's used for REST APIs where each method returns data (JSON/XML) directly in the response body. For modern Spring Boot REST APIs, `@RestController` is almost always the right choice.

6. @RequestMapping- The Parent of All Mappings

Before `@GetMapping` and `@PostMapping` existed, everything was `@RequestMapping`. You'd specify the path and the HTTP method together. It still works, and you'll see it a lot in older codebases.

```
@RestController
@RequestMapping("/api/users") // base path for all methods below
public class UserController {

    @RequestMapping(method = RequestMethod.GET) // old style
    public List<User> getAll() { ... }

    @GetMapping // modern style- same thing, shorter
    public List<User> getAllUsers() { ... }

}
```

The most useful thing about `@RequestMapping` at the class level is grouping. When you put `@RequestMapping("/api/products")` on the class, every method inside automatically starts with `/api/products`. So if a method has `@GetMapping("/search")`, the full URL becomes `/api/products/search`. Neat.

You can also specify multiple paths: `@RequestMapping({" /home", "/index"})`- both URLs map to the same method. Rarely used but good to know.

7. @GetMapping, @PostMapping, @PutMapping, @DeleteMapping

These are just shortcuts. Spring introduced them to make code cleaner. Each one is a `@RequestMapping` with a method pre-set. Let's write a full CRUD controller so you can see all four in action.

```
@RestController
@RequestMapping("/api/products")
public class ProductController {

    // GET /api/products
    @GetMapping
    public List<Product> getAll() {
        return productService.getAll();
    }

    // GET /api/products/5
    @GetMapping("/{id}")
    public Product getById(@PathVariable Long id) {
        return productService.getById(id);
    }

    // POST /api/products
    @PostMapping
    public ResponseEntity<Product> create(@RequestBody Product product) {
        Product saved = productService.save(product);
        return ResponseEntity.status(201).body(saved);
    }

    // PUT /api/products/5
    @PutMapping("/{id}")
    public Product update(@PathVariable Long id,
        @RequestBody Product product) {
        return productService.update(id, product);
    }

    // DELETE /api/products/5
    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable Long id) {
        productService.delete(id);
        return ResponseEntity.noContent().build(); // 204
    }
}
```

Read that code carefully. This is what a real CRUD controller looks like. Practice writing this from scratch without looking at notes. If you can do that in an interview, you're already ahead of most candidates.

Notice the DELETE method returns `ResponseEntity<Void>` with a 204 No Content. That's the correct way. Some people return a string like "Deleted successfully"- that's not wrong, but it's not standard either.

8. @PathVariable vs @RequestParam

This is probably the most common question in Spring Boot fresher interviews. And the confusion is real- both are used to extract data from a URL. The difference is where in the URL they live.

@PathVariable

Used when the value is part of the URL path itself. Like /users/42- the 42 is in the path.

```
// URL: GET /users/42
@GetMapping("/users/{id}")
public User getUser(@PathVariable Long id) {
    return userService.findById(id);
}
```

@RequestParam

Used when the value comes as a query parameter after a ? in the URL. Like /products?category=electronics&sort=price.

```
// URL: GET /products?category=electronics&sort=price
@GetMapping("/products")
public List<Product> getProducts(
    @RequestParam String category,
    @RequestParam(defaultValue = "name") String sort) {
    return productService.getFiltered(category, sort);
}
```

The defaultValue attribute is useful- if the parameter isn't sent, it falls back to that value instead of throwing an error. You can also set required = false to make it optional.

@PathVariable	@RequestParam
Part of the URL path: /users/{id}	Query string: /users?name=rahul
Always required (by design)	Can be optional with required=false
Used to identify a specific resource	Used for filtering, sorting, pagination
Example: GET /orders/101	Example: GET /orders?status=pending

The quick rule: if it identifies the resource, it's @PathVariable. If it filters or modifies the response, it's @RequestParam.

 Interview Question

When would you use `@PathVariable` vs `@RequestParam`?

How to answer:

Use `@PathVariable` when the parameter is part of the resource identity- like `/users/{id}` or `/orders/{orderId}`.
Use `@RequestParam` for optional data like filters, search terms, or pagination- like `/products?category=shoes&page=2`. The URL structure itself hints at which one to use.

9. `@RequestBody`- Reading What the Client Sends

When a client sends a POST or PUT request, they usually include data in the request body as JSON. `@RequestBody` tells Spring to take that JSON and convert it into a Java object automatically.

This automatic conversion is done by Jackson- a JSON library that Spring Boot includes by default. You don't configure it. It just works.

```
// Client sends this JSON:
// { "name": "Laptop", "price": 75000 }

@PostMapping("/products")
public ResponseEntity<Product> create(@RequestBody Product product) {
    // 'product' is already a Java object here
    // product.getName() → "Laptop"
    // product.getPrice() → 75000
    Product saved = productService.save(product);
    return ResponseEntity.status(201).body(saved);
}
```

The field names in the JSON must match the field names in your Java class. If the JSON has "productName" but your class has "name", Jackson won't map it. You'd either rename the field or use `@JsonProperty("productName")` on the Java field.

Common Mistake

Using `@RequestBody` on a GET request. GET requests don't have a body- or at least, they shouldn't. If you need to send data with GET, use `@RequestParam` instead.
Some frameworks allow body in GET technically, but it's bad practice and most clients won't send one.

Also- if the incoming JSON doesn't match the expected format, Spring throws a 400 Bad Request automatically. You don't need to handle that manually.

10. `ResponseEntity`- Full Control Over Your Response

Most of the time, returning a plain Java object from a controller method is fine. Spring wraps it in a 200 OK response automatically. But sometimes you need more control- you want to set a specific status code, or add custom headers.

That's what `ResponseEntity` is for. It represents the entire HTTP response- body, status code, and headers.

```
// Simple return- Spring sets 200 OK automatically
@GetMapping("/user/{id}")
public User getUser(@PathVariable Long id) {
    return userService.findById(id);
}

// ResponseEntity- you control the status code
@GetMapping("/user/{id}")
public ResponseEntity<User> getUser(@PathVariable Long id) {
    User user = userService.findById(id);

    if (user == null) {
        return ResponseEntity.status(404).build(); // no body
    }

    return ResponseEntity.ok(user); // 200 with body
}

// Creating a resource- return 201 with Location header
@PostMapping("/users")
public ResponseEntity<User> createUser(@RequestBody User user) {
    User saved = userService.save(user);
    return ResponseEntity
        .status(201)
        .header("Location", "/users/" + saved.getId())
        .body(saved);
}
```

`ResponseEntity.ok()` is a shortcut for status 200 with a body. `ResponseEntity.noContent().build()` gives you 204 with no body. `ResponseEntity.notFound().build()` gives 404. Learn these shortcuts- they're cleaner than writing `.status(200)` every time.

Why Does This Matter in an Interview?

When you return a plain object, you give up control. If the user doesn't exist, you'd have to throw an exception. With `ResponseEntity`, you can handle it gracefully right in the method and return the appropriate status code. Both approaches have their place, but understanding `ResponseEntity` shows you understand HTTP properly.



Interview Question

Why would you use `ResponseEntity` instead of returning an object directly?

How to answer:

ResponseEntity gives you full control over the HTTP response- status code, headers, and body. Returning a plain object is simpler but Spring always sends 200 OK. With ResponseEntity, I can return 201 for creation, 404 when a resource isn't found, or add custom headers like Location. It's especially useful when you need to communicate the outcome clearly through HTTP semantics.

Chapter Summary

We covered a lot of ground here. Let's quickly close the loop.

- REST is a style for building APIs over HTTP- stateless, resource-based, JSON-friendly
- The request flows: Client → Controller → Service → Repository- each layer has one job
- HTTP methods (GET, POST, PUT, DELETE) tell the server what action to perform
- HTTP status codes communicate the outcome- use them correctly, don't always return 200
- @RestController = @Controller + @ResponseBody- use it for REST APIs
- @RequestMapping sets the base path; @GetMapping, @PostMapping etc. are method-specific shortcuts
- @PathVariable extracts from the URL path; @RequestParam reads query parameters
- @RequestBody deserializes the JSON request body into a Java object automatically
- ResponseEntity gives you full control over status codes, headers, and the response body

**Before Your Interview**

Write a simple ProductController from scratch- no IDE hints, no copy-paste.

It should have GET all, GET by ID, POST, PUT, and DELETE methods.

Use ResponseEntity where appropriate, include @PathVariable and @RequestParam in different endpoints.

If you can write that cleanly, you're ready for 80% of Spring Boot interview questions on this chapter.

CHAPTER 4

Database & JPA Foundation

Where your data actually lives - and how Spring Boot talks to it without driving you mad.

Every application you build eventually needs to store something. A user. An order. A product. Whether it's MySQL or PostgreSQL, the database is where real work happens. This chapter is about how Spring Boot connects to a database - and how JPA makes the whole thing a lot less painful than you'd expect.

We're going to keep this grounded. No massive theory dumps. Just the things you genuinely need to understand - and explain - in an interview.

1. What Is JPA?

Start with this: JPA is not a library you download. It's a **specification**. A set of rules that says, "here's how Java objects should map to database tables, and here's the API you should follow." JPA itself doesn't do the actual work - something else does. We'll get to that in a second.

JPA stands for **Jakarta Persistence API** (you'll still see it called Java Persistence API in older resources - same thing). Think of it like a contract. It defines interfaces like `EntityManager`, annotations like `@Entity`, `@Id`, `@Column` - and any ORM framework that wants to be JPA-compliant has to implement these.

JPA is the "what to do". Hibernate is the "how to do it".

So why does this matter in an interview? Because interviewers often ask, "Are JPA and Hibernate the same thing?" - and a lot of freshers say yes. They're not. JPA is the specification. Hibernate is the most popular implementation of that specification. Spring Boot uses Hibernate under the hood by default, but you interact with it through JPA's interfaces and annotations.

INTERVIEW QUESTION

What is JPA and how is it different from Hibernate?

How to answer: JPA is a specification - a set of rules and interfaces defined by Jakarta EE for mapping Java objects to relational databases. Hibernate is the most popular implementation of JPA. When you write code against JPA annotations and interfaces, the actual database operations are handled by Hibernate. Spring Boot uses Hibernate as the default JPA provider out of the box.

2. Hibernate - The One Doing the Actual Work

Hibernate is basically a translator at a business meeting. You speak in Java - objects, fields, methods. The database speaks in SQL - tables, rows, columns. Hibernate sits in the middle and translates between the two.

Hibernate is an ORM framework. ORM stands for **Object-Relational Mapping**. The whole idea is simple: you define a Java class, annotate it properly, and Hibernate figures out how to store it in a database table. It generates the SQL. It runs the queries. You mostly don't write SQL at all.

When you add Spring Data JPA to your project, Hibernate gets included automatically. You don't configure it manually - Spring Boot sets it up for you. That's one of the nice things about the whole ecosystem.

TIP

In most Spring Boot interviews, when they say 'JPA', they're really asking about the combination of JPA + Hibernate working together.

You don't need to explain the internals of Hibernate - just show you understand the relationship: JPA is the standard, Hibernate is the engine that powers it.

3. Why Do We Even Use ORM?

Before ORM, developers wrote plain JDBC code. You'd manually create a connection, write SQL strings, execute them, loop through ResultSets, and map each column to a Java field by hand. It worked - but it was tedious, error-prone, and full of boilerplate.

Here's a rough idea of what that looked like:

```
// Without ORM - old school JDBC
Connection conn = DriverManager.getConnection(url, user, pass);
PreparedStatement ps = conn.prepareStatement(
    "SELECT * FROM users WHERE id = ?"
);
ps.setLong(1, userId);
ResultSet rs = ps.executeQuery();
if (rs.next()) {
    User user = new User();
    user.setId(rs.getLong("id"));
    user.setName(rs.getString("name"));
    user.setEmail(rs.getString("email"));
}
// ... close everything manually. Don't forget or you'll leak connections.
```

Now compare that to what JPA gives you:

```
// With JPA + Spring Data
```

```
User user = userRepository.findById(userId).orElseThrow();
// That's it. One line.
```

Same result. A hundred fewer lines. No SQL string. No manual mapping. The trade-off is you need to understand what JPA is doing behind the scenes - which is exactly what this chapter covers.

ORM doesn't remove SQL from your life completely. It hides it. For complex queries, you still need to understand SQL - JPA just handles the boring CRUD for you.

4. @Entity - Telling JPA This Class Is a Table

The moment you put `@Entity` on a class, you're saying: "Hey Hibernate, this Java class maps to a database table." That's it. Every field in the class maps to a column. Every instance of the class maps to a row.

Here is a simple one:

```
import jakarta.persistence.*;

@Entity
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    private double price;

    // Getters and setters
}
```

By default, Hibernate maps this to a table called `product` (or `Product` depending on the naming strategy). The field `name` maps to a column called `name`. Clean and straightforward.

One thing beginners miss: **every @Entity class must have a no-arg constructor**. It can be private or protected, but it must exist. JPA needs it to create instances when loading data from the database. If you're using Lombok's `@AllArgsConstructor`, you have to also add `@NoArgsConstructor` manually or things break at runtime in a slightly confusing way.

5. @Table - When the Default Name Doesn't Work

Most of the time you don't need `@Table`. Hibernate uses the class name as the table name by default. But sometimes - especially when you're connecting Spring Boot to an existing database someone else designed - the table is called something like `tbl_products` or `PRODUCT_MASTER`. That's when `@Table` becomes useful.

```
@Entity
@Table(name = "tbl_products") // maps to this exact table name in the DB
public class Product {
    // ...
}
```

You can also specify the schema: `@Table(name = "products", schema = "inventory")`. Useful in large databases where tables are organized under different schemas. But for most projects and for most interviews, just knowing the `name` attribute is more than enough.

6. @Id and @GeneratedValue - The Primary Key Story

Every entity needs a primary key. That's not a JPA rule - that's a database rule. `@Id` tells JPA which field is the primary key. Simple enough.

The interesting part is `@GeneratedValue`. This tells JPA how the primary key value should be generated. You don't want to assign IDs manually - that's a recipe for conflicts. You let the database or JPA handle it. There are a few strategies:

Strategy	What Actually Happens
<code>GenerationType.AUTO</code>	JPA picks the strategy based on the database. Default option but unpredictable across DBs.
<code>GenerationType.IDENTITY</code>	Uses the database's auto-increment. Most common choice for MySQL and PostgreSQL.
<code>GenerationType.SEQUENCE</code>	Uses a DB sequence object. Common in Oracle and PostgreSQL for performance reasons.
<code>GenerationType.TABLE</code>	Uses a separate table to track ID values. Rarely used. Slow. Avoid in most cases.

For most beginner and intermediate projects - especially ones using MySQL - you'll see `GenerationType.IDENTITY`. This tells the database to handle the auto-increment. When you insert a row, the DB gives it the next available ID, and Hibernate fetches that ID back into your Java object.

```
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
private Long id;

// After calling repository.save(product),
// product.getId() will return the auto-generated value from the DB.
```

A mistake that trips people up: using `GenerationType.AUTO` without knowing what database they're connecting to. With some databases it works fine. With others it tries to create a sequence table and fails. Just use `IDENTITY` for MySQL and you'll be fine 90% of the time.

INTERVIEW QUESTION

What is the difference between `GenerationType.AUTO` and `GenerationType.IDENTITY`?

How to answer: AUTO lets JPA pick the strategy based on the database. It's flexible but less predictable. IDENTITY delegates ID generation to the database's auto-increment feature - which is what MySQL and PostgreSQL support natively. In most Spring Boot projects with MySQL, IDENTITY is the right choice. It's explicit, reliable, and maps directly to how the DB column is defined.

7. Entity Lifecycle - Keep It Simple for Now

This is a topic that can go very deep - but for fresher-level interviews, you just need the basic picture. A JPA entity can be in one of four states at any given time:

- **Transient:** The object exists in Java memory but JPA doesn't know about it. You just did `new Product()` and haven't saved it yet.
- **Managed (Persistent):** JPA is tracking this object. Any changes you make to it will automatically be saved to the database when the transaction commits. This is the state after `entityManager.persist()` or after loading from a repository.
- **Detached:** The object was managed, but the persistence context closed (session ended). Changes to it won't be saved automatically.
- **Removed:** Marked for deletion. Will be deleted from the database when the transaction commits.

In everyday Spring Boot development with Spring Data JPA repositories, you rarely interact with these states directly. But interviewers do ask about them. The one state that trips people up is Managed - because it has a side effect that isn't obvious: if you load an entity, change one of its fields, and then do nothing else, JPA still saves the change to the database when the transaction ends. That's called **dirty checking**.

TIP

If an interviewer asks 'what is dirty checking in JPA?' - say this:

JPA's persistence context tracks all managed entities. Before a transaction commits, it compares the current state of each entity to its original snapshot. If anything changed, it generates an UPDATE query automatically. You never called `save()` - it just happened.

8. Connecting Spring Boot to a Database

Here's where things get practical. You've got a Spring Boot app and a MySQL database running. How do you tell Spring Boot to connect to it? Through `application.properties`.

There's no code to write for this. Just a few lines of configuration, and Spring Boot handles the rest - creates a connection pool, sets up Hibernate, wires it all together.

The Basic Setup

```
# Database connection
spring.datasource.url=jdbc:mysql://localhost:3306/myappdb
spring.datasource.username=root
spring.datasource.password=yourpassword
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

# JPA / Hibernate settings
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
```

Worth walking through what each line does - because the `ddl-auto` property is the one that confuses people the most, and it's also the one that can silently destroy your data if you use it wrong.

Understanding ddl-auto

Value	What It Does - and When to Use It
<code>none</code>	Hibernate does nothing to the schema. You manage it yourself. Use this in production.
<code>validate</code>	Checks that the schema matches your entities. Throws an error if it doesn't. Safe for production.
<code>update</code>	Adds new columns/tables as you add new fields. Never drops anything. Good for development.
<code>create</code>	Drops and recreates all tables on every startup. You lose all data. Development only.
<code>create-drop</code>	Like create, but also drops everything when the app shuts down. Testing only.

WATCH OUT

Never use create or create-drop in production. You will lose all your data on every restart.

A common mistake: someone sets `ddl-auto=create` during development and forgets to change it before deploying.

The safe production setup is `ddl-auto=none` or `ddl-auto=validate` with a proper database migration tool like Flyway or Liquibase.

The `show-sql=true` property makes Hibernate print every SQL query it runs to the console. This is really useful during development - you can see exactly what's going into the database. Turn it off in production though. Logging every query at scale is a performance hit.

9. How It All Connects - The Full Picture

Here is the full picture. When your Spring Boot app starts up and you have JPA configured, this is the sequence:

1. Spring Boot reads `application.properties`
→ Gets the DB URL, username, password
2. Spring Boot creates a `DataSource` (connection pool via HikariCP by default)
→ Manages a pool of database connections so we're not creating one per request
3. Hibernate scans for `@Entity` classes
→ Finds `Product`, `User`, `Order` etc. and maps them to tables
4. Hibernate applies `ddl-auto` strategy
→ Creates, updates, or validates the schema
5. Spring Data JPA creates Repository implementations
→ `UserRepository`, `ProductRepository` etc. are ready to use
6. Your app is up. Requests come in, repositories do their job.

You never wrote a line of JDBC. You never manually created a connection. The Spring Boot starter dependency and about 10 lines in `application.properties` handled all of it. That's the whole point.

INTERVIEW QUESTION

How does Spring Boot connect to a database? Walk me through it.

How to answer: Spring Boot reads the datasource configuration from `application.properties` - the URL, credentials, and driver. It creates a `DataSource` bean backed by HikariCP as the connection pool. Hibernate then scans for `@Entity` classes and maps them to database tables. Spring Data JPA generates repository implementations on top of that. By the time the app finishes starting, you have a fully wired data layer - no manual JDBC or connection code required.

10. Putting It Together - A Working Entity

Here is a proper entity with everything we have covered. This is the kind of code you would write in a real project and also the kind of thing an interviewer might ask you to write on a whiteboard.

```
package com.example.app.entity;

import jakarta.persistence.*;

@Entity
@Table(name = "products")
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "product_name", nullable = false, length = 100)
    private String name;

    @Column(nullable = false)
    private double price;

    @Column(name = "in_stock")
    private boolean inStock;

    // JPA requires a no-arg constructor
    public Product() {}

    public Product(String name, double price, boolean inStock) {
        this.name = name;
        this.price = price;
        this.inStock = inStock;
    }

    // Getters and setters...
}
```

A few things worth noting here. The `@Column` annotation lets you customize the column - set `nullable = false` to add a NOT NULL constraint, `length` to set a VARCHAR size, and `name` to override the column name. These are all optional - if you skip `@Column` entirely, Hibernate uses sensible defaults.

Notice we have both a no-arg constructor and a parameterized one. The no-arg is for JPA. The parameterized one is for you - so you can create `new Product("Laptop", 75000, true)` instead of setting each field one by one.

WATCH OUT

Senior Dev Warning: Never use `@Data` on a `@Entity` class.

This is one of those mistakes that doesn't blow up immediately. `@Data` generates `toString()`, `equals()`, and `hashCode()` methods that traverse all fields - including your relationship fields like `@OneToMany` collections.

The moment you have two related entities pointing at each other, those generated methods will call each other in an infinite loop and crash your application with a `StackOverflowError`.

On JPA entities, use `@Getter` and `@Setter` only. Nothing else.

For constructors, add `@NoArgsConstructor` for JPA and `@AllArgsConstructor` if you need it. That is the correct Lombok setup for entities.

Chapter Summary

Quick recap before you move on:

- **JPA** is the specification. **Hibernate** is the implementation that does the real work.
- **ORM** maps Java objects to database tables - saving you from writing repetitive JDBC boilerplate.
- `@Entity` marks a class as a database table. Requires a no-arg constructor.
- `@Table` lets you override the table name - useful when connecting to existing databases.
- `@Id` marks the primary key. `@GeneratedValue(strategy = GenerationType.IDENTITY)` tells the DB to auto-generate it.
- Entity lifecycle has four states: **Transient**, **Managed**, **Detached**, **Removed**. The key one to know is **Managed** - JPA tracks it and saves changes automatically (dirty checking).
- `application.properties` is where you put your DB URL, credentials, and Hibernate settings like `ddl-auto` and `show-sql`.
- `ddl-auto=update` for development. `ddl-auto=none` or `validate` for production. Never `create` in production.

BEFORE THE NEXT CHAPTER

Set up a local MySQL database, create a Spring Boot project with `spring-boot-starter-data-jpa` and `mysql-connector-j`, and try to get one entity to map to a table. Use `ddl-auto=create-drop` so you can restart cleanly. Once the table appears in your database client when you run the app - you've got it. That small win is worth a hundred theory readings.

Senior Insights (What Most Freshers Miss)

These are the three JPA concepts that separate someone who read the docs from someone who has actually built with it.

1. Persistence Context

Every time a request runs inside a transaction, JPA opens a Persistence Context. Think of it as a short-lived cache that holds every entity you load or save during that transaction.

While the context is open, JPA tracks every managed entity. Before the transaction commits, it compares the current state of each entity to the snapshot it took at load time. If anything changed, it fires an UPDATE automatically - that is dirty checking. No explicit `save()` needed.

The Persistence Context closes when the transaction ends. After that, entities become detached and JPA no longer tracks them.

2. LazyInitializationException

If a field is lazily loaded, JPA only fetches it when your code actually touches it - and only if the session is still open. Access that field after the transaction has closed and you get a `LazyInitializationException`.

This is one of the most common runtime errors in real Spring Boot projects. It usually happens when you load an entity in a service method, close the transaction, and then try to access a lazy collection in the controller or view layer.



Fix

Either keep access inside the transaction, or use `@EntityGraph` or `JOIN FETCH` to eagerly load what you need upfront. Avoid `spring.jpa.open-in-view=true` - it hides the problem instead of solving it.

3. The N+1 Query Problem

You load 10 users. For each user, JPA lazily fetches their orders. That is 1 query to get the users, then 10 more queries to get each user's orders. 11 queries total instead of 1. With 100 users it becomes 101 queries.

This happens silently. Your code looks clean. The SQL log (`show-sql=true`) tells the real story. Under any real load, this kind of query pattern will be noticeable.

The fix: use `JOIN FETCH` in a JPQL query or annotate your repository method with `@EntityGraph`. Both tell JPA to load the users and their orders in a single query.

```
// N+1 problem - one extra query per user
List<User> users = userRepository.findAll();
users.forEach(u -> System.out.println(u.getOrders())); // triggers N queries

// Fix with JOIN FETCH
@EntityGraph(attributePaths = {"orders"})
@Query("SELECT u FROM User u JOIN FETCH u.orders")
List<User> findAllWithOrders();
```

CHAPTER 5

Repository Layer & Data Access

The layer where your code actually touches the database, and where a lot of things can quietly go wrong.

By now you have your entity set up. Your database is configured. But nothing has actually read from or written to the database yet. That's what the repository layer does.

This is the part of the code that talks to the database. Spring Boot makes it so easy that beginners sometimes wonder if they're missing something. You're not. You write an interface, and Spring Boot generates the full implementation at runtime.

What Is JpaRepository?

You know how a TV remote comes pre-loaded with buttons, volume, channels, power, and you don't have to wire any of them yourself? JpaRepository is basically that remote, but for your database. The standard database operations are already there, ready to go.

JpaRepository is an interface from Spring Data JPA. When you extend it for your entity, you get a full set of CRUD operations without writing a single line of implementation code. Spring generates everything at startup, behind the scenes.

```
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface ProductRepository extends JpaRepository<Product, Long> {
    // Nothing here yet. And yet - it already works.
}
```

Two things in that generic signature matter. The first type parameter is the entity class, so Spring knows which table you're working with. The second is the type of the primary key, Long in this case, because we annotated the id field with @Id and it's of type Long.

Get those two wrong and you'll get confusing compile errors. The entity type must match your @Entity class, and the ID type must match whatever you used on the @Id field. It's an easy mistake to make.

If they ask...

...what JpaRepository gives you out of the box, keep it concrete. You get CRUD methods like `save()`, `findById()`, `findAll()`, `deleteById()` and more, generated at runtime by Spring, no SQL needed. Mention the two type parameters: the entity class and its primary key type. That's the full answer.

How Spring Generates the Implementation

This one surprises a lot of people. Your repository is just an interface, no class, no implementation, no SQL. So how does it actually work?

When Spring Boot starts up, it scans your project for interfaces that extend JpaRepository (or CrudRepository, PagingAndSortingRepository, etc.). For each one it finds, it creates a proxy class at runtime. A dynamically generated implementation that uses Hibernate under the hood to run the actual database operations.

You never see this class. It lives in memory. When you call `productRepository.findAll()`, that call goes into the proxy, which calls Hibernate, which runs `SELECT * FROM products`, maps the rows to Product objects, and returns them.

You write the interface. Spring writes the class. Hibernate writes the SQL. You just call the method.

This pattern is called the Spring Data Repository abstraction. Interviewers like to check whether you actually understand this, or whether you just use it blindly.

Worth knowing for interviews

If someone asks how Spring Data JPA works internally, bring up the proxy pattern. Spring scans for repository interfaces at startup and creates dynamic proxy implementations using reflection. Those proxies delegate to Hibernate for the actual database operations. That answer will set you apart from candidates who just say 'it gives you CRUD methods'.

The Basic CRUD Methods

Four methods will cover maybe 70% of what you do with a repository in most applications. Worth knowing cold.

`save(entity)` is the one you'll use most. It inserts when the entity has no ID, and updates when it does. `findById(id)` fetches a single row by primary key and gives you back an `Optional<T>`, more on that in the next section. `findAll()` returns every row in the table as a `List<T>`, which is fine for small datasets but something to be careful with as data grows. `deleteById(id)` removes the row with that primary key and throws an exception if it doesn't exist.

There's a catch with `save()` that trips people up. If the entity has no ID, Spring treats it as new and does an INSERT. If it already has an ID, Spring does an UPDATE. So if you accidentally assign an ID before

saving, Spring will try to update a row that might not exist. Depending on your setup, it might fail silently or throw an error.

```
// INSERT - id is null, Spring calls persist()
Product p = new Product("Laptop", 75000, true);
productRepository.save(p); // inserts new row

// UPDATE - id is set, Spring calls merge()
p.setPrice(70000);
productRepository.save(p); // updates existing row

// DELETE
productRepository.deleteById(5L);

// GET ALL
List<Product> all = productRepository.findAll();
```

Optional and the findById Return Type

Before Optional existed in Java, finding something by ID was risky. The method returned either an object or null. Developers forgot to check. Code crashed with NullPointerExceptions. It happened constantly.

Java 8 introduced Optional<T> to fix this. Instead of returning a Product or null, findById() returns an Optional<Product>, a container that either holds the product or is empty. It forces you to think about the case where nothing was found.

```
Optional<Product> result = productRepository.findById(42L);

// Option 1 - safe check before use
if (result.isPresent()) {
    Product p = result.get();
    System.out.println(p.getName());
}

// Option 2 - throw a meaningful exception if not found
Product p = result.orElseThrow(() ->
    new RuntimeException("Product not found with id: 42")
);

// Option 3 - provide a default value if not found
Product p = result.orElse(new Product());
```

In real projects, `orElseThrow()` is the most common pattern. You throw something specific like `ProductNotFoundException`, catch it with a global exception handler, and return a clean 404 response. That's the production-ready approach.

Common mistake

Calling `.get()` directly without checking first: `Optional<Product> result = productRepository.findById(99L); Product p = result.get();` // throws `NoSuchElementException` if 99 doesn't exist This is exactly the `NullPointerException` problem `Optional` was designed to prevent. You just brought it back in a different form. Use `orElseThrow()`, `orElse()`, or check `isPresent()` before calling `get()`.

If the question comes up

...why `findById` returns `Optional` instead of the entity directly: because the record might not exist. `Optional` forces the caller to handle both cases. In practice, `orElseThrow()` is how most people handle it. You throw a meaningful exception, catch it somewhere central, and return a proper error response.

Custom Queries with Method Names

Here's something that genuinely feels like magic the first time you see it. You need to find a user by their email. You don't write SQL. You don't write any implementation. You just declare a method on your repository with the right name, and Spring figures out the query itself.

```
public interface UserRepository extends JpaRepository<User, Long> {

    // Spring generates: SELECT * FROM users WHERE email = ?
    User findByEmail(String email);

    // Returns a list when multiple results are possible
    List<User> findByName(String name);

    // Combine conditions
    List<User> findByNameAndCity(String name, String city);

    // Partial match
    List<User> findByNameContaining(String keyword);

    // Sort the result
    List<User> findByActiveOrderByNameAsc(boolean active);
}
```

Spring Data JPA parses these method names at startup. It reads keywords like `findBy`, `And`, `Or`, `Containing`, `OrderBy`, `Like`, `GreaterThan`, `Between`, and generates the corresponding JPQL query. If the method name is invalid or references a field that doesn't exist, you get a startup error. That's actually a good thing. You catch the mistake early instead of at runtime.

One thing to watch: field names in the method must match field names on your entity exactly, correct casing and all. If your entity has `emailAddress`, the method must be `findByEmailAddress`, not `findByEmail`. Spring won't find the field and will blow up on startup. You'll run into this sooner or later.

Quick interview answer

If they ask you to write a query to find users by city, just write: `List<User> findByCity(String city);` One line, no SQL, complete working implementation. It shows you actually understand how Spring Data works rather than just memorizing it.

@Query: When the Method Name Isn't Enough

Method naming is great for simple queries. But once things get complex, multiple joins, subqueries, aggregates, the method name gets unreadable fast. Sometimes it's just impossible to express. That's when you reach for `@Query`.

With `@Query` you write the query yourself. By default it uses JPQL (Java Persistence Query Language), which looks like SQL but operates on your Java class and field names rather than table and column names.

```
public interface ProductRepository extends JpaRepository<Product, Long> {

    // JPQL - uses class name 'Product' and field name 'price'
    @Query("SELECT p FROM Product p WHERE p.price < :maxPrice")
    List<Product> findAffordable(@Param("maxPrice") double maxPrice);

    // Native SQL - uses actual table and column names
    @Query(value = "SELECT * FROM products WHERE in_stock = 1",
            nativeQuery = true)
    List<Product> findInStockProducts();
}
```

JPQL is database-agnostic. The same query works on MySQL, PostgreSQL, Oracle, whatever. Native SQL is tied to your specific database, so if you ever switch, those queries might need rewriting. For most things, stick with JPQL.

The `:maxPrice` in that query is a named parameter. `@Param("maxPrice")` on the method argument binds the value to it. You can use positional parameters like `?1`, `?2` instead, but named ones are easier to follow when you have several arguments.

For the interview

Method name queries are generated by Spring from the method name. Good for simple lookups. @Query is for when you need to write it yourself: joins, subqueries, aggregations. JPQL by default, nativeQuery = true for raw SQL. Just know that native SQL locks you to one database.

Pagination: Fetching Data in Chunks

Imagine you have a products table with 50,000 rows. Calling findAll() loads all 50,000 into memory at once. Your server struggles. Your API slows down. Your users leave.

Pagination solves this. You fetch one page at a time, say 20 products per page. Spring Data JPA has built-in support through the Pageable interface. It feels like overhead at first, but it becomes natural pretty quickly.

```
// In your repository
public interface ProductRepository extends JpaRepository<Product, Long> {
    Page<Product> findAll(Pageable pageable);

    // Works with custom queries too
    Page<Product> findByInStock(boolean inStock, Pageable pageable);
}

// In your service
int pageNumber = 0; // zero-indexed, page 0 = first page
int pageSize   = 20;

Pageable pageable = PageRequest.of(pageNumber, pageSize);
Page<Product> page = productRepository.findAll(pageable);

// What you get back
List<Product> products = page.getContent(); // the actual data
long totalElements    = page.getTotalElements(); // 50,000
int totalPages        = page.getTotalPages();    // 2,500
boolean hasNext       = page.hasNext();
```

The Page<T> object gives you the data plus metadata, total items, total pages, whether there's a next page. Return this in your API response and your frontend developer has everything they need to build the Next and Previous buttons.

Page numbers are zero-indexed, and this catches people out. Page 0 is the first page. If someone clicks Page 1 in your UI, you send PageRequest.of(0, 20). Page 2 is PageRequest.of(1, 20). It's a small thing but it causes off-by-one bugs that take a while to spot.

Also worth knowing

You can add sorting directly to `PageRequest`: `PageRequest.of(0, 20, Sort.by("price").ascending())` gives you the first 20 products sorted cheapest first. Pagination and sorting in one call.

On pagination in interviews

Spring Data JPA provides `Pageable` and `PageRequest` for this. Instead of `findAll()` which loads everything, you pass a `PageRequest` with a page number and size. The repository returns a `Page<T>` with the data and metadata. For any endpoint that might return a lot of rows, this is the right approach.

And that's the repository layer.

On paper it sounds like the boring plumbing part. In practice it's the layer you spend a surprising amount of time thinking about, because it's where performance problems live, where lazy loading bites you, and where a query that works fine in dev falls apart against real data.

Spring Data JPA removes most of the low-level noise. You're not writing SQL for every operation. You're not managing connections manually. You focus on what the query should do and let the framework handle how.

The patterns in this chapter cover the vast majority of what you'll need in real projects. Basic CRUD, method name queries, `@Query` for complex cases, `Optional` for safe lookups, `Pageable` for large datasets. Get comfortable with these and the rest builds naturally on top.

Before moving on, take the `Product` entity from the last chapter and wire it up end-to-end. Create a `ProductRepository`, write a method to find products by name, add a `@Query` for price filtering, and expose a paginated endpoint from the controller. When that all works, you'll have a solid feel for how the data layer fits together.

SPRING BOOT INTERVIEW SERIES

The Spring Boot Playbook

From First API to Hired Developer

CHAPTER 6

Entity Relationships

TOP 1% TEACHING EDITION

OneToOne · OneToMany · ManyToMany · Fetch · Cascade

CHAPTER 6 - ENTITY RELATIONSHIPS

This is the chapter most freshers get wrong. And the one interviewers lean on hardest.

Not because the annotations are complex. Because the **failures are invisible until they break in production**. Infinite JSON loops. LazyInitializationExceptions. Cascade deletes wiping data nobody meant to remove. N+1 queries silently multiplying under load.

The developers who get these questions right in interviews don't just know what `@OneToMany` means. They know **what goes wrong** when you misuse it- and exactly how to fix it. That's what this chapter teaches.

1. Why Relationships Exist

Real applications have connected data. A user places orders. An order contains items. A student enrolls in courses. The database stores these in separate tables. Java works with them as connected objects.

Without JPA relationships, you'd manually query orders by user ID, manually query items by order ID, and manually stitch everything together in your service layer. Every time. JPA relationships let you declare the connection once. Hibernate writes the JOIN SQL. You navigate the object graph in Java.

That's the core idea. Now let's talk about where it goes wrong.

2. One-to-One

One User has exactly one UserProfile. One person holds exactly one passport. In the database, one table holds a foreign key column pointing to the other.

The entity that holds the foreign key column is called the **owning side**. The owning side always has `@JoinColumn`. The other side, if mapped at all, uses `mappedBy`.

```
@Entity
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```

    private Long id;
    private String name;

    @OneToOne(cascade = CascadeType.ALL)
    @JoinColumn(name = "profile_id") // users table gets a profile_id
column
    private UserProfile profile; // owning side
}

@Entity
public class UserProfile {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String bio;

    // Optional: map back to User. If you don't need navigation, leave it
out.
    @OneToOne(mappedBy = "profile") // inverse side- does NOT manage FK
    private User user;
}

```

The `@JoinColumn` annotation names the foreign key column in the database. Here, the `users` table gets a `profile_id` column. `UserProfile` has no column pointing back. It's the inverse side, declared with `mappedBy`.

Watch Out

If you only set the inverse side (`UserProfile.user = user`) and never set the owning side (`User.profile = profile`), Hibernate will NOT write the foreign key.

Only the owning side- the one with `@JoinColumn`- updates the database.

Setting the inverse side alone does nothing. This is the most common bidirectional relationship mistake and it produces no error. Data just silently doesn't save.

3. One-to-Many and Many-to-One

This is the relationship you'll use most. One Order has many OrderItems. Each OrderItem belongs to one Order. In the database: the `order_items` table has an `order_id` column. The `orders` table has nothing pointing at items.

That's the key fact: **the foreign key always lives on the Many side**. The child table. Not the parent. This is standard relational database design, and JPA follows it exactly.

```
// MANY side- owns the relationship, holds the FK column
@Entity
public class OrderItem {

    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String productName;
    private int quantity;

    @ManyToOne(fetch = FetchType.LAZY)    // always LAZY on @ManyToOne
    @JoinColumn(name = "order_id")        // FK column in order_items table
    private Order order;
}

// ONE side- inverse, uses mappedBy
@Entity
public class Order {

    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String status;

    @OneToMany(mappedBy = "order", cascade = CascadeType.PERSIST)
    private List<OrderItem> items = new ArrayList<>();
}
```

Maintaining Both Sides- The Mistake That Breaks Saves

In a bidirectional relationship, you must set **both sides** in code, even though only one side (**@ManyToOne**) controls the database. This catches freshers constantly.

```
// WRONG- only setting the parent's collection
Order order = new Order();
OrderItem item = new OrderItem("Laptop", 1);
order.getItems().add(item);           // added to collection in memory
// But item.order is null!
// Hibernate reads the @ManyToOne field to write the FK.
// order_id will be NULL in the database. The item is orphaned.

// CORRECT- set both sides
Order order = new Order();
OrderItem item = new OrderItem("Laptop", 1);
item.setOrder(order);                 // sets the FK- this is what matters
```

```
order.getItems().add(item);           // keeps in-memory object graph
consistent
orderRepository.save(order);
```

The `item.setOrder(order)` line is what actually writes the foreign key. Without it, `order_id` is null. No error is thrown. The data just saves incorrectly.

Use Helper Methods to Keep This Clean

Production codebases use helper methods on the parent entity to enforce correct both-side assignment:

```
// In Order.java- helper method enforces both-side assignment
public void addItem(OrderItem item) {
    item.setOrder(this);           // owning side- writes FK to DB
    this.items.add(item);          // inverse side- keeps object graph consistent
}

// Usage- clean, correct, impossible to forget
Order order = new Order();
order.addItem(new OrderItem("Laptop", 1));
order.addItem(new OrderItem("Mouse", 2));
orderRepository.save(order);
```

This pattern prevents the bug at the API level. The caller can't add an item incorrectly because the method handles both sides. It's a standard senior-developer practice in any real codebase.

INTERVIEW TRAP

What most freshers say:

"The foreign key lives wherever you put @JoinColumn."

What the interviewer expects:

"In a One-to-Many / Many-to-One, the foreign key always lives on the Many side.

@JoinColumn goes on the @ManyToOne entity- the child. The parent (@OneToMany) has no FK column. It uses mappedBy to say: the other side owns this relationship.

Only the @ManyToOne side controls database writes. Setting only the parent's collection without setting `item.setOrder()` will save a null foreign key."

→ IF YOU REMEMBER ONE THING

Only the owning side (@JoinColumn / @ManyToOne) writes the foreign key. Always set both sides in code- or use a helper method that does it for you.

4. mappedBy- The One Everyone Gets Wrong

This is where most beginners hit a wall. They set up both sides of a relationship and Hibernate creates an unexpected join table, or the foreign key is missing, or saves don't persist correctly. Almost always the cause is a misunderstanding of **mappedBy**.

Here's the rule: in a bidirectional relationship, **exactly one side must own it**. Hibernate needs one side to be responsible for writing the foreign key column. If both sides try, you get duplicate columns or unexpected tables. If neither side owns it, Hibernate defaults to a join table you didn't ask for.

mappedBy is the annotation you put on the **non-owning side**. It says: I am not the owner. Go look at the field I name on the other entity. That field has **@JoinColumn** and controls the database.

```
// OWNING SIDE- @JoinColumn defines the FK column, manages the database
@ManyToOne
@JoinColumn(name = "order_id")
private Order order;           // field name: 'order'

// INVERSE SIDE- mappedBy says 'the order field on the other entity owns
// this'
@OneToMany(mappedBy = "order")  // 'order' must match the JAVA FIELD
// NAME above
private List<OrderItem> items = new ArrayList<>();
```

The string inside **mappedBy** must match the **Java field name** on the owning side. Not the class name. Not the column name. The field name. If the field is **private Order parentOrder**, then **mappedBy = "parentOrder"**. Wrong value causes a **AnnotationException** at startup that looks confusing until you know this rule.

mappedBy Rules- Get These Right:

ONE side must have **@JoinColumn**. That side is the OWNER.

The other side uses **mappedBy**. That side is the INVERSE.

mappedBy value = the Java field name on the OWNING side. Not class name. Not column name.

No **mappedBy** on either side: Hibernate creates an extra join table for **@OneToMany**.

mappedBy on BOTH sides: Spring throws **AnnotationException** at startup.

Inverse side changes (like adding to the collection) do NOT update the database.

INTERVIEW TRAP

What most freshers say:

"mappedBy takes the class name of the related entity."

Or: "I put mappedBy on both sides to make it fully bidirectional."

What the interviewer expects:

"mappedBy marks the inverse side of a bidirectional relationship. It takes the exact Java field name on the owning side- the field annotated with @JoinColumn.

This tells Hibernate not to manage the foreign key from this side, preventing duplicate columns. If mappedBy is on both sides, Spring fails at startup.

If it's on neither side and you have @OneToMany, Hibernate creates a join table that you almost certainly didn't intend."

→ IF YOU REMEMBER ONE THING

mappedBy = Java field name on the owning side. One side owns. One side observes. Only the owner writes to the database.

5. Many-to-Many

A student enrolls in many courses. A course has many students. Neither table holds the foreign key. You need a third table- a join table- with two FK columns: `student_id` and `course_id`.

```
@Entity
public class Student {

    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;

    @ManyToMany
    @JoinTable(
        name = "student_courses",
        joinColumns = @JoinColumn(name = "student_id"),
        inverseJoinColumns = @JoinColumn(name = "course_id")
    )
    private List<Course> courses = new ArrayList<>(); // OWNING side
}

@Entity
public class Course {

    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```

private Long id;
private String title;

@ManyToMany(mappedBy = "courses")    // INVERSE side
private List<Student> students = new ArrayList<>();
}

```

The `@JoinTable` annotation controls the join table name and column names. The `joinColumns` parameter names the FK for the owning entity (Student). The `inverseJoinColumns` names the FK for the other entity (Course).

When to Replace @ManyToMany With Two @ManyToOne Relationships

The moment you need to store extra data on the relationship itself, `@ManyToMany` breaks down. When does a student enroll? What grade did they receive? What is the enrollment status? That data belongs to the relationship between Student and Course, not to either entity alone.

```

// When you need extra data on the relationship- use an intermediate entity
@Entity
public class Enrollment {

    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne
    @JoinColumn(name = "student_id")
    private Student student;

    @ManyToOne
    @JoinColumn(name = "course_id")
    private Course course;

    private LocalDate enrolledDate;    // relationship-specific data
    private String grade;
}

```

This replaces the `@ManyToMany` with two `@ManyToOne` relationships via an `Enrollment` entity. It's more code, but it gives you full control. This is the pattern production systems use.

Watch Out

`CascadeType.REMOVE` on `@ManyToMany` is almost always wrong.

If Student has cascade = CascadeType.REMOVE on its courses collection, deleting one student will DELETE ALL THEIR COURSES- including courses that other students are still enrolled in.

For @ManyToMany, manage the join table entries manually. Remove the student from the collection, save, and let Hibernate delete only the join table row. Never cascade REMOVE across a many-to-many relationship.

INTERVIEW TRAP

What most freshers say:

"I'd use @ManyToMany for students and courses."

(When asked: what if you need to store the enrollment date?)

What the interviewer expects:

"@ManyToMany works when you only need the association itself.

The moment you need to store data on the relationship- enrollment date, grade, status- you replace it with an intermediate entity like Enrollment, and use two @ManyToOne relationships pointing at it. This is the production approach.

I'd also avoid CascadeType.REMOVE on @ManyToMany because it risks deleting shared entities that other records still reference."

→ IF YOU REMEMBER ONE THING

@ManyToMany is fine for pure associations. The moment the relationship has its own data, model it as an entity.

6. FetchType: EAGER vs LAZY

FetchType controls when related data gets loaded from the database.

FetchType	When Loaded	Default For
EAGER	Immediately with parent- always	@ManyToOne, @OneToOne
LAZY	Only when you access the field	@OneToMany, @ManyToMany

LAZY is better for performance. It means Hibernate only hits the database when you actually need the related data. EAGER means Hibernate fetches everything upfront, whether you use it or not.

Why LAZY Causes LazyInitializationException

LAZY loading works while a Hibernate session is active. A session is open during a transaction. When the transaction closes, the session closes. If you try to access a lazy collection **after** the transaction has ended, Hibernate tries to load it but has no session. It throws **LazyInitializationException**.

```
// Service layer- transaction is active here
@Transactional
public Order getOrder(Long id) {
    return orderRepository.findById(id).orElseThrow();
    // items list is LAZY- not loaded yet, Hibernate has a proxy in its place
}

// Controller- transaction has ALREADY closed when this runs
Order order = orderService.getOrder(1L);
order.getItems(); // Hibernate tries to load- but session is gone
                // LazyInitializationException
```

Two correct fixes:

```
// FIX 1: JOIN FETCH in the query- loads everything in one SQL statement
@Query("SELECT o FROM Order o JOIN FETCH o.items WHERE o.id = :id")
Optional<Order> findWithItems(@Param("id") Long id);

// FIX 2: Keep the transaction open through the service method
@Transactional
public OrderDTO getOrderWithItems(Long id) {
    Order order = orderRepository.findById(id).orElseThrow();
    order.getItems().size(); // triggers load while session is active
    return mapToDTO(order); // return DTO, not entity
}
```

JOIN FETCH is cleaner for read operations. It pulls both entities in a single SQL query, no second trip to the database. Use it in any query method where you know you need the collection.

Watch Out

Never set FetchType.EAGER on @OneToMany collections to 'fix' LazyInitializationException.

EAGER on a collection means every time you load one Order, Hibernate also loads ALL its items. Load 50 orders in a list- Hibernate fires 50 separate queries for items.

This is the N+1 problem. It's invisible in testing with small data.

In production with real load, it kills performance.

Fix `LazyInitializationException` with `@Transactional` or `JOIN FETCH`.

Never with `EAGER` on a collection.

INTERVIEW TRAP

What most freshers say:

"I switch to `EAGER` loading when I get `LazyInitializationException`."

Or: "`LAZY` is a problem so I just use `EAGER` to be safe."

What the interviewer expects:

"`LAZY` is the correct default for collections and I keep it that way.

`LazyInitializationException` means the collection was accessed outside an active session. I fix it either with `JOIN FETCH` in the query- which loads the collection in one SQL statement- or by ensuring the service method is `@Transactional` so the session stays open.

Setting `EAGER` on `@OneToMany` causes N+1 queries, which is a far worse problem at scale than a `LazyInitializationException` you can actually see."

→ IF YOU REMEMBER ONE THING

`LAZY` is not the problem- accessing lazy data outside a session is the problem. Fix the session boundary, not the fetch type.

7. Cascade Types

Cascade controls whether JPA operations on the parent automatically propagate to the related entities. Save, delete, merge, refresh- each can be cascaded or not.

CascadeType	What Propagates
PERSIST	parent.save() cascades to children
REMOVE	parent.delete() also deletes children
MERGE	parent.merge() propagates to children
REFRESH	parent.refresh() reloads children from DB
DETACH	parent.detach() detaches children too
ALL	All of the above. Convenient- but think before using.

The two you'll use most:

CascadeType.PERSIST: Save the parent, children are saved too. You don't call `save()` on every child separately.

CascadeType.ALL: Use when the child entity's entire lifecycle is tied to the parent. An `OrderItem` without an `Order` makes no sense. `UserProfile` without a `User` has no meaning. `All` is appropriate for those.

```
// PERSIST only- saving Order saves its items. Deleting Order does NOT delete items.
@OneToMany(mappedBy = "order", cascade = CascadeType.PERSIST)
private List<OrderItem> items = new ArrayList<>();

// ALL- full lifecycle tied together. Profile lives and dies with User.
@OneToOne(cascade = CascadeType.ALL)
@JoinColumn(name = "profile_id")
private UserProfile profile;

// With PERSIST: save parent + children in one call
Order order = new Order();
order.addItem(new OrderItem("Laptop", 1)); // helper method sets both sides
order.addItem(new OrderItem("Mouse", 2));
orderRepository.save(order); // both items saved automatically
```

Watch Out

`CascadeType.REMOVE` on `@ManyToMany` is almost always wrong.

If you cascade `REMOVE` from `Student` to `Course` in a many-to-many, deleting a student deletes ALL the courses they were enrolled in- including courses with other students.

Also watch out for `CascadeType.ALL` on `@ManyToOne`. If `OrderItem` cascades `ALL` to `Order`, deleting one item would delete the entire `Order`. That's almost never intentional.

Use cascade deliberately. Ask: should the parent's delete remove the child? If the answer isn't an immediate yes, don't cascade `REMOVE`.

INTERVIEW TRAP

What most freshers say:

"I always use CascadeType.ALL- it's the most complete option."

What the interviewer expects:

"I use cascade selectively. CascadeType.PERSIST makes sense when children are always created with the parent- like OrderItems with an Order.

CascadeType.ALL is appropriate when the child's lifecycle is completely owned by the parent- like a UserProfile. I avoid cascading REMOVE across @ManyToMany because you risk deleting shared entities other records still reference. CascadeType.ALL on @ManyToOne is also a red flag- it can cause accidental parent deletion when a child is removed."

→ IF YOU REMEMBER ONE THING

Cascade ALL is not lazy configuration. It's a commitment: the child lives and dies with the parent. Only use it when that's genuinely true.

8. The Infinite Loop Problem

This is the most common production bug freshers introduce with relationships. You return a JPA entity from a REST controller. The response never arrives. The server logs a `StackOverflowError`. Or it keeps running until it crashes.

The cause: bidirectional relationships create a circular reference. Jackson serializes `Order`. `Order` has `items`. Each item has `order`. Which has `items`. Which have `order`. It never ends.

```
// What Jackson sees:
Order
  items: [
    OrderItem
      order: Order          // <-- back to Order
      items: [OrderItem    // <-- infinite
        order: Order      // <-- still going
        ...]
  ]
```

Fix 1: @JsonManagedReference and @JsonBackReference

Jackson annotations that break the cycle. `@JsonManagedReference` on the parent side includes that field in serialization. `@JsonBackReference` on the child side **excludes** it from serialization entirely.


```
// In Order.java
@JsonManagedReference
@OneToMany(mappedBy = "order", cascade = CascadeType.ALL)
private List<OrderItem> items;

// In OrderItem.java
@JsonBackReference          // 'order' field is excluded from JSON output
@ManyToOne
@JoinColumn(name = "order_id")
private Order order;
```

Fix 2: @JsonIgnore

Simpler. Put **@JsonIgnore** on the back-reference field. It won't appear in JSON output at all. No cycle, no error. Downside: if you later need that field in some responses, you can't- it's always excluded.

```
// In OrderItem.java
@JsonIgnore
@ManyToOne
@JoinColumn(name = "order_id")
private Order order;
```

Fix 3: Use DTOs (The Production Approach)

This is what production codebases actually do. Don't serialize entities directly. Map to a DTO- a plain Java object with only the fields you want in the response. No entity, no circular reference. Ever.

```
// DTO- only the data the client needs
public class OrderResponse {
    private Long id;
    private String status;
    private List<OrderItemResponse> items;
}

public class OrderItemResponse {
    private Long id;
    private String productName;
    private int quantity;
    // No 'order' reference- the circular reference problem doesn't exist
    here
}

// In service- map entity to DTO before returning
@Transactional(readOnly = true)
public OrderResponse getOrder(Long id) {
    Order order = orderRepository.findWithItems(id).orElseThrow();
```

```
return mapToResponse(order); // map to DTO, return DTO
}
```

DTOs also give you full control over what data leaves your API. Entity fields like `internalNotes`, `costPrice`, or `passwordHash` should never reach the client. With DTOs, you decide exactly what goes out. With entity serialization, you're one missed `@JsonIgnore` away from a data leak.

Real-World Context

In production APIs, returning entities directly from controllers is considered a code smell for exactly these reasons: circular references, accidental data exposure, and tight coupling between your database structure and your API contract.

Use DTOs. Map with a dedicated mapper (ModelMapper, MapStruct, or manual mapping). It's more code, but it's the right separation.

INTERVIEW TRAP

What most freshers say:

*"I add @JsonIgnore to fix the infinite loop."
(Works but shows you don't know the production pattern)*

What the interviewer expects:

"There are three approaches. `@JsonManagedReference` and `@JsonBackReference` break the cycle at the Jackson level. `@JsonIgnore` is simpler but permanently excludes the field from all serialization. The proper production solution is DTOs- never serialize entities directly. Map to a DTO in the service layer. This eliminates circular references entirely and gives you full control over what data leaves your API. It also decouples your API contract from your database schema, which matters when either side needs to change."

→ IF YOU REMEMBER ONE THING

Never serialize entities from controllers. Map to DTOs. It eliminates circular references and prevents accidental data exposure in one move.

Chapter 6 Summary

JPA relationships are where interviews get won or lost. This is the chapter to re-read the night before any technical round. Not because the annotations are hard- because the failure modes are subtle and interviewers know all of them.

1. **@OneToOne / @ManyToOne:** The owning side holds @JoinColumn. The inverse side uses mappedBy. Only the owning side writes the FK. Setting the inverse alone does nothing.
2. **@OneToMany:** FK lives on the Many side, always. Initialize collections (new ArrayList<>) to avoid NullPointerExceptions.
3. **Both sides must be set:** In bidirectional relationships, set both sides in code. Use helper methods on the parent to enforce this consistently.
4. **mappedBy:** Marks the inverse side. Value = Java field name on the owning side. One owner. One observer. Wrong value causes startup error.
5. **@ManyToMany:** Fine for pure associations. When you need data on the relationship, replace with an intermediate entity. Never cascade REMOVE.
6. **LAZY / EAGER:** LAZY is the correct default for collections. Fix LazyInitializationException with JOIN FETCH or @Transactional. Never with EAGER on a collection.
7. **Cascade:** PERSIST for save propagation. ALL when child lifecycle is tied to parent. Never cascade REMOVE on @ManyToMany. Never ALL on @ManyToOne.
8. **Infinite loop:** Caused by serializing bidirectional entities. Fix with DTOs- don't return entities from controllers.

Before Your Interview:

Draw the tables for Order / OrderItem and explain which table holds the FK and why.

Explain mappedBy from scratch: what it does, what value it takes, what breaks without it.

Walk through the singleton-prototype injection trap- the problem, why it happens, the fix.

Explain N+1 and how JOIN FETCH prevents it.

Be ready for: 'You have LazyInitializationException- what do you do?' Answer: not EAGER.

Know CascadeType.ALL on @ManyToMany is dangerous- and say it unprompted.

Build This Before Your Next Interview:

Create an Order / OrderItem domain with proper bidirectional @OneToMany / @ManyToOne.

Use the addItem() helper method pattern.

Return Order from a REST controller- deliberately trigger the infinite loop.

Fix it with a DTO. Verify the response is clean.

Add JOIN FETCH to the repository query and confirm only one SQL fires.

Seeing these problems and fixing them yourself makes the theory stick permanently.

Up next: Chapter 7- Spring Data JPA: Custom Queries and Repository Patterns

@JavaInterviewready

THE SPRING BOOT PLAYBOOK

From First API to Hired Developer

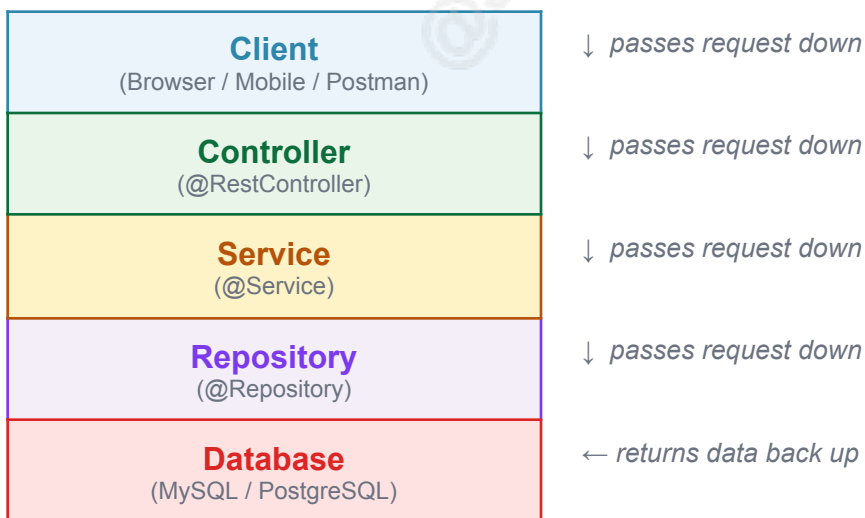
CHAPTER 7: BUILDING A REAL API (PROJECT FLOW)

Chapter 7: Building a Real API

This chapter builds a User API from scratch. Every decision maps to how real teams structure production code. By the end, you can explain the architecture, justify each layer, and handle the edge cases interviewers use to separate candidates who understand the design from those who just followed a tutorial.

1. The Layered Architecture

Layered architecture means each component has a single, clearly scoped responsibility and communicates only with the layer directly adjacent to it. The reason this matters in production is not organisation for its own sake. It is testability, replaceability, and controlled failure. When layers are properly separated, you can swap the database, change a business rule, or test a controller without touching unrelated code.



This is what interviewers mean when they say "walk me through a request." Trace it both directions: request flows down through layers, response flows back up. Each layer touches only its immediate neighbour. Skipping a layer is not a shortcut, it is a structural failure with specific consequences.

◆ Mental Model: Layered Architecture in One Sentence

```
Controller = HTTP adapter      (no logic)
Service    = business rules    (all logic)
Repository = data access       (no logic)
```

If you can say what each layer **REJECTS**, you understand the design.
Controller rejects business logic. Service rejects HTTP concerns.
Repository rejects business rules. Each boundary is deliberate.

Request Lifecycle

```
Client (HTTP Request)
|
v
@RestController --> deserialise JSON, bind to DTO
|                  call service, return ResponseEntity
v
@Service --> validate business rules, apply @Transactional
|            orchestrate one or more repositories
v
@Repository --> generate SQL via JPA, execute against DB
|
v
Database --> persist or return data
|
^ (response travels back up the same chain as DTOs)

// Failure at any layer throws a typed exception.
// Exception propagates up to @RestControllerAdvice.
// Client receives a structured error response, never a stack trace.
```

The Controller Layer

The controller has exactly one job: translate an HTTP request into a service call and return the result as an HTTP response. Input extraction, delegation, response building. Nothing else.

Any logic in the controller is logic you cannot unit test without starting a web context. It is also logic you cannot reuse if the same operation is triggered by a scheduled job, a message queue

consumer, or a different endpoint. Controllers that grow beyond five lines per method are absorbing responsibility that belongs in the service.

```
@RestController
@RequestMapping("/api/users")
public class UserController {

    private final UserService userService;

    // Constructor injection: preferred over @Autowired on fields.
    // Dependencies are explicit, final, and easy to mock in tests.
    public UserController(UserService userService) {
        this.userService = userService;
    }

    @PostMapping
    public ResponseEntity<UserResponseDTO> createUser(
        @Valid @RequestBody UserRequestDTO dto) {
        UserResponseDTO saved = userService.createUser(dto);
        return ResponseEntity.status(HttpStatus.CREATED).body(saved);
    }

    @GetMapping
    public ResponseEntity<List<UserResponseDTO>> getAllUsers() {
        return ResponseEntity.ok(userService.getAllUsers());
    }

    @GetMapping("/{id}")
    public ResponseEntity<UserResponseDTO> getUserById(@PathVariable Long id) {
        return ResponseEntity.ok(userService.getUserById(id));
    }
}
```

► Senior Insight

In code reviews, the signal I look for is whether the controller method can be read in under 10 seconds. If it requires more context to understand, it is doing too much. The controller is an HTTP adapter. Its job is to translate between the HTTP world and the service world. Nothing more.

✓ Production standard: if a controller method needs a comment to explain the business logic, that logic belongs in the service. Controller comments describe the HTTP contract: which verb, which status code, what body shape.

The Service Layer

The service layer owns the business logic. Uniqueness rules, permission checks, complex validations beyond field-level constraints, multi-step operations that span more than one repository. If a business rule exists, it lives here.

`@Transactional` on write methods is not optional. If your service calls `save()` twice and the second one fails, without a transaction the first write is already committed. You now have partial data in the database. `@Transactional` guarantees that all operations within the method either commit together or roll back together.

⚠ Production Failure

A money transfer service credits account B and then tries to debit account A. The debit throws a database constraint violation. Without `@Transactional`, account B already has the money but account A was never debited. The transaction is in a permanently inconsistent state. `@Transactional` rolls back the credit when the debit fails. This is not an edge case. It is the default failure mode for multi-step write operations.

```
@Service
public class UserService {

    private final UserRepository userRepository;

    public UserService(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    @Transactional
    public UserResponseDTO createUser(UserRequestDTO dto) {
        // Real business rule: enforce email uniqueness
        if (userRepository.existsByEmail(dto.getEmail())) {
            throw new DuplicateEmailException(
                "An account with this email already exists: " + dto.getEmail()
            );
        }

        User user = new User();
        user.setName(dto.getName());
        user.setEmail(dto.getEmail());
        // In production: hash the password before storing.
        // user.setPasswordHash(passwordEncoder.encode(dto.getPassword()));

        User savedUser = userRepository.save(user);
        return toResponseDTO(savedUser);
    }

    public List<UserResponseDTO> getAllUsers() {
        return userRepository.findAll()
            .stream()
            .map(this::toResponseDTO)
            .collect(Collectors.toList());
    }
}
```



```

public UserResponseDTO getUserById(Long id) {
    User user = userRepository.findById(id)
        .orElseThrow(() -> new UserNotFoundException(
            "No user found with id: " + id
        ));
    return toResponseDTO(user);
}

private UserResponseDTO toResponseDTO(User user) {
    UserResponseDTO dto = new UserResponseDTO();
    dto.setId(user.getId());
    dto.setName(user.getName());
    dto.setEmail(user.getEmail());
    dto.setCreatedAt(user.getCreatedAt());
    return dto;
}
}

```

The entity-to-DTO conversion happens inside the service, not in the controller. The controller receives a DTO and returns a DTO. The entity never crosses the layer boundary into the web layer. This is enforced by design: the controller import list should never include an entity class.

► Senior Insight

A practical rule I apply on every project: the controller package should have zero imports from the entity package. If a code review shows a controller importing an entity, that is an immediate comment requesting a DTO. The entity belongs to the persistence layer. Letting it leak into the web layer creates coupling between your database schema and your API contract.

⚠ Common mistake: throwing `RuntimeException` with a string message. Generic exceptions cannot be caught specifically. Your `@RestControllerAdvice` fallback handler catches them all as 500 errors. A typed `UserNotFoundException` maps cleanly to 404. The exception type is the signal, not the message string.

The Repository Layer

The repository is a pure data access interface. No business logic, no conditional branching, no awareness of HTTP. Its contract is simple: persist data, retrieve data, delete data.

Spring Data JPA generates the SQL implementation at runtime from the method signatures you define. Keep the repository focused. If you find yourself adding service-level logic to a repository method, it is a responsibility leak that makes both layers harder to test.

```

@Repository
public interface UserRepository extends JpaRepository<User, Long> {

    // Spring reads the method name and generates:
    // SELECT * FROM users WHERE email = ?
    Optional<User> findByEmail(String email);

    // Used for the duplicate-check in the service.
    // More efficient than findByEmail- no data transfer, just a COUNT query.
    boolean existsByEmail(String email);
}

```

existsByEmail issues a COUNT query. findByEmail issues a SELECT and transfers the full row. For a pre-save uniqueness check, existsByEmail is the correct choice: it does less work and communicates the intent clearly. Choosing the right query method matters at scale when these checks run thousands of times per minute.

► Senior Insight

When you define custom repository methods, use the most specific return type the use case requires. existsBy for presence checks, countBy for quantities, deleteBy for bulk removals. findBy when you actually need the data. Over-fetching at the repository level is one of the quietest performance problems in JPA applications.

2. DTO vs Entity

Exposing entities directly from controllers is a design failure with concrete consequences. It is not a style preference.

DTO vs Entity: What Each One Is

[ENTITY]	[DTO]
@Entity	Plain Java class
Managed by Hibernate	No ORM lifecycle
Tied to DB schema	Shaped for the API
Full row (all columns)	Only fields client needs
Can cause serialisation failures (recursion, lazy loading crashes)	Safe to serialise
Lives in persistence	No circular references
	No session dependency
	Lives at layer boundary

```

// Rule: entities stay in the persistence layer.
// DTOs travel between layers and across the network.

```

Entity	DTO (Data Transfer Object)
Mapped to a database table	Plain Java class, no persistence annotations
Has @Entity annotation	No @Entity, no JPA concerns
Managed by Hibernate session	Detached from any ORM lifecycle
May contain sensitive fields (password hash, tokens)	You choose exactly which fields are exposed
Tied to database structure	Tied to what the client actually needs
Can cause infinite recursion with bidirectional relations	Safe to serialize- no circular references

Why We Don't Expose the Entity Directly

Four reasons why returning entities from controllers is unsafe in production:

Sensitive fields. An entity carries everything the database stores: passwordHash, resetToken, internalFlags. Jackson will serialise all of it unless every sensitive field has @JsonIgnore. That annotation gets forgotten on the next field someone adds. A DTO has no fields you did not put there deliberately.

Infinite recursion. A User has a list of Orders. Each Order has a reference back to User. Jackson follows that cycle until you get a StackOverflowError. This crashes the thread and returns a 500 to the client. DTOs contain only the fields you declare, so the cycle cannot exist.

Lazy loading failures. Entities have JPA-managed state. Access a lazy-loaded collection outside an active Hibernate session and you get LazyInitializationException. This happens the moment the entity leaves the service layer and enters the controller. DTOs are plain Java objects with no ORM state attached.

Schema coupling. Your API is a contract. Your database schema is an implementation detail. If they are the same object, every schema migration is a breaking API change. A DTO lets you version the API independently of the database structure.

⚠ Production Failure

A team returns User entities from the controller. The User entity has a @OneToMany relationship to Orders, and each Order has a @ManyToOne back to User. Jackson enters an infinite loop serialising the graph. The thread throws StackOverflowError. The endpoint returns a 500 with no useful error body. Adding @JsonIgnore on one side stops the recursion but creates a hidden dependency between the serialisation strategy and the entity model. The correct fix is a DTO that contains only the fields the client actually needs.

✓ In interviews, most candidates mention only sensitive field leakage. Lead with infinite recursion and LazyInitializationException instead. Both are runtime failures with specific exception types and specific production symptoms. Naming them signals that you have actually debugged JPA serialisation problems, not just read about them.

Request DTO vs Response DTO

A single DTO used for both input and output forces nullable fields on both sides, makes validation annotations confusing, and creates a structural path for sensitive fields to leak into responses. The shapes are different enough to warrant separate classes.

UserRequestDTO accepts name, email, password. UserResponseDTO returns id, name, email, createdAt. No password field exists in the response class, so it cannot be accidentally included. This is enforced by the type system, not by discipline or @JsonIgnore annotations that someone will eventually remove.

```
// What arrives from the client
public class UserRequestDTO {
    @NotBlank private String name;
    @Email private String email;
    @Size(min=8) private String password;
}

// What goes back to the client -- no password field, ever
public class UserResponseDTO {
    private Long id;
    private String name;
    private String email;
    private LocalDateTime createdAt;
}
```

► Senior Insight

A useful test: if you can delete UserResponseDTO and replace it with the entity, the only thing stopping you should be the missing fields in the response. If nothing stops you, the

DTO is not doing its job. A well-designed response DTO makes field leakage structurally impossible, not just unlikely.

3. What Happens When Things Go Wrong

Every public API endpoint has at least two paths: the path where everything works, and the path where something does not. The failure path is not optional. It is part of the contract. Clients need predictable error responses, not 500s with stack traces.

The two failure scenarios every User API must handle explicitly:

```
// Scenario 1: Resource not found
// GET /api/users/999- no user with that ID
//
// The service throws:
// new UserNotFoundException("No user found with id: 999")
//
// @ControllerAdvice catches it and returns:
// HTTP 404 Not Found
// { "error": "No user found with id: 999" }

// Scenario 2: Duplicate email
// POST /api/users- email already in the database
//
// The service throws:
// new DuplicateEmailException("Email already registered")
//
// @ControllerAdvice catches it and returns:
// HTTP 409 Conflict
// { "error": "An account with this email already exists" }
```

The service throws typed exceptions. The controller has no try-catch blocks. A centralised `@RestControllerAdvice` intercepts every exception and maps it to the correct HTTP response. This is covered in Chapter 8. The principle to understand now: exception handling is a cross-cutting concern, not controller logic.

Error Flow: Exception to Client

```
Service throws UserNotFoundException("No user with id: 99")
|
v (propagates up the call stack)
Controller receives the exception
| (no try-catch in the controller)
v
```

```

@RestControllerAdvice intercepts
|
+-- @ExceptionHandler(UserNotFoundException.class)
|
v
Maps to: HTTP 404
Body:    { "error": "No user with id: 99", "path": "/api/users/99" }

// The client receives a structured JSON error.
// The stack trace never leaves the server.
// All exception-to-status mapping lives in one class.

```

4. API Response Design

Some teams wrap every response in a generic envelope: { success, message, data }. It provides a predictable shape for client code and simplifies error handling on the frontend.

The problem arises when teams return HTTP 200 for every response regardless of outcome, and use the success field to signal failure. That breaks REST semantics and makes monitoring harder. A 500 that returns 200 with success: false will not trigger your alerting rules. HTTP status codes exist for a reason. Use them correctly, with or without a wrapper.

```

// Option A: No wrapper. Simpler, follows REST semantics.
// Success:
// HTTP 201 { "id": 1, "name": "Ravi", "email": "ravi@example.com" }

// Option B: Generic wrapper. Common in larger teams.
// HTTP 201 { "success": true, "message": "User created",
//           "data": { "id": 1, "name": "Ravi", ... } }

// Both are valid. What matters is consistency.
// Pick one and use it everywhere. Mixed patterns are the real problem.

```

✓ Interview framing: "We use a response wrapper for a consistent envelope, but we still map HTTP status codes correctly. The wrapper does not replace status codes, it complements them. A 404 returns HTTP 404 with a structured error body, not a 200 with success: false." That answer shows you understand REST semantics and real-world pragmatism.

5. Common Mistakes to Avoid

⚠ Field injection with `@Autowired` makes dependencies hidden and mutable. Constructor injection makes them explicit and final. More importantly, constructor-injected classes can be unit tested by passing mocks directly, with no Spring context required. That is not a theoretical benefit: it is the difference between a test that runs in 50ms and one that takes 10 seconds to start a context.

⚠ Returning entities from controllers is a production incident waiting to happen. Sensitive field leakage, serialisation failures from circular references, and `LazyInitializationException` from accessing unloaded collections outside the session. Any one of these will cause a 500 in production. Return a DTO.

⚠ Controller calling the repository directly. This works until it does not. The first time you need an audit log, a permission check, or a second repository call, you have no clean place to put it. The service layer exists to give business logic a home that is testable and reusable.

⚠ Catching all exceptions in the controller with a generic try-catch. This hides failures, makes debugging harder, and bypasses your global error handler. Throw typed exceptions from the service and let `@RestControllerAdvice` handle them in one place.

6. The Interview-Ready Summary

When an interviewer asks you to walk through your architecture, the answer that passes is not a description of the layers. It is a justification of the boundaries. Here is how a senior candidate answers it:

"I use a strict three-layer architecture. The controller handles HTTP concerns only: deserialising the request, calling the service, and serialising the response. No business logic belongs there. The service owns all business rules, enforces constraints like email uniqueness, and manages transaction boundaries with `@Transactional` on write operations. The repository is a pure data access interface with no knowledge of HTTP or business rules. I never return entities from the controller. Entities are Hibernate-managed objects tied to the database schema. Exposing them directly causes serialisation failures with bidirectional relationships, exposes sensitive fields, and couples the API contract to the database structure. DTOs give me control over exactly what the client receives."

If they push on error handling: "Validation failures return 400, missing resources 404, duplicate data 409. All exceptions are handled in a single `@RestControllerAdvice`. Controllers have no try-catch blocks. The exception type determines the HTTP response, and the mapping is centralised."

If they ask why you use DTOs: "Entities carry Hibernate state. Returning them outside a session causes `LazyInitializationException`. Bidirectional relationships cause infinite recursion during serialisation. DTOs are plain objects: no session dependency, no circular references, no accidental field exposure. The DTO is also the API contract. The entity is the database contract. Keeping them separate means one can change without breaking the other."

Chapter 7 Complete

Every boundary in this architecture exists because a specific failure mode made it necessary. Controllers absorb HTTP concerns so services do not have to. Services own business logic so controllers stay testable. DTOs sit at the boundary so entities never reach the serialisation layer. Understand the failure each boundary prevents, and you understand the architecture.

The Spring Boot Playbook | Chapter 7 | Premium Edition

THE SPRING BOOT PLAYBOOK

From First API to Hired Developer

CHAPTER 8: VALIDATION & ERROR HANDLING

Chapter 8: Validation & Error Handling

Every API you have ever used that returns a clean error message, with the right status code, and no stack trace in the response, had someone who thought carefully about this chapter.

Validation and error handling are not defensive features you add at the end. They are load-bearing parts of a production API. Get them wrong and your system becomes a debugging nightmare. Get them right and your API communicates clearly with both clients and engineers.

This chapter covers the full picture: how Spring validates input, how exceptions propagate, how you intercept them globally, and what a production-grade error response actually looks like.

SECTION 1: VALIDATION

Rejecting bad data before it reaches your business logic

1.1 The Validation Lifecycle

Before any code in your service runs, Spring can inspect the incoming request and reject it if it does not meet your rules. This is Bean Validation, and it runs at the controller boundary.

REQUEST VALIDATION FLOW

```
Client Request
|
v
DispatcherServlet -> routes to controller method
|
v
@RequestBody -> Jackson deserialises JSON into DTO
|
v
```

```

@Valid / @Validated -> Bean Validation runs on DTO fields
|
+-- FAIL --> MethodArgumentNotValidException
|
|           |
|           v
|           @RestControllerAdvice -> structured error response
|
+-- PASS --> Controller method executes -> Service

```

The key insight: if validation fails, your service method never runs. The exception is thrown by the framework before your code sees the data. That is exactly what you want.

1.2 Triggering Validation with @Valid

Adding annotations like @NotBlank and @Email to your DTO fields does nothing by itself. Spring only runs the validation when you explicitly ask it to with @Valid on the method parameter.

@Valid on the controller parameter

```

@PostMapping("/users")
public ResponseEntity<UserResponseDTO> createUser(
    @Valid @RequestBody UserRequestDTO dto) {
    // This line only runs if all validations pass.
    // If any fail, MethodArgumentNotValidException is thrown
    // before we reach here.
    return ResponseEntity
        .status(HttpStatus.CREATED)
        .body(userService.createUser(dto));
}

```

Senior Developer Note

A common mistake: adding constraints to the DTO class but forgetting @Valid on the controller parameter.

The application compiles and runs. No errors at startup. Invalid data passes straight through.

The constraints are silently ignored. Always verify @Valid is present when debugging unexpected validation failures.

1.3 @Valid vs @Validated- The Distinction That Matters

@Valid is part of the standard Java Bean Validation API (javax.validation). @Validated is Spring's own extension. They both trigger validation but @Validated adds one capability: validation groups.

@Valid vs @Validated comparison

```
// @Valid- standard, works on method parameters in controllers
public ResponseEntity<?> create(@Valid @RequestBody UserRequestDTO dto) {}

// @Validated- Spring extension, supports validation groups
// Also required when you want to validate method parameters
// in a @Service class (not just @RequestBody in controllers)

@Service
public class UserService {

    public void processUser(@NotNull @Valid UserRequestDTO dto) {
        // Validation runs here too, inside the service
    }
}

// Also used for group-based conditional validation:
public interface OnCreate {}
public interface OnUpdate {}

public class UserRequestDTO {
    @NotNull(groups = OnCreate.class)
    private String email;    // required on create, optional on update
}

// Controller selects which group to validate
@PostMapping
public ResponseEntity<?> create(
    @Validated(OnCreate.class) @RequestBody UserRequestDTO dto) {}
```

✓ In interviews: @Valid is sufficient for controller-level validation. Use @Validated when you need method-level validation inside service classes, or when you have different rules for create vs update operations.

1.4 The Constraint Annotations- Know Them Cold

These four cause the most confusion in interviews. The differences are small but consequential in production.

Annotation	Passes	Fails
<code>@NotNull</code>	"hello" "" " "	null
<code>@NotEmpty</code>	"hello" " "	null ""
<code>@NotBlank</code>	"hello"	null "" " "
<code>@Size(min=2, max=50)</code>	"hi" "hello"	"a" 66-char string

For any text field a user fills in, `@NotBlank` is almost always the right choice. It covers null, empty string, and whitespace-only strings in one annotation. `@NotNull` on a String field passes for blank inputs, which is rarely what you want.

`@NotNull` has a legitimate use on non-String types: Integer, Long, Boolean, enum fields. You cannot put `@NotBlank` on an Integer. Use `@NotNull` there, combined with `@Min` or `@Max` if you need range checks.

One place to NOT use them: on fields you expect to be optional. If a user's phone number is genuinely optional in your system, do not put `@NotBlank` on it. Add `@Size` only if it is provided, using `@Size` alone (which allows null) or a custom validator.

1.5 A Production-Quality Request DTO

Here is what a real request DTO looks like when validation is applied thoughtfully, not just sprayed on every field.

UserRequestDTO.java

```
public class UserRequestDTO {

    @NotBlank(message = "Name is required")
    @Size(min = 2, max = 50, message = "Name must be between 2 and 50 characters")
    private String name;

    @NotBlank(message = "Email is required")
    @Email(message = "Must be a valid email address")
    private String email;

    @NotBlank(message = "Password is required")
    @Size(min = 8, message = "Password must be at least 8 characters")
    private String password;

    @NotNull(message = "Age is required")
    @Min(value = 18, message = "Must be at least 18 years old")
    private Integer age;
}
```

```

    @Max(value = 120, message = "Age value is not realistic")
    private Integer age;

    // Optional field- no @NotBlank, only size constraint if provided
    @Size(max = 15, message = "Phone number too long")
    private String phone;

    // getters and setters
}

```

1.6 Nested Object Validation

When a DTO contains another object as a field, constraints on the inner object are ignored unless you explicitly annotate the field with `@Valid`. This is one of the most missed pieces of validation in fresher code.

Nested validation with `@Valid` on the inner object

```

public class OrderRequestDTO {

    @NotBlank(message = "Product name is required")
    private String productName;

    @NotNull(message = "Shipping address is required")
    @Valid // <-- Without this, AddressDTO constraints are NOT checked
    private AddressDTO shippingAddress;
}

public class AddressDTO {

    @NotBlank(message = "Street is required")
    private String street;

    @NotBlank(message = "City is required")
    private String city;

    @Pattern(regexp = "[0-9]{6}$", message = "Pincode must be 6 digits")
    private String pincode;
}

```

1.7 Custom Validators- Awareness, Not Deep Dive

Built-in annotations cover most cases. When they do not, Spring lets you define custom constraints using `@Constraint`. A common example: validating that a string matches a specific business pattern your application defines.

Custom constraint annotation

```
// Step 1: Define the annotation
@Documented
@Constraint(validatedBy = ValidUUIDValidator.class)
@Target({ ElementType.FIELD })
@Retention(RetentionPolicy.RUNTIME)
public @interface ValidUUID {
    String message() default "Must be a valid UUID";
    Class<?>[] groups() default {};
    Class<? extends Payload>[] payload() default {};
}

// Step 2: Implement the validator
public class ValidUUIDValidator
    implements ConstraintValidator<ValidUUID, String> {

    @Override
    public boolean isValid(String value, ConstraintValidatorContext ctx) {
        if (value == null) return true; // let @NotNull handle nulls
        try {
            UUID.fromString(value);
            return true;
        } catch (IllegalArgumentException e) {
            return false;
        }
    }
}

// Step 3: Use it
public class RequestDTO {
    @ValidUUID
    private String correlationId;
}
```

Senior Developer Note

Custom validators integrate cleanly with the rest of the validation pipeline.

If the custom validator returns false, Spring throws the same `MethodArgumentNotValidException`

as any built-in constraint. Your `@RestControllerAdvice` handles it identically.

No special wiring required.

SECTION 2: ERROR HANDLING

What happens when things go wrong, and how to control it

2.1 The Exception Flow

When an exception is thrown anywhere in your application, Spring needs to catch it and produce a response. Without a global handler, Spring returns its own default error structure, which is inconsistent and often exposes internal details.

EXCEPTION PROPAGATION FLOW

```
Controller / Service / Repository
|
| throws exception
v
DispatcherServlet -> looks for a handler
|
v
@RestControllerAdvice
|
+--> @ExceptionHandler(MethodArgumentNotValidException)
+--> @ExceptionHandler(UserNotFoundException)
+--> @ExceptionHandler(HttpMessageNotReadableException)
+--> @ExceptionHandler(Exception) // fallback
|
v
Structured ApiErrorResponse -> Client

// No stack trace. No internal details. Clean JSON.
```

2.2 @RestControllerAdvice vs @ControllerAdvice

@ControllerAdvice is the base annotation. It intercepts exceptions globally, but it does not assume anything about the response format. If you use it, you need to add **@ResponseBody** to each handler method, or your methods will try to resolve a view instead of returning JSON.

@RestControllerAdvice combines **@ControllerAdvice** and **@ResponseBody**. Every handler method automatically serialises its return value to JSON. That is what you want in a REST API.

@ControllerAdvice vs @RestControllerAdvice

```
// Old way- requires @ResponseBody on each method
@ControllerAdvice
public class GlobalExceptionHandler {
    @ExceptionHandler(UserNotFoundException.class)
    @ResponseBody // <-- must not forget this
    public ResponseEntity<?> handleNotFound(...) { ... }
}
```

```
// Modern way- @ResponseBody is implicit on all methods
@RestControllerAdvice
public class GlobalExceptionHandler {
    @ExceptionHandler(UserNotFoundException.class)
    public ResponseEntity<?> handleNotFound(...) { ... }
}
```

✓ Use `@RestControllerAdvice`. It is the correct choice for REST APIs. If an interviewer asks why, say: it combines `@ControllerAdvice` and `@ResponseBody`, ensuring all exception handler methods return JSON without needing additional annotations on each one.

2.3 Designing a Consistent Error Response

Spring's default error response looks different from your success responses. In a production system, clients should be able to parse errors the same way they parse successes. A consistent structure is not optional. It is the contract.

Target error response structure

```
// The structure every error response follows
{
  "timestamp": "2024-06-15T10:32:44",
  "status": 400,
  "message": "Validation failed",
  "errors": {
    "email": "Must be a valid email address",
    "name": "Name is required"
  },
  "path": "/api/users"
}
```

Every field earns its place. The timestamp helps correlate errors with logs. The status is included in the body so clients don't have to parse the HTTP header. The message is human-readable. The errors map tells you exactly which fields failed. The path identifies which endpoint triggered the error.

ApiErrorResponse.java

```
public class ApiErrorResponse {

    private LocalDateTime timestamp;
    private int status;
    private String message;
    private Map<String, String> errors;
}
```



```

private String path;

// Private constructor- use the builder
private ApiErrorResponse() {}

public static ApiErrorResponse of(int status, String message,
                                  Map<String, String> errors,
                                  String path) {
    ApiErrorResponse r = new ApiErrorResponse();
    r.timestamp = LocalDateTime.now();
    r.status     = status;
    r.message    = message;
    r.errors     = errors;
    r.path       = path;
    return r;
}

// getters
}

```

2.4 The Global Exception Handler- Complete Implementation

This is the production version. Four exception types covered explicitly, each mapped to the right HTTP status. One fallback for everything else. Logging inside, clean response outside.

GlobalExceptionHandler.java- production implementation

```

@RestControllerAdvice
public class GlobalExceptionHandler {

    private static final Logger log =
        LoggerFactory.getLogger(GlobalExceptionHandler.class);

    // Handles @Valid failures
    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ApiErrorResponse> handleValidation(
        MethodArgumentNotValidException ex,
        HttpServletRequest request) {

        Map<String, String> fieldErrors = new LinkedHashMap<>();
        ex.getBindingResult().getFieldErrors()
            .forEach(e -> fieldErrors.put(e.getField(), e.getDefaultMessage()));

        log.warn("Validation failed on {}: {}", request.getRequestURI(),
            fieldErrors);

        return ResponseEntity
            .status(HttpStatus.BAD_REQUEST)
            .body(ApiErrorResponse.of(400, "Validation failed",
                fieldErrors, request.getRequestURI()));
    }
}

```

```

// Handles malformed JSON body
@ExceptionHandler(HttpMessageNotReadableException.class)
public ResponseEntity<ApiErrorResponse> handleUnreadable(
    HttpMessageNotReadableException ex,
    HttpServletRequest request) {

    log.warn("Malformed JSON on {}: {}", request.getRequestURI(),
ex.getMessage());

    return ResponseEntity
        .status(HttpStatus.BAD_REQUEST)
        .body(ApiErrorResponse.of(400, "Malformed or unreadable request body",
            null, request.getRequestURI()));
}

// Handles wrong type in path variables or request params
@ExceptionHandler(MethodArgumentTypeMismatchException.class)
public ResponseEntity<ApiErrorResponse> handleTypeMismatch(
    MethodArgumentTypeMismatchException ex,
    HttpServletRequest request) {

    String msg = "Parameter '" + ex.getName() + "' expects type " +
        ex.getRequiredType().getSimpleName();
    log.warn("Type mismatch on {}: {}", request.getRequestURI(), msg);

    return ResponseEntity
        .status(HttpStatus.BAD_REQUEST)
        .body(ApiErrorResponse.of(400, msg, null, request.getRequestURI()));
}

// Handles custom domain exceptions (resource not found)
@ExceptionHandler(UserNotFoundException.class)
public ResponseEntity<ApiErrorResponse> handleNotFound(
    UserNotFoundException ex,
    HttpServletRequest request) {

    log.info("User not found on {}: {}", request.getRequestURI(),
ex.getMessage());

    return ResponseEntity
        .status(HttpStatus.NOT_FOUND)
        .body(ApiErrorResponse.of(404, ex.getMessage(),
            null, request.getRequestURI()));
}

// Safety net- catches anything not handled above
@ExceptionHandler(Exception.class)
public ResponseEntity<ApiErrorResponse> handleGeneral(
    Exception ex,
    HttpServletRequest request) {

    // Log the full stack trace internally- never send it to the client
    log.error("Unhandled exception on {}", request.getRequestURI(), ex);

    return ResponseEntity
        .status(HttpStatus.INTERNAL_SERVER_ERROR)
        .body(ApiErrorResponse.of(500,

```

```
        "An unexpected error occurred. Please try again later.",  
        null, request.getRequestURI());  
    }  
}
```

Senior Developer Note

The fallback handler logs the full exception with `log.error(..., ex)`- that last argument tells SLF4J to print the complete stack trace to your log file.

The client gets "An unexpected error occurred." No stack trace. No internal details.

This is the separation that matters: log everything internally, expose nothing externally.

2.5 Logging Strategy Inside Exception Handlers

Every handler in the example above logs at a different level. That is intentional.

Log levels in exception handlers

```
// WARN: Expected but notable. Validation failures, bad input.  
// Developers see these, but they do not page anyone.  
log.warn("Validation failed on {}: {}", path, fieldErrors);  
  
// INFO: Normal business events. User not found is often fine.  
// Maybe the client is checking existence before creating.  
log.info("User not found on {}: {}", path, message);  
  
// ERROR: Unexpected. Something broke that should not have.  
// The second argument after ex prints the full stack trace.  
log.error("Unhandled exception on {}", path, ex);
```

✗ Never log sensitive data in exception handlers. If the DTO contains a password field and validation fails on it, log the field name, not the value. If you log the value, you have just put a plaintext password in your log files.

2.6 HTTP Status Codes- The Right Code for Every Situation

Using the correct status code is part of the API contract. When every 200 means success and every non-200 means "check the body and guess," your API becomes harder to integrate and harder to monitor.

Code	Meaning	When to use	Exception to throw
400	Bad Request	Validation failed, malformed JSON, wrong parameter type	MethodArgumentNotValidException
401	Unauthorized	No token, expired token, invalid token	AuthenticationException (Spring Security)
403	Forbidden	Valid token, insufficient permissions for this resource	AccessDeniedException (Spring Security)
404	Not Found	Resource with the given ID does not exist	UserNotFoundException (custom)
409	Conflict	Duplicate email, duplicate username, concurrency clash	DuplicateResourceException (custom)
422	Unprocessable	JSON is valid, but business rules reject the data	BusinessValidationException (custom)
500	Server Error	Unexpected exception, unhandled state, infrastructure failure	Exception (fallback)

Senior Developer Note

400 vs 422 is a common debate. 400 means the request is structurally malformed. 422 means the request is structurally correct but the content violates business rules. For most teams, 400 for both is pragmatic and acceptable. Use 422 only if your clients actually distinguish between the two.

2.7 Production Warnings

✗ Never expose stack traces in API responses. A stack trace tells attackers your package structure, framework versions, and internal class names. Spring's default error page can leak this. Override it with your `GlobalExceptionHandler` fallback.

✗ Never rely on frontend validation alone. A browser can be bypassed with Postman, curl, or any HTTP client in seconds. Backend validation is not optional and frontend validation is not sufficient. They serve different purposes.

✗ Never return an entity from a controller. If you return an entity with a lazy-loaded collection, Jackson will trigger the collection load outside the Hibernate session. You get `LazyInitializationException`. Use DTOs.

✗ Do not swallow exceptions with empty catch blocks. If you catch an exception and do nothing, the failure becomes invisible. Log it at minimum. If you cannot handle it, rethrow it.

SECTION 3: INTERVIEW Q&A

Sharp answers to real interview questions

Q What is the difference between `@Valid` and `@Validated`?

`@Valid` is the standard Java Bean Validation annotation. It triggers constraint checking on the annotated parameter.

`@Validated` is Spring's extension. It adds two capabilities: validation groups, which allow different rules for create vs update operations, and method-level validation inside Spring beans like `@Service` classes.

In a REST controller with `@RequestBody`, `@Valid` is sufficient. Use `@Validated` when you need either of those two additional capabilities.

Q What is the difference between `@NotNull`, `@NotEmpty`, and `@NotBlank`?

`@NotNull` only checks that the value is not null. An empty string passes.

`@NotEmpty` checks for not null and not empty, but does not strip whitespace. A string of spaces passes.

`@NotBlank` checks for not null, not empty, and not whitespace-only. It is the strictest of the three and the right default for user-facing text fields.

Use `@NotNull` on non-String types. Use `@NotBlank` on String fields the user must fill in.

Q What is `@RestControllerAdvice` and why use it instead of `@ControllerAdvice`?

`@ControllerAdvice` is a global exception interceptor, but it does not assume the response format.

If you return an object from a `@ControllerAdvice` handler, Spring tries to resolve it as a view name, not as JSON. You would need `@ResponseBody` on every handler method.

`@RestControllerAdvice` combines `@ControllerAdvice` with `@ResponseBody`. Every handler method automatically returns JSON. For REST APIs, this is always the right annotation.

Q What happens if you forget `@Valid` on a controller parameter?

Nothing breaks. The application starts, requests are accepted, and data reaches the service.

But none of the validation annotations on the DTO are checked. Null values, blank strings, and invalid emails all pass through.

This is one of the most common silent bugs in Spring Boot applications. The constraints are declared but never enforced because the trigger is missing.

Q How do you handle validation errors and return a clean response?

When `@Valid` fails, Spring throws `MethodArgumentNotValidException`. You intercept it in a `@RestControllerAdvice` class with an `@ExceptionHandler` method.

Inside the handler, you call `ex.getBindingResult().getFieldErrors()` to get a list of which fields failed and why. You convert those into a map and return it inside a structured `ApiErrorResponse` with HTTP 400.

The client gets a clean JSON body with field-level error messages. The internal exception details never leave the server.

Q Why should you log the full exception in the fallback handler but not send it to the client?

The stack trace contains internal information: package names, class structures, framework versions, and sometimes data values. Sending that to a client exposes your system to reconnaissance.

Internally, you need the full stack trace to debug the problem. Externally, the client needs to know that something went wrong and that they should try again or contact support.

These are different audiences with different needs. The logging layer serves engineers. The response layer serves clients. Keep them separate.

Chapter 8 Complete

Validation and error handling are not afterthoughts. They are the difference between an API that clients can rely on and one that clients have to work around. You now have the full picture: how

validation is triggered, how exceptions propagate, how to intercept them globally, and what the response should look like at each step.

The next chapter goes deeper into the data layer: JPA, transactions, and the performance traps that only show up under real load.

The Spring Boot Playbook | Chapter 8 | Premium Edition

@JavaInterviewready

THE SPRING BOOT PLAYBOOK

From First API to Hired Developer

CHAPTER 9: ADVANCED INTERVIEW CONCEPTS

Chapter 9: Advanced Interview Concepts

This chapter is different from the earlier ones. We're not building features anymore. We're going deeper into the stuff that separates a candidate who "knows Spring Boot" from one who actually understands what's happening under the hood.

Interviewers at mid-to-senior levels ask about transactions, N+1 queries, and lazy loading not to trick you, but to see if you've thought beyond just making an API work. These are the kind of topics where one good answer shifts the entire feel of an interview.

Read this chapter slowly. Some of it takes a moment to sink in.

SECTION 1: TRANSACTIONS

Transactions and @Transactional

Here's a real scenario. You're building a payment feature. A user transfers money from Account A to Account B. Your code does two things: debit Account A, then credit Account B.

Now imagine the server crashes after the debit but before the credit. The money is gone from A but never reached B. The user is going to have a very bad day, and so are you.

This is exactly what transactions solve. A transaction says: either all of these operations succeed together, or none of them happen at all. There's no partial state.

In database terms this is called ACID (Atomicity, Consistency, Isolation, Durability). You don't need to memorise all four definitions for most interviews, but knowing what Atomicity means, all or nothing, is worth having ready.

How @Transactional Works in Spring

Spring uses a proxy around your service method. When the method starts, Spring opens a database transaction. When the method finishes without errors, Spring commits it. If any unchecked exception is thrown, Spring rolls back everything that happened inside that method.

The annotation itself is simple:

```
@Service
public class PaymentService {

    @Autowired
    private AccountRepository accountRepo;

    @Transactional
    public void transfer(Long fromId, Long toId, Double amount) {
        Account from = accountRepo.findById(fromId)
            .orElseThrow(() -> new RuntimeException("Account not found"));
        Account to = accountRepo.findById(toId)
            .orElseThrow(() -> new RuntimeException("Account not found"));

        from.setBalance(from.getBalance() - amount);
        to.setBalance(to.getBalance() + amount);

        accountRepo.save(from);
        accountRepo.save(to);
        // if anything above throws, BOTH saves are rolled back
    }
}
```

Without `@Transactional` here, if the second save fails, the first one already wrote to the database. You'd have corrupted data with no clean way to fix it.

✓ Put `@Transactional` on service methods, not on controller methods or repository methods. The service layer is where your business operations live, so that's the right boundary for your transactions.

The Rollback Gotcha

By default, Spring only rolls back on unchecked exceptions (subclasses of `RuntimeException`). If you throw a checked exception, Spring will NOT roll back unless you tell it to explicitly.

```
// This will NOT roll back by default if someCheckedException is thrown
```

```
@Transactional
public void doSomething() throws SomeCheckedException { }

// This WILL roll back for checked exceptions too
@Transactional(rollbackFor = Exception.class)
public void doSomething() throws SomeCheckedException { }
```

★ This is a real interview question. "Does `@Transactional` roll back on checked exceptions?" The answer is no, not by default. If you know this, you will visibly stand out.

SECTION 2: THE N+1 PROBLEM

The N+1 Problem

This one comes up constantly in interviews and almost every fresher either doesn't know it or explains it vaguely. Let's fix that.

Imagine you have two tables: Orders and Items. Each order has multiple items. You want to fetch all orders and display their items.

The naive code does this:

```
List<Order> orders = orderRepository.findAll(); // 1 query

for (Order order : orders) {
    List<Item> items = order.getItems(); // 1 query per order
    // if there are 100 orders, this runs 100 more queries
}
```

If you have 100 orders, you just fired 101 database queries. 1 to fetch the orders, then 1 for each order's items. That's the N+1 problem. N is the number of orders, plus 1 for the initial query.

In development with 5 rows in the database, you never notice. In production with 50,000 orders, your application slows to a crawl and your DBA starts asking questions.

Why It Happens: LAZY Loading

By default, JPA uses LAZY loading for collections. This means: when you fetch an Order, it does NOT automatically fetch that order's items. The items are loaded only when you actually access them in code.

That sounds like a good thing, load only what you need. And it usually is. But when you loop through 100 orders and access items on each one, JPA fires a separate query each time. That's where it becomes a problem.

The Fix: JOIN FETCH

The right solution is to tell JPA to load the associated data in a single query using a JOIN.

```
@Repository
public interface OrderRepository extends JpaRepository<Order, Long> {

    // Fetches orders AND items in a single query
    @Query("SELECT o FROM Order o JOIN FETCH o.items")
    List<Order> findAllWithItems();
}
```

One query. Done. The database does the join and returns everything you need in a single trip.

★ The N+1 problem is one of the most common performance issues in Java applications. Mentioning it in an interview, unprompted, when discussing JPA or Hibernate, signals that you've thought about production behaviour. Most freshers haven't.

SECTION 3: DTO MAPPING

DTO Mapping: Keeping It Practical

We covered why DTOs exist back in Chapter 7. Now let's talk about mapping, which is the act of converting between your Entity and your DTO.

It sounds boring. It kind of is. But mapping is something you do in every real project, so it's worth knowing the options.

Manual Mapping

The simplest approach. You write a method that copies fields from one object to another. No magic, no library, just code.

```
private UserDTO toDTO(User user) {
    UserDTO dto = new UserDTO();
    dto.setId(user.getId());
    dto.setName(user.getName());
    dto.setEmail(user.getEmail());
    return dto;
}

private User toEntity(UserDTO dto) {
    User user = new User();
    user.setName(dto.getName());
    user.setEmail(dto.getEmail());
    return user;
}
```

This is verbose but completely transparent. You know exactly what's being mapped and you're in full control. For smaller projects or interviews, this is perfectly fine. Don't let anyone make you feel like you need a library for everything.

MapStruct (The Industry Standard)

In real companies, you'll often see MapStruct. It generates the mapping code at compile time based on an interface you define. No reflection, no runtime overhead.

```
@Mapper(componentModel = "spring")
public interface UserMapper {

    UserDTO toDTO(User user);
    User toEntity(UserDTO dto);
}
```

MapStruct generates the implementation for you. It's fast, type-safe, and the standard choice in enterprise projects. You don't need to know it deeply for a fresher interview, but knowing it exists and what it does is a good sign of awareness.

✓ If asked "how do you convert entities to DTOs?", start with manual mapping (shows understanding), then mention MapStruct exists for larger projects (shows awareness). That answer covers both bases.

SECTION 4: PERFORMANCE BASICS

Performance: The Basics You Should Know

You don't need to be a performance engineer to ace a fresher interview. But you should know the common pitfalls. These are things that look fine in code review but destroy performance in production.

LAZY vs EAGER: Use LAZY by Default

JPA lets you control how related entities are fetched. There are two strategies:

Strategy	What It Does
<code>FetchType.LAZY</code>	Load related data only when you access it. Default for collections (<code>@OneToMany</code> , <code>@ManyToMany</code>).
<code>FetchType.EAGER</code>	Load related data immediately with the parent. Default for single objects (<code>@ManyToOne</code> , <code>@OneToOne</code>).

EAGER sounds appealing because you always have the data. But it means every time you load an entity, JPA also loads all its related entities, even when you don't need them. If a User has 200 Orders and you just want the user's name, EAGER still fetches all 200 orders.

Stick with LAZY for collections and fetch specifically when you need the related data using JOIN FETCH or separate queries. This is the right default.

Avoid Unnecessary DB Calls

This sounds obvious but it's easy to miss in the middle of writing code.

```
// BAD: Two queries when one is enough
if (userRepository.existsById(id)) {
    User user = userRepository.findById(id).get();
    // do something with user
}

// BETTER: One query, handle Optional properly
userRepository.findById(id).ifPresent(user -> {
    // do something with user
});
```

Every database call has a cost. Network latency, connection pool usage, database processing time. In a loop this multiplies fast. Always ask yourself: am I hitting the database more times than I need to?

SECTION 5: COMMON REAL-WORLD MISTAKES

Mistakes That Show Up in Real Code Reviews

These aren't hypothetical. Every single one of these is something I've seen in actual pull requests from junior developers. Knowing these makes you a better developer and a stronger interview candidate.

1. Using EAGER Everywhere

The most common JPA mistake. A developer notices lazy loading causes an error (usually a `LazyInitializationException`), switches to EAGER, and the error goes away. Problem solved, right?

No. The real fix was to load the data properly within the right transaction scope. The EAGER workaround just hides the symptom and creates a performance problem that only shows up later under load.

⚠ `LazyInitializationException` means you're trying to access lazy-loaded data outside of an active Hibernate session. The fix is not to switch to EAGER. The fix is to load what you need while the session is still open, inside your service method marked `@Transactional`.

2. Returning the Entity Directly from the Controller

We covered this in Chapter 7 but it keeps coming up because people keep doing it. When you return an entity directly, you expose your entire database structure to the client. You lose control over what gets serialized. You can also run into infinite recursion if you have bidirectional relationships (`@OneToMany` on both sides), which will crash your application with a `StackOverflowError`.

Always return a DTO. Even if it looks the same as the entity right now.

3. Not Handling Optional Properly

Repository methods like `findById` return an `Optional`, not the entity directly. `Optional` is Java's way of saying "this might be null, handle it properly."

```
// WRONG: Will throw NoSuchElementException if user doesn't exist
User user = userRepository.findById(id).get();

// RIGHT: Throw a meaningful exception
User user = userRepository.findById(id)
    .orElseThrow(() -> new UserNotFoundException(id));

// ALSO RIGHT: Return empty or default
Optional<User> user = userRepository.findById(id);
if (user.isEmpty()) {
    // handle the not-found case explicitly
}
```

Calling `.get()` on an `Optional` without checking first is an accident waiting to happen. You're basically saying "I'm sure this exists," and when it doesn't, your application throws an ugly exception with no helpful message.

4. No Validation on Incoming Data

Covered in Chapter 8, but worth mentioning here in the context of mistakes. Skipping validation is one of those things that works perfectly in development (because you're sending well-formed test data) and breaks in production (because real users send everything imaginable).

A null email in the database. A negative price. A username that's 500 characters long. All of these happen if you don't validate. Add `@Valid` and the appropriate constraints on every incoming DTO. It takes five minutes and prevents a category of bugs entirely.

5. Business Logic in the Controller

The controller's job is to receive a request and return a response. That's it. When developers start adding if statements, calculations, and database calls inside controller methods, the code becomes untestable, unreadable, and impossible to reuse.

If you're writing more than two or three lines of logic in a controller method, that logic probably belongs in the service layer.

MINI PROJECT: PUTTING IT ALL TOGETHER

Mini Project: A Production-Ready User API

Everything we've covered across Chapters 7, 8, and 9 comes together here. This is a complete User API that you can build, run, and show in an interview. It's small enough to understand in one sitting and structured well enough to look professional.

We're going to go through each file in order, from the Entity down to the Controller, with everything connected properly.

Project Structure

```
src/main/java/com/example/userapi/  
  entity/  
    User.java  
  dto/  
    UserRequestDTO.java  
    UserResponseDTO.java  
  repository/  
    UserRepository.java  
  service/  
    UserService.java  
  controller/  
    UserController.java  
  exception/
```

```
UserNotFoundException.java
GlobalExceptionHandler.java
```

1. The Entity

```
@Entity
@Table(name = "users")
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String name;

    @Column(nullable = false, unique = true)
    private String email;

    @Column(nullable = false)
    private String passwordHash;

    private LocalDateTime createdAt;

    @PrePersist
    public void prePersist() {
        this.createdAt = LocalDateTime.now();
    }

    // getters and setters
}
```

2. The DTOs

Separate request and response DTOs. This is the right approach in production code.

```
// What comes IN from the client
public class UserRequestDTO {

    @NotBlank(message = "Name is required")
    @Size(min = 2, max = 50)
    private String name;

    @NotBlank(message = "Email is required")
    @Email(message = "Must be a valid email")
    private String email;
```

```

@NotBlank(message = "Password is required")
@Size(min = 8, message = "Password must be at least 8 characters")
private String password;

// getters and setters
}

```

```

// What goes OUT to the client (no password, ever)
public class UserResponseDTO {

    private Long id;
    private String name;
    private String email;
    private LocalDateTime createdAt;

    // getters and setters
}

```

3. The Repository

```

@Repository
public interface UserRepository extends JpaRepository<User, Long> {

    boolean existsByEmail(String email);
    Optional<User> findByEmail(String email);
}

```

4. The Custom Exception

```

public class UserNotFoundException extends RuntimeException {
    public UserNotFoundException(Long id) {
        super("No user found with id: " + id);
    }
}

```

5. The Service

All business logic lives here. Notice the `@Transactional` on the write operation.

```

@Service
public class UserService {

    @Autowired
    private UserRepository userRepository;

    @Transactional
    public UserResponseDTO createUser(UserRequestDTO dto) {
        if (userRepository.existsByEmail(dto.getEmail())) {
            throw new RuntimeException("Email already in use");
        }

        User user = new User();
        user.setName(dto.getName());
        user.setEmail(dto.getEmail());
        // In real code, hash the password with BCrypt
        user.setPasswordHash(dto.getPassword());

        User saved = userRepository.save(user);
        return toResponseDTO(saved);
    }

    public List<UserResponseDTO> getAllUsers() {
        return userRepository.findAll()
            .stream()
            .map(this::toResponseDTO)
            .collect(Collectors.toList());
    }

    public UserResponseDTO getUserById(Long id) {
        User user = userRepository.findById(id)
            .orElseThrow(() -> new UserNotFoundException(id));
        return toResponseDTO(user);
    }

    private UserResponseDTO toResponseDTO(User user) {
        UserResponseDTO dto = new UserResponseDTO();
        dto.setId(user.getId());
        dto.setName(user.getName());
        dto.setEmail(user.getEmail());
        dto.setCreatedAt(user.getCreatedAt());
        return dto;
    }
}

```

6. The Global Exception Handler

```

@ControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<Map<String, String>> handleValidation(
        MethodArgumentNotValidException ex) {
        Map<String, String> errors = new HashMap<>();
    }
}

```

```

        ex.getBindingResult().getFieldErrors()
            .forEach(e -> errors.put(e.getField(), e.getDefaultMessage()));
        return ResponseEntity.badRequest().body(errors);
    }

    @ExceptionHandler(UserNotFoundException.class)
    public ResponseEntity<Map<String, String>> handleNotFound(
        UserNotFoundException ex) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND)
            .body(Map.of("error", ex.getMessage()));
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<Map<String, String>> handleGeneral(Exception ex) {
        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
            .body(Map.of("error", "Something went wrong"));
    }
}

```

7. The Controller

Thin, clean, no logic. Just HTTP handling.

```

@RestController
@RequestMapping("/api/users")
public class UserController {

    @Autowired
    private UserService userService;

    @PostMapping
    public ResponseEntity<UserResponseDTO> create(
        @Valid @RequestBody UserRequestDTO dto) {
        UserResponseDTO created = userService.createUser(dto);
        return ResponseEntity.status(HttpStatus.CREATED).body(created);
    }

    @GetMapping
    public ResponseEntity<List<UserResponseDTO>> getAll() {
        return ResponseEntity.ok(userService.getAllUsers());
    }

    @GetMapping("/{id}")
    public ResponseEntity<UserResponseDTO> getById(@PathVariable Long id) {
        return ResponseEntity.ok(userService.getUserById(id));
    }
}

```

★ This mini project demonstrates layered architecture, DTO separation, validation, custom exceptions, global error handling, and correct use of `@Transactional`. If you can explain every file and why it's written the way it is, you're ready for most fresher Spring Boot interviews.

Quick Interview Reference

If you have five minutes before an interview, read this section.

What does `@Transactional` do?

It wraps the method in a database transaction. If the method completes successfully, the transaction commits. If any `RuntimeException` is thrown, everything rolls back.

What is the N+1 problem?

When fetching N parent records triggers N additional queries to fetch their related data. Fixed by using JOIN FETCH to load everything in a single query.

LAZY vs EAGER?

LAZY loads related data only when accessed. EAGER loads it immediately with the parent. Use LAZY for collections by default and fetch explicitly when needed.

Why DTOs instead of entities?

To control what the client sees, to decouple your API from your database structure, and to avoid accidentally exposing sensitive fields or causing serialization issues with bidirectional relationships.

Does `@Transactional` roll back on checked exceptions?

No. Only on `RuntimeException` and its subclasses by default. To roll back on checked exceptions, use `@Transactional(rollbackFor = Exception.class)`.

Chapter 9 Complete

That's the end of Part 2. You've gone from building a basic API to understanding transactions, performance traps, and common mistakes that show up in real code. The mini project ties everything together into something you can actually run and talk about.

Part 3 moves into Spring Security, JWT authentication, and a few other topics that come up in mid-level interviews. If you're going for a fresher role, everything up to this point is your core preparation.

The Spring Boot Playbook | Chapter 9 | Premium Edition

@JavaInterviewready

CHAPTER 10

Spring Security & JWT

Locking the door, checking the badge, and letting the right people through.

Honestly, Spring Security is the topic most freshers are scared of. And I get it - the first time you add the spring-boot-starter-security dependency, your entire app suddenly returns 401 Unauthorized for every endpoint. No warning. Just locked out.

But once you understand the actual idea behind it, the fear goes away. Security is really just two questions: **who are you?** and **what are you allowed to do?** Everything in this chapter answers one of those two questions.

1. Authentication vs Authorization

Picture an office building. At the entrance, security checks your ID badge. They verify you are who you say you are - that's **Authentication**. You prove your identity.

Once you're inside, you swipe your badge to enter a specific floor. Not every floor lets you in - maybe only engineers can enter the server room, and only managers can access the boardroom. That's **Authorization**. You already proved who you are - now the system decides what you're allowed to do.

Concept	Simple Definition
Authentication	Verifying WHO you are. Login with username + password is authentication.
Authorization	Deciding WHAT you can do. Checking if a logged-in user can access a resource.

In a Spring Boot app - a user logs in with their email and password. Spring Security verifies those credentials against the database. That's authentication. After login, when that user tries to hit `/admin/dashboard`, Spring Security checks their role. If they're not an ADMIN, they get a 403 Forbidden. That's authorization.

The mistake beginners make all the time: they use these two words interchangeably in interviews. Don't. The moment you separate them clearly, the interviewer knows you actually get it.



INTERVIEW QUESTION

What is the difference between Authentication and Authorization?

How to answer: Authentication is the process of verifying identity - confirming who you are, typically via credentials like username and password. Authorization is what happens after that - it determines what the authenticated user is permitted to do. Technically, a 401 Unauthorized response means the request lacks valid credentials (the user is not authenticated). A 403 Forbidden means the user is authenticated but does not have permission for that resource. One thing interviewers check: 401 doesn't mean authentication 'failed' - it means credentials were absent or unrecognizable. A wrong password should still return 401, not 403. Authentication is the process of verifying a user's identity - confirming who they are, usually via username and password. Authorization is what happens after authentication - it determines what resources or actions the verified user is allowed to access. A 401 error means authentication failed. A 403 error means authorization failed.

2. What Is JWT?

JWT stands for **JSON Web Token**. At its core, it's just a string - but a very specific kind of string that carries information securely between two parties.

Here's what a JWT actually looks like:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9
.eyJzdWIiOiJlcnVycGVtYWlsLmNvbSIsInJvbGUiOiJVU0VSIiwiaWF0IjoxNjk
.SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c

// Three parts, separated by dots (.)
// Header.Payload.Signature
```

The Three Parts - Briefly

Header: Contains the token type (JWT) and the signing algorithm. The most common is HS256 - a symmetric algorithm where the same secret key is used to both sign and verify the token. You'll also see RS256 in enterprise systems - asymmetric, where a private key signs and a public key verifies. It's Base64Url encoded, not encrypted. Contains the token type (JWT) and the hashing algorithm used. Usually HS256 (HMAC with SHA-256). It's Base64 encoded - not encrypted. Anyone can decode it.

Payload: The actual data, called claims. Standard claims include sub (subject - usually user ID or email), iat (issued at timestamp), exp (expiry timestamp), and roles. Also Base64Url encoded - not encrypted. Anyone who holds the token can decode this. Keep it minimal: user ID, email, role, expiry. Nothing sensitive. This is where the actual data lives. Also called **claims**. Things like the user's ID, email, role, and when the token expires (**exp**). Again - Base64 encoded, not encrypted. Do NOT put passwords or sensitive data here.

Signature: The server signs the Base64Url-encoded Header and Payload using the secret key and the algorithm specified in the header. The result is the signature. On every subsequent request, the server re-computes the signature from the incoming token's header and payload and checks if it matches. If someone tampers with the payload - say, changes role from USER to ADMIN - the signature won't match and the token is rejected. This is the integrity guarantee JWT provides. This is the important part. The server takes the Header + Payload, combines them with a secret key, and runs it through the hashing algorithm. The result is the signature. When the server receives a token later, it re-computes this signature and checks if it matches. If someone tampered with the payload, the signature won't match and the token gets rejected.

JWT is not encryption - it is signing. The Header and Payload are Base64Url encoded (a URL-safe variant of Base64). Anyone holding the token can decode them instantly - paste any JWT into jwt.io and you'll see the full payload in plain text. The Signature only proves the token hasn't been tampered with. It does NOT hide the data inside.

3. Why JWT Instead of Sessions?

Old-school web apps used server-side sessions. You log in, the server creates a session, stores it in memory (or a database), and gives you a session ID as a cookie. Every request, the server looks up that session ID to identify you.

This works fine for a single server. But modern applications run on multiple servers - sometimes dozens. If you log in on Server A and your next request lands on Server B, that server has no idea who you are. Your session doesn't exist there.

JWT solves this differently. Think of it like a **concert wristband**. When you enter the concert, the staff puts a wristband on you. Now anywhere inside the venue, any security person can look at your wristband and know you're a valid ticket holder - without calling headquarters to verify. The wristband carries all the information they need.

JWT works the same way. After you log in, the server gives you a token. You store it on the client. Every request, you send the token along in the Authorization header. Any server can verify it using the secret key - no central session storage needed. That's stateless. **stateless** means.

Session-Based Auth	JWT-Based Auth
Server stores session data	Server stores nothing - token is self-contained
Session ID sent in cookie	JWT sent in Authorization header
Hard to scale across multiple servers	Works with any number of servers
Logout is easy - just delete session	Logout needs extra work - token stays valid until expiry (solved with refresh token rotation or blacklist)

That last row is worth spending a moment on in interviews. JWT's main weakness is that you can't truly invalidate a token once issued - it remains valid until the expiry time. The industry-standard solution is the access token + refresh token pattern. You issue a short-lived access token (5-15 minutes) and a longer-lived refresh token (days or weeks). When the access token expires, the client uses the refresh token to get a new one - without logging in again. On logout, you invalidate the refresh token. The short access token lifetime limits the damage window if one is stolen. Some teams also maintain a token blacklist (stored in Redis) for immediate revocation, but that adds stateful infrastructure.



INTERVIEW QUESTION

Why do we use JWT instead of session-based authentication in REST APIs?

How to answer: REST APIs are stateless by design. Session-based auth requires the server to store session data, which doesn't scale well across multiple instances. JWT is self-contained - the token carries user information and the server just validates the signature on each request without maintaining any state. This makes JWT ideal for distributed systems and microservices.

Where Should You Store the JWT?

This is a question interviewers ask more often now, and most candidates have no answer. There are two common places to store a JWT on the client side, and each has a real trade-off.

localStorage is easy to use. Your JavaScript can read it and attach it to every request. But it is accessible to any JavaScript running on the page - which means an XSS (Cross-Site Scripting) attack can steal it. If someone injects a script into your page, your token is gone.

HttpOnly cookies store the token in a cookie that JavaScript cannot read at all. The browser sends it automatically with every request. XSS attacks cannot steal it because JS has no access. The trade-off: you now need to handle CSRF protection again (use SameSite=Strict or the double-submit cookie pattern).



TIP

Industry preference: HttpOnly + Secure + SameSite=Strict cookies. This gives you XSS protection without localStorage's exposure. Never store tokens in sessionStorage either - same XSS risk as localStorage.

4. The JWT Flow - Step by Step

This is probably the single most-asked topic in Spring Security interviews. If you can walk through this flow clearly, you'll impress most interviewers at the fresher level. Practice saying it out loud.

JWT AUTHENTICATION FLOW

Step 1: Client sends a POST request to /auth/login with username and password in the request body.

Step 2: Server receives the credentials. Spring Security's AuthenticationManager verifies them against the database (via UserDetailsService).

Step 3: If credentials are valid, the server generates a JWT using a secret key. The token contains user info (email, role) and an expiry time.

Step 4: Server sends the JWT back to the client in the response body.

Step 5: Client stores the token - in memory, an HttpOnly cookie, or localStorage (with XSS risk awareness).

Step 6: For every subsequent request, the client includes the token in the Authorization header: Bearer <token>

Step 7: Server intercepts the request through a filter (JwtAuthFilter), extracts the token, validates the signature and expiry, and sets the user in the security context.

Step 8: The request proceeds to the controller. Spring Security already knows who the user is.

Here is what that login endpoint and token generation look like in code:

```
// AuthController.java
@RestController
@RequestMapping("/auth")
public class AuthController {

    @Autowired
    private AuthenticationManager authManager;

    @Autowired
    private JwtUtil jwtUtil;

    @PostMapping("/login")
    public ResponseEntity<String> login(@RequestBody LoginRequest req) {
        // Step 1: authenticate the user
        Authentication auth = authManager.authenticate(
            new UsernamePasswordAuthenticationToken(
                req.getEmail(), req.getPassword()
            )
        );
        // Step 2: generate token
        String token = jwtUtil.generateToken(auth.getName());
        return ResponseEntity.ok(token);
    }
}
```

And then on every request, your `JwtAuthFilter` intercepts before the request reaches the controller:

```
// JwtAuthFilter.java (simplified)
@Component
public class JwtAuthFilter extends OncePerRequestFilter {

    @Override
    protected void doFilterInternal(HttpServletRequest request,
        HttpServletResponse response, FilterChain filterChain)
        throws ServletException, IOException {

        String authHeader = request.getHeader("Authorization");

        if (authHeader != null && authHeader.startsWith("Bearer ")) {
            String token = authHeader.substring(7); // remove 'Bearer '
            String email = jwtUtil.extractEmail(token);
        }
    }
}
```

```

        if (email != null && jwtUtil.isTokenValid(token)) {
            UsernamePasswordAuthenticationToken authToken =
                new UsernamePasswordAuthenticationToken(
                    email, null, List.of(new SimpleGrantedAuthority("USER")))
                );
            SecurityContextHolder.getContext().setAuthentication(authToken);
        }
    }
    filterChain.doFilter(request, response);
}
}

```



TIP

When explaining this in interviews, mention `SecurityContextHolder` - that's where Spring Security stores the authenticated user for the current request. It's `ThreadLocal`-scoped, meaning each request thread has its own isolated security context. One thread cannot accidentally see another user's authentication. If no valid token is found, it stays empty, and the request gets a 401.

5. How Spring Security Works Under the Hood

You don't need to know every detail here. But you do need to understand the basic idea - because interviewers love asking this to see if you're just copy-pasting code or actually understand the flow.

Every HTTP request in a Spring Boot app goes through a **filter chain** before it reaches your controller. Spring Security inserts its own filters into this chain. Your `JwtAuthFilter` is one of them.

```

HTTP Request
↓
Spring Security Filter Chain
↓
JwtAuthFilter ← your custom filter runs here
    ↓ (validates token, sets user in SecurityContext)
UsernamePasswordAuthenticationFilter
↓
ExceptionTranslationFilter ← handles 401 / 403
↓
FilterSecurityInterceptor ← checks authorization rules
↓
Your Controller ← only reaches here if everything passed

```

The key thing: **your controller never sees an invalid request**. Security is handled before your business logic ever runs. If the token is missing, expired, or tampered with - the filter chain sends back a 401 and the request stops there.



INTERVIEW QUESTION

How does Spring Security process an incoming HTTP request?

How to answer: Spring Security uses a filter chain that intercepts every request before it reaches the controller. Filters run in a specific order. For JWT-based security, a custom filter extracts and validates the token early in the chain. If valid, it populates the SecurityContext with the user's authentication. If not, the request is rejected with a 401 before ever reaching any controller method.

6. Security Configuration - The Important Parts

When you configure Spring Security for a REST API, your `SecurityFilterChain` bean does most of the work. There are two specific things in there that interviewers often ask about.

Why We Disable CSRF for REST APIs

CSRF (Cross-Site Request Forgery) is an attack where a malicious website tricks a logged-in user's browser into making unintended requests. Spring Security enables CSRF protection by default.

But CSRF protection works by generating a token tied to the user's browser session - and sharing it via cookies. REST APIs don't use sessions or cookies like that. Clients (mobile apps, other services, Postman) send JWT in the Authorization header instead. CSRF protection doesn't apply here and would just break everything.

So: **disable CSRF for REST APIs, keep it enabled for traditional web apps with forms.**

Stateless Session Policy

By default, Spring Security creates and maintains HTTP sessions. For JWT-based APIs, we don't want that. We tell Spring Security to never create a session - to treat every request as if it came from a stranger:

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http
        .csrf(csrf -> csrf.disable()) // no CSRF for REST
        .sessionManagement(session -> session
            .sessionCreationPolicy(SessionCreationPolicy.STATELESS) // no sessions
        )
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/auth/**").permitAll() // login endpoint is open
            .anyRequest().authenticated() // everything else needs
        )
        .addFilterBefore(jwtAuthFilter,
            UsernamePasswordAuthenticationFilter.class); // add our JWT filter

    return http.build();
}
```

Read that config carefully. Every line has a reason. `permitAll()` on `/auth/**` means login doesn't require a token - obviously, because you don't have one yet. `anyRequest().authenticated()` locks everything else behind auth. And `addFilterBefore` registers your JWT filter at the right position in the chain.

7. Role-Based Access Control

Once authentication is working, you'll want different users to have different access. A USER can see their own orders. An ADMIN can see all orders, delete users, update prices. That's role-based authorization.

The simplest way in Spring Boot is `@PreAuthorize`. You put it on a controller method and specify which roles can call it.

```
@RestController
@RequestMapping("/api")
public class AdminController {

    // Only ADMIN can access this
    @PreAuthorize("hasRole('ADMIN')")
    @GetMapping("/admin/users")
    public List<User> getAllUsers() {
        return userService.getAll();
    }

    // Any authenticated user can access this
    @PreAuthorize("hasAnyRole('USER', 'ADMIN')")
    @GetMapping("/my-profile")
    public User getProfile(@AuthenticationPrincipal String email) {
        return userService.findByEmail(email);
    }
}
```

For `@PreAuthorize` to work, you need `@EnableMethodSecurity` on your config class. It's easy to forget this and then wonder why the annotation isn't doing anything.

```
@Configuration
@EnableMethodSecurity // ← don't forget this
public class SecurityConfig {
    // ...
}
```

You can also define role-based rules directly in the filter chain using `.requestMatchers("/admin/**").hasRole("ADMIN")` - but `@PreAuthorize` is more flexible and keeps the access logic close to the code it protects. Teams use both. Know both exist.

8. Mistakes People Actually Make

These aren't invented examples. These are real things that come up in codebases - and sometimes in interviews when they describe a security bug and ask you to spot it.

Storing Sensitive Data in the Token

The JWT payload is Base64 encoded - not encrypted. Anyone who gets the token can decode it and read the payload in about two seconds. Paste a JWT into jwt.io and you'll see the full payload in plain text.

People sometimes store passwords, credit card info, or other sensitive data in the payload thinking it's safe. It's not. Store only what's needed - user ID, email, role, expiry. Nothing sensitive.

Not Validating the Token Properly

Token Invalidation - The Real Production Problem

Every serious interview eventually gets here. JWT is stateless - so how do you log a user out? How do you revoke a compromised token immediately? There is no single perfect answer. Here are the three approaches teams actually use:

Short expiry + refresh tokens. Issue access tokens that expire in 5-15 minutes. Use a longer-lived refresh token (stored securely, often in an `HttpOnly` cookie) to get new access tokens silently. On logout, delete or revoke the refresh token. This is the most common real-world approach.

Token blacklist. Store revoked token IDs (the `jti` claim) in a fast store like Redis with TTL equal to the token's remaining lifetime. Check the blacklist on every request. Downside: you're adding stateful infrastructure - which partially defeats JWT's purpose. Use only when immediate revocation is a hard requirement.

Refresh token rotation. Every time a refresh token is used, it is replaced with a new one. If an old refresh token is used again, it signals theft and you invalidate the entire refresh token family for that user. This is the approach recommended by OAuth 2.0 security best practices.

⚠ WATCH OUT

The `jti` (JWT ID) claim is a unique identifier you can add to every token. If you're implementing a blacklist, this is the value you store - not the full token.

The signature check is only one part of validation. A production-grade validator should check all of these: the signature is valid, the token has not expired (`exp` claim), the issuer matches (`iss` claim - important in multi-service systems), and the token was intended for this service (`aud` claim). Most freshers only check signature and expiry. Mentioning `iss` and `aud` validation in an interview is a strong signal.**expired**. A lot of beginners validate the signature but forget to check the `exp` claim. Expired tokens should be rejected, even if the signature is valid.

```
// What proper token validation looks like:
public boolean isValidToken(String token, String expectedEmail) {
    String email = extractEmail(token);
    return (email.equals(expectedEmail) && !isTokenExpired(token));
}
```

```
//
```

```
^^^^ this part matters
```

```
}
```

Confusing Authentication and Authorization Errors

A 401 Unauthorized means: the request lacks valid authentication credentials. The name is misleading (it should be called '401 Unauthenticated') but that's what RFC 7235 gives us. A 403 Forbidden means: we know who you are, but you don't have permission. Spring Security gets this right by default:

AuthenticationException triggers 401, AccessDeniedException triggers 403. The ExceptionTranslationFilter handles both. One common mistake: returning 403 when no token is present. That's wrong. No token means unauthenticated, which is 401.

WATCH OUT

Don't expose internal error messages in security exceptions.

If token validation fails, return a simple 401 with a generic message.

Detailed error messages ('token expired', 'invalid signature', 'user not found') can help attackers understand how your auth works.

Log the detail internally - but keep the response generic.



BEFORE THE NEXT CHAPTER

Build a small project: two endpoints - one public (/auth/login) and one protected (/api/profile). Implement JWT generation on login and JWT validation via a filter. Test it in Postman - first without the token (should get 401), then with it (should get through). If you can build that from scratch, you understand this chapter. Theory is good, but this is the thing that actually sticks.

THE SPRING BOOT PLAYBOOK

From First API to Hired Developer

CHAPTER 11: TESTING BASICS

Testing Basics

Because 'it worked on my machine' is not an answer interviewers want to hear.

Most freshers treat testing like homework they'll do later. Later never comes. And then in the interview, when someone asks "how do you verify your code works?", they say "I test it in Postman."

That's fine for manual checking. But it's not what teams mean when they say testing. Real teams write automated tests. Code that tests other code. And interviewers notice when you know this vs. when you don't.

This chapter is not about becoming a testing expert. It's about knowing enough to write a basic test, explain what you're doing, and not look lost when the interviewer asks about it. That's a realistic goal. Let's hit it.

1. Why This Chapter Exists

Here's a thing that doesn't get said enough: testing is one of the fastest ways to signal that you're job-ready. Most freshers skip it entirely. So if you walk into an interview and can talk about unit tests, mocking, and `@WebMvcTest` vs `@SpringBootTest` with actual understanding, you're already ahead of 70% of the competition.

Real development teams write tests because software breaks. Requirements change. Someone refactors a method and doesn't realize it affected three other things. Automated tests catch that. Without them, you find bugs in production- which is the worst place to find them.

You don't need to write perfect tests on day one. You need to understand what they're for, how to write a simple one, and not flinch when someone brings it up in a code review.

i If you can write a unit test for a service method and a controller test with `@WebMvcTest`, you are already doing better than most junior candidates interviewing at the same companies.

2. Unit Testing vs Integration Testing

Think of building a car. Unit testing is checking each individual part- does the engine start on its own? Does the braking system work independently? Integration testing is putting all the parts together and checking that the car actually moves and stops when it should.

In code terms:

Unit Test	Integration Test
Tests one class or method in isolation	Tests multiple components working together
Dependencies are replaced with mocks	Real dependencies- DB, services- are used (or near-real)
Runs in milliseconds, no Spring context needed	Slower- starts Spring context, sometimes a real DB
Easy to pinpoint failures	Failure could come from anywhere in the chain
Example: testing a Service method alone with Mockito	Example: full HTTP request through controller to DB

The practical rule most teams follow: write a lot of unit tests, fewer integration tests. Unit tests are cheap and fast. Integration tests are slower and heavier. You want a mix- mostly unit, some integration for the key flows.

✓ The Testing Pyramid- the distribution that matters in interviews:

Unit tests - 70% or more. Fast, isolated, cheap to run in CI.

Integration tests- 20%. Tests that a request flows correctly through the stack.

End-to-end tests - 10%. Full browser or API-level tests. Slow and fragile.

When an interviewer asks "how do you structure your test suite?", this pyramid is the answer.

Inverted pyramid (lots of E2E, few unit tests) is a warning sign in any codebase.

Interview Q What is the difference between unit testing and integration testing?

Unit testing isolates a single component- usually a class or method- and tests it independently. Dependencies are mocked with Mockito. Integration testing checks how multiple components work together in a real or near-real environment.

Unit tests are fast and targeted. Integration tests are slower but give you confidence that the whole system connects properly.

In Spring Boot: I use Mockito with `@ExtendWith(MockitoExtension.class)` for service-level unit tests, `@WebMvcTest` for controller-level tests with the service mocked out, and `@SpringBootTest` for end-to-end integration tests. I follow the testing pyramid- more unit tests, fewer integration tests.

3. What Should You Actually Test?

This is the question nobody tells freshers clearly. And it matters, because testing everything equally is a waste of time, and testing the wrong things gives you false confidence.

Test Your Service Layer First

The service layer is where your business logic lives. If logic is wrong there, your whole app is wrong. This is where unit tests give you the most value.

Test the happy path- the normal case where everything works. Then test edge cases. What if the user doesn't exist? What if the list is empty? What if a value is null? Error cases are where bugs actually hide, and they're the ones most beginners skip.

Test Your Controller Responses

You want to make sure your API returns the right status codes and response bodies. If a product is not found, does it return 404? If the input is invalid, does it return 400? These are testable things- and they break more often than you'd think when someone refactors the controller.

The One Most People Skip

Error handling. Most beginners write tests for the happy path and call it done. But what happens when your service throws an exception? Does the controller return 500? Or did you handle it properly and return 400 with a message?

Test error cases deliberately. An interviewer who sees "I also test error scenarios and edge cases" in your test file knows you think like a professional.

✓ A simple rule: for every service method, ask three questions.

1. What happens when everything goes right? (happy path)
2. What happens when the input is invalid or missing?
3. What happens when a dependency returns nothing or throws?

Write at least one test for each answer.

4. Mocking- The Part That Makes Unit Tests Possible

Here's the problem with testing a service class in isolation. The service depends on a repository. The repository talks to a database. If you want to test just the service, you'd need a running database. That's slow, complicated, and defeats the point of a unit test.

Mocking solves this. A mock is a fake version of a dependency that you control completely. You tell it: "when this method is called with this input, return this value." Your service doesn't know the difference. It calls the method, gets the response, and continues. You test the logic- not the database.

Think of crash-test dummies in car safety testing. You don't use real people. You use a dummy that behaves like a person- same weight, same response to impact- but nobody gets hurt. Mocks are your crash-test dummies.

The most common mocking library in the Java world is Mockito. Spring Boot's test starter includes it by default. You don't need to add anything extra.

Here's the simplest version of mocking in action:

Basic Mockito unit test

```
// You're testing this service method:
public Product getProductById(Long id) {
    return productRepository.findById(id)
        .orElseThrow(() -> new RuntimeException("Product not found"));
}

// The test:
@ExtendWith(MockitoExtension.class)
class ProductServiceTest {

    @Mock
    private ProductRepository productRepository;

    @InjectMocks
```

```

private ProductService productService;

@Test
void getProductById_shouldReturnProduct_whenFound() {
    // Arrange
    Product fakeProduct = new Product(1L, "Laptop", 75000);
    when(productRepository.findById(1L))
        .thenReturn(Optional.of(fakeProduct));

    // Act
    Product result = productService.getProductById(1L);

    // Assert
    assertThat(result.getName()).isEqualTo("Laptop");
}
}

```

Walk through this test carefully. `@Mock` creates a fake `productRepository`. `@InjectMocks` creates a real `productService` and injects the mock into it. `when(...).thenReturn(...)` scripts the fake behavior. Then you call the real method and assert the output.

The database never runs. The test is purely about whether the service logic works correctly.

5. @SpringBootTest vs @WebMvcTest

There are two annotations you'll use constantly in Spring Boot testing. They sound similar. They work very differently. And picking the wrong one costs you time- because loading the wrong context can slow your test suite from seconds to minutes at scale.

@SpringBootTest

This loads the entire Spring application context. Every bean, every configuration, every component. It's the closest thing to running your actual app in a test. Use this when you want to test how everything works together from top to bottom- a real integration test.

The downside is speed. Loading the full context takes time. If you're just testing one controller, you're paying the cost of loading your entire application for no reason.

i `@SpringBootTest` by default does NOT start a real embedded server. It wires up the full context but uses `MockMvc` to simulate HTTP requests. If you need a real server (for testing with `RestTemplate`, for example), add `webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT`. Most controller tests do not need this.

@SpringBootTest with MockMvc

```
@SpringBootTest
@AutoConfigureMockMvc // gives you MockMvc without a real server
class ProductIntegrationTest {

    @Autowired
    private MockMvc mockMvc;

    @Test
    void getProduct_shouldReturn200() throws Exception {
        mockMvc.perform(get("/api/products/1"))
            .andExpect(status().isOk());
    }
}
```

@WebMvcTest

This loads only the web layer- controllers, filters, and MVC configuration. Nothing else. No service beans. No repositories. No database. It's lean and fast, and that's the point.

Because it doesn't load services, you mock them. You bring in exactly the dependencies your controller needs, tell them what to return, and test that your controller handles the response correctly.

@WebMvcTest- fast controller-level test

```
@WebMvcTest(ProductController.class) // only loads this controller
class ProductControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @MockBean // mocks the service in the Spring context
    private ProductService productService;

    @Test
    void getProduct_shouldReturn200_whenProductExists() throws Exception {
        Product product = new Product(1L, "Laptop", 75000.0);
        when(productService.getProductById(1L)).thenReturn(product);

        mockMvc.perform(get("/api/products/1"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.name").value("Laptop"));
    }

    @Test
    void getProduct_shouldReturn404_whenNotFound() throws Exception {
        when(productService.getProductById(99L))
            .thenReturn(new ProductNotFoundException("Not found"));

        mockMvc.perform(get("/api/products/99"))
            .andExpect(status().isNotFound());
    }
}
```

```
}  
}
```

Notice the second test- testing the error case. That 404 test is the kind of thing that separates a thoughtful developer from someone who only checks if things work when they're supposed to work.

✓ Other Spring test slices worth knowing for interviews:

@DataJpaTest - loads only JPA layer (repositories, entity config).

Uses an in-memory H2 database by default.

Ideal for testing custom **@Query** methods and repository logic.

@RestClientTest- loads only the client-side HTTP infrastructure.

Useful for testing classes that call external REST APIs.

You don't need to master these now. Knowing they exist and what they target is enough for a fresher interview.

@WebMvcTest	@SpringBootTest
Loads only the web layer	Loads the full Spring application context
Fast- no DB, no service beans initialized	Slower- everything gets initialized
Dependencies must be @MockBean'd	Real beans are wired together
Best for controller-level unit tests	Best for end-to-end integration tests
Does not test real service or DB logic	Can test full request-to-database flow

Interview Q What is the difference between @SpringBootTest and @WebMvcTest?

@SpringBootTest loads the full application context- every bean, every layer- making it suitable for true integration tests.

@WebMvcTest loads only the web layer and is much faster. You mock the service layer with **@MockBean** and test the controller's behavior in isolation. Real services and repositories are not loaded.

For unit testing a controller, `@WebMvcTest` is the right choice. `@SpringBootTest` is for when you need to verify the entire request-response-database cycle together.

There is also `@DataJpaTest` for testing repositories in isolation against an in-memory database, without loading controllers or services.

6. Mockito- The Three Things You Need to Know

Mockito is the mocking library. You'll use it in almost every unit test you write. The good news is you only really need three things to get started.

`@Mock` and `@InjectMocks`

You already saw these above. `@Mock` creates a fake version of a class. `@InjectMocks` creates the real class you're testing and automatically injects all the `@Mock`-annotated fields into it. Simple contract: mock the dependencies, inject them into the class under test.

One thing people forget: you need `@ExtendWith(MockitoExtension.class)` at the top of the test class for these annotations to work without Spring. Without that line, your mocks are null and you get a `NullPointerException` that makes no sense.

`when(...).thenReturn(...)`

This is how you script the mock's behavior. You're telling Mockito: when this specific method is called with this specific argument, return this value. The real method is never called. Your fake one is.

Scripting mock behavior

```
// Script the mock
when(userRepository.findByEmail("test@email.com"))
    .thenReturn(Optional.of(new User("test@email.com", "Rahul")));

// For void methods that should throw
doThrow(new RuntimeException("DB error"))
    .when(userRepository).save(any());
```

i Mockito matchers- used when you don't care about the exact argument value:

`any()` - matches any argument of any type
`any(User.class)` - matches any User instance
`eq("exact-value")` - matches only this exact value
`anyString()` - matches any non-null String

Example:

```
when(repository.save(any(User.class))).thenReturn(savedUser);
```

Use `eq()` when the exact value matters. Use `any()` when you only care that the method was called, not what was passed.

ArgumentCaptor- captures what was actually passed to a mock:

```
ArgumentCaptor<User> captor = ArgumentCaptor.forClass(User.class);  
verify(repository).save(captor.capture());  
assertThat(captor.getValue().getEmail()).isEqualTo("test@email.com");
```

verify(...)

Sometimes you don't just want to check the return value- you want to confirm that a certain method was actually called. `verify()` does this.

verify()- confirming interactions

```
// After calling the service method, verify the repo was called once  
verify(productRepository, times(1)).save(any(Product.class));  
  
// Or verify something was never called  
verify(emailService, never()).sendWelcomeEmail(any());
```

This is useful when testing methods that don't return anything- like a delete operation or a notification dispatch. The only thing you can verify is whether the right downstream call happened.

⚠ In `@WebMvcTest`, use `@MockBean` instead of `@Mock`. `@Mock` works with `MockitoExtension` outside the Spring context. `@MockBean` registers the mock inside the Spring application context so it gets injected into controllers properly. Using the wrong one is a very common mistake- the test runs but the mock doesn't get injected where you think it does.

7. Putting It Together- A Full Service Test

Here's a complete, realistic test class for a service. Read it top to bottom. This is the kind of thing you should be able to write from scratch after reading this chapter.

UserServiceTest.java- three tests, three scenarios

```
@ExtendWith(MockitoExtension.class)
class UserServiceTest {

    @Mock
    private UserRepository userRepository;

    @InjectMocks
    private UserService userService;

    @Test
    void getUserById_shouldReturnUser_whenUserExists() {
        // Arrange
        User user = new User(1L, "Rahul", "rahul@email.com");
        when(userRepository.findById(1L)).thenReturn(Optional.of(user));

        // Act
        User result = userService.getUserById(1L);

        // Assert
        assertThat(result).isNotNull();
        assertThat(result.getName()).isEqualTo("Rahul");
    }

    @Test
    void getUserById_shouldThrowException_whenUserNotFound() {
        // Arrange
        when(userRepository.findById(99L)).thenReturn(Optional.empty());

        // Act & Assert
        assertThatThrownBy(() -> userService.getUserById(99L))
            .isInstanceOf(RuntimeException.class)
            .hasMessageContaining("not found");
    }

    @Test
    void createUser_shouldSaveAndReturnUser() {
        // Arrange
        User newUser = new User(null, "Priya", "priya@email.com");
        User savedUser = new User(2L, "Priya", "priya@email.com");
        when(userRepository.save(any(User.class))).thenReturn(savedUser);

        // Act
        User result = userService.createUser(newUser);

        // Assert
        assertThat(result.getId()).isEqualTo(2L);
        verify(userRepository, times(1)).save(any(User.class));
    }
}
```

Three tests. Three scenarios. Happy path, not-found error case, and a create operation with a verify check. That structure- Arrange, Act, Assert- is a standard pattern you'll see everywhere. It makes tests readable. Use it.

The method names are also deliberate. `getUserById_shouldReturnUser_whenUserExists` tells you everything about what the test does before you even read the body. Clear test names save time during debugging.

✓ In `createUser_shouldSaveAndReturnUser`, the mock uses `any(User.class)` instead of the exact `newUser` object. This is intentional- if the service creates a new `User` instance internally rather than passing the exact same reference, matching on `any()` keeps the test flexible and realistic. Use `eq()` only when the exact object identity matters.

8. Mistakes People Make- And How to Avoid Them

These aren't warnings from a textbook. These are the actual mistakes you'll see in real codebases- and sometimes in your own tests when you're starting out.

Testing Everything Together

Some beginners write one giant test that starts the whole app, hits the database, runs the full flow, and asserts at the end. It works when it works. But when it breaks, you have no idea where in the chain it failed. Unit tests exist precisely because you want to isolate failure. Test each layer separately.

Not Mocking Dependencies

The test needs a database connection. The database isn't running in the CI/CD pipeline. The build fails. This is a real problem in teams that don't mock properly. If your unit test needs a real database to run, it's not a unit test- it's a fragile integration test that will randomly break in environments you don't control.

i For integration tests that do need a database, you have two real options:

H2 (in-memory) - fast, zero setup. Enabled by default in `@DataJpaTest`.

Good for basic CRUD. Limitation: H2 SQL dialect may not match production (e.g., MySQL-specific queries can fail).

Testcontainers - spins up a real Docker container of your actual database.

More realistic. Slower. Better for catching dialect issues.

Used in senior-level and production test suites.

For a fresher interview, knowing H2 is sufficient.

Mentioning Testcontainers as "the production-grade alternative" shows awareness.

Writing Tests That Always Pass

This one is subtle and genuinely dangerous. You write a test, it goes green, you move on. But did the test actually verify anything? Look at this:

Assertion matters

```
// This test is useless. It always passes.
@Test
void createUser_test() {
    User user = new User("Rahul", "rahul@email.com");
    userService.createUser(user);
    // No assertion. Nothing being checked.
}

// This actually tests something.
@Test
void createUser_shouldReturnSavedUserWithId() {
    when(userRepository.save(any())).thenReturn(new User(1L, "Rahul",
"rahul@email.com"));
    User result = userService.createUser(new User("Rahul", "rahul@email.com"));
    assertThat(result.getId()).isNotNull();
}
```

A test without an assertion is just code that runs. It proves nothing. Always ask: if my code was completely broken right now, would this test catch it? If the answer is no, the test is not doing its job.

⚠ A related mistake: mocking the method you're actually supposed to be testing. If you mock `productService.getProductById()` in a test for `ProductService`, you're not testing anything- you're just testing that Mockito works. Mock the dependencies. Never mock the class under test itself.

✓ Before you close this chapter:

Pick any service class you've already written. Write three tests for one of its methods: a happy path, an error case, and one that uses `verify()` to check a downstream call happened.

Don't look up the exact syntax at first- try to write it from memory and then fix the compile errors. That struggle is what makes it stick.

If you can do that for one method, you can do it for any method.

Key Improvements Made

- ✓ **Testing Pyramid added.** Section 2 now includes the 70/20/10 distribution (unit/integration/E2E) with an interview-ready framing.
- ✓ **Unit vs Integration table corrected.** "Real dependencies used" is now qualified as "real or near-real" to account for H2 in `@DataJpaTest`.
- ✓ **@SpringBootTest behavior clarified.** Added explicit note that `@SpringBootTest` does NOT start a real server by default- a common misconception. `RANDOM_PORT` explained.
- ✓ **Spring test slices introduced.** `@DataJpaTest` and `@RestClientTest` added with a one-line purpose each. Sufficient for fresher awareness.
- ✓ **Mockito matchers section added.** `any()`, `any(Type.class)`, `eq()`, `anyString()`, and `ArgumentCaptor` explained with examples- directly answers interview follow-ups.
- ✓ **Integration test database options clarified.** H2 vs Testcontainers callout added in Mistakes section- explains the realistic trade-off interviewers expect seniors to know.
- ✓ **createUser test mock corrected.** Changed `when(repository.save(newUser))` to `when(repository.save(any(User.class)))`- more realistic when service creates a new entity internally.
- ✓ **Interview answers sharpened.** Unit vs Integration and `@SpringBootTest` vs `@WebMvcTest` answers now include `@DataJpaTest` reference and the distinction between mocked and real context loading.
- ✓ **404 test exception type corrected.** Changed `RuntimeException` to `ProductNotFoundException` in the `@WebMvcTest` example- aligns with production practice of using typed exceptions.

Chapter 11 Complete

You now know enough to write unit tests, use Mockito, and explain the difference between `@WebMvcTest` and `@SpringBootTest` without hesitating. That alone puts you ahead of most freshers going into their first backend interview.

@JavaInterviewready

THE SPRING BOOT PLAYBOOK

From First API to Hired Developer

CHAPTER 12 · QUICK REVISION CHEAT SHEET

Interview in the next few hours? Good. This is not for learning- it's for recall.

Scan each section fast. Trust what you've already studied. Focus on the why, not just the what.

PART 1: CORE CONCEPTS AT A GLANCE

One-line versions of everything that matters

Spring vs Spring Boot

Spring Framework	Spring Boot
Powerful but requires explicit configuration of every component- DataSource, dispatcher servlet, transaction manager, etc.	Spring + auto-configuration + embedded server (Tomcat). Convention over configuration. Start in seconds, write far less boilerplate.

IoC and Dependency Injection

- **IoC (Inversion of Control)**: Spring owns the object lifecycle- you don't call `new`, Spring does.
- **DI (Dependency Injection)**: Spring injects collaborators into your class- constructor injection preferred over field injection.
- `ApplicationContext` is the IoC container- it creates, wires, and manages all beans.

Interview one-liner: "IoC = Spring controls object lifecycle. DI = Spring injects dependencies instead of you calling `new`. Prefer constructor injection- it makes dependencies explicit and supports immutability."

Interview Traps & Gotchas

- Field injection (`@Autowired` on a field) hides dependencies and makes testing harder- prefer constructor injection.
- Circular dependency with constructor injection → Spring throws `BeanCurrentlyInCreationException` at startup. Redesign or use `@Lazy`.

Bean Stereotypes: `@Component`, `@Service`, `@Repository`

- All three register a bean in the `ApplicationContext`- functionally equivalent for DI.
- `@Component`: generic stereotype. Use when no specific role applies.
- `@Service`: signals business logic layer. No extra Spring magic beyond `@Component`.
- `@Repository`: signals data-access layer. **Adds automatic PersistenceException translation** (converts JPA/SQL exceptions to Spring's `DataAccessException` hierarchy).

Key distinction: `@Repository` is the only stereotype with extra behaviour- exception translation. The rest are semantic labels.

PART 2: ANNOTATIONS GROUPED

Scan this. Don't read it like prose.

Core

Annotation	What It Does
<code>@SpringBootApplication</code>	Meta-annotation: combines <code>@Configuration</code> + <code>@EnableAutoConfiguration</code> + <code>@ComponentScan</code> . Entry point. Scan starts from this class's package.
<code>@Bean</code>	Declares a factory method returning a Spring-managed bean. Only valid inside <code>@Configuration</code> classes.
<code>@Configuration</code>	Marks a class as a source of bean definitions. Methods annotated with <code>@Bean</code> are called via CGLIB proxy to ensure singleton semantics.

Web Layer

Annotation	What It Does
<code>@RestController</code>	<code>@Controller</code> + <code>@ResponseBody</code> . Every method serialises return value to JSON (via Jackson). No view resolution.
<code>@RequestMapping</code>	Base URL mapping on a class or method. Specifies path, method, consumes, produces.
<code>@GetMapping</code> / <code>@PostMapping</code> / <code>@PutMapping</code> / <code>@DeleteMapping</code> / <code>@PatchMapping</code>	HTTP-method-specific shortcuts for <code>@RequestMapping</code> .
<code>@PathVariable</code>	Extracts a value from the URI template. e.g., <code>/users/{id}</code> → <code>@PathVariable Long id</code>
<code>@RequestParam</code>	Binds a query string parameter. e.g., <code>/users?page=2</code> → <code>@RequestParam int page</code> . Optional with <code>defaultValue</code> .
<code>@RequestBody</code>	Deserialises the JSON request body into a Java object (via Jackson). Add <code>@Valid</code> to trigger validation.
<code>@ResponseStatus</code>	Sets the HTTP status code on a method or exception class. e.g., <code>@ResponseStatus(HttpStatus.CREATED)</code>

Dependency Injection

Annotation	What It Does
<code>@Autowired</code>	Injects a bean by type. Constructor injection preferred- makes dependencies explicit, supports final fields.
<code>@Qualifier("name")</code>	Disambiguates when multiple beans of the same type exist. Pair with <code>@Autowired</code> .
<code>@Primary</code>	Designates the default bean when multiple candidates exist- lower precedence than <code>@Qualifier</code> .
<code>@Component</code>	Generic stereotype- registers a bean. No additional behaviour.
<code>@Service</code>	Semantic alias for <code>@Component</code> . Use for business/service classes.
<code>@Repository</code>	Alias for <code>@Component</code> + <code>PersistenceException</code> translation.

JPA / Database

Annotation	What It Does
<code>@Entity</code>	Maps a Java class to a DB table. Class name = table name by default.
<code>@Table(name="...")</code>	Overrides the default table name. Also accepts schema, uniqueConstraints.
<code>@Id</code>	Marks the primary key field.
<code>@GeneratedValue(strategy=...)</code>	Auto-generates PK values. Strategies: AUTO, IDENTITY (DB auto-increment), SEQUENCE, TABLE.
<code>@Column</code>	Customises column name, nullable, unique, length, insertable, updatable.
<code>@OneToMany</code> / <code>@ManyToOne</code>	<code>@OneToMany</code> is LAZY by default (good). <code>@ManyToOne</code> is EAGER by default- override with <code>fetch=FetchType.LAZY</code> .
<code>@ManyToMany</code>	Requires a join table. Use <code>@JoinTable</code> to customise. Both sides LAZY by default.
<code>@Transactional</code>	Wraps method in a DB transaction. Commits on success, rolls back on <code>RuntimeException</code> . Does NOT roll back on checked exceptions unless <code>rollbackFor=Exception.class</code> is added.
<code>@JoinColumn(name="...")</code>	Specifies the FK column on the owning side of a relationship.

Interview Traps & Gotchas

- `@ManyToOne` is EAGER by default- always set `fetch=FetchType.LAZY` to avoid N+1 on joins.
- `@OneToMany` without `mappedBy` creates an extra join table- always define the owning side.
- `@Transactional` on a private method is silently ignored- Spring can't proxy it.
- `@Transactional` within the same class (self-invocation) bypasses the proxy- transaction won't start.

Validation

Annotation	What It Does
<code>@Valid</code>	Triggers Bean Validation on the parameter. Place in controller method signature alongside <code>@RequestBody</code> .
<code>@Validated</code>	Spring's variant. Required for group validation and for triggering validation on <code>@PathVariable</code> / <code>@RequestParam</code> .
<code>@NotNull</code>	Value cannot be null. Does not check for empty string.
<code>@NotBlank</code>	Not null, not empty, not whitespace-only. Use this for String fields- stricter than <code>@NotEmpty</code> .
<code>@NotEmpty</code>	Not null and not empty. Whitespace string ' ' passes <code>@NotEmpty</code> but fails <code>@NotBlank</code> .
<code>@Size(min, max)</code>	String length or Collection size within bounds.
<code>@Email</code>	Must match valid email format.
<code>@Min</code> / <code>@Max</code>	Numeric value must be $\geq$ min or $\leq$ max.
<code>@Pattern(regex)</code>	String must match the given regex.

Key distinction: `@NotBlank` is stricter than `@NotEmpty`. For any user-supplied String (name, email, etc.), always prefer `@NotBlank`.

Security

Annotation	What It Does
<code>@EnableWebSecurity</code>	Activates Spring Security. Pair with a class extending <code>SecurityFilterChain</code> bean definition.
<code>@PreAuthorize("...")</code>	Method-level security using SpEL. e.g., <code>@PreAuthorize("hasRole('ADMIN')")</code> . Requires <code>@EnableMethodSecurity</code> .
<code>@EnableMethodSecurity</code>	Enables <code>@PreAuthorize</code> , <code>@PostAuthorize</code> , <code>@Secured</code> on methods (replaces deprecated <code>@EnableGlobalMethodSecurity</code>).

PART 3: KEY DIFFERENCES

Interviewers love these- know them cold

Don't wait to be asked 'what is X'. Expect: 'what's the difference between X and Y?'

Topic	Option A	Option B
@Component vs @Service vs @Repository	@Component: generic. No special behaviour.	@Service: semantic marker for business logic. @Repository: same + PersistenceException translation. All create beans.
@RestController vs @Controller	@Controller: returns a view name (Thymeleaf, JSP). Use for server-side rendering.	@RestController = @Controller + @ResponseBody. Returns data (JSON/XML). No view resolution.
@PathVariable vs @RequestParam	@PathVariable: bound to URI segment. GET /users/{id}- id is a path variable.	@RequestParam: bound to query string. GET /users?page=2- page is a request param. Supports defaultValue and required=false.
EAGER vs LAZY loading	EAGER: related data loaded immediately with parent. Can cause N+1 queries. @ManyToOne default- override it.	LAZY: related data loaded on first access. Default for @OneToMany, @ManyToMany. Triggers LazyInitializationException if accessed outside a transaction.
@Primary vs @Qualifier	@Primary: class-level default for a bean type. Lowest specificity- any @Qualifier overrides it.	@Qualifier("name"): injection-site specificity. Pinpoints exactly which bean. Always wins over @Primary.
@Transactional rollback	Rolls back automatically on unchecked exceptions (RuntimeException and subclasses).	Does NOT roll back on checked exceptions. Add rollbackFor=Exception.class to handle them.
save()- INSERT vs UPDATE	Entity with null ID → INSERT (JPA calls persist()).	Entity with existing ID → UPDATE (JPA calls merge()). Accidentally setting an ID before save() will attempt an update on a potentially non-existent row.

PART 4: API REQUEST FLOW

Trace this in 10 seconds if asked

Client → Controller (@RestController) → Service (@Service) → Repository (@Repository) → Database

Response travels back up the same chain:

- **Repository** returns Entity to Service.
- **Service** maps Entity → DTO (never expose raw entities to the client).
- **Controller** wraps DTO in ResponseEntity<DTO> and returns with HTTP status.

Signal architectural awareness: "The controller returns a DTO, not an entity- this decouples the API contract from the DB schema and prevents accidental data exposure."

Interview Traps & Gotchas

- Returning an entity directly can expose sensitive fields (e.g., passwords, internal IDs) and creates tight coupling to your DB schema.
- Business logic in the Controller is a red flag- it belongs in the Service layer.
- @Transactional on the Controller is wrong- transactions belong on Service methods.

PART 5: JWT AUTHENTICATION FLOW

Stateless auth in 7 steps

Core concept: JWT is stateless- the server stores no session. The signed token is the proof of identity.

#	Action	What Happens
1	POST /login	Client sends username + password in request body.
2	Validate credentials	Server loads user via UserDetailsService, checks password with PasswordEncoder.
3	Generate JWT	Server signs token with secret key (HS256) or private key (RS256). Sets subject + expiry.
4	Return token	Server returns JWT to client (response body or header).
5	Store token	Client stores in memory (preferred) or localStorage.
6	Send with requests	Client adds Authorization: Bearer <token> header on every protected request.
7	Validate token	JwtFilter (OncePerRequestFilter) intercepts request, verifies signature + expiry, loads SecurityContext.

JWT Structure:

Header (algorithm + token type) . **Payload** (claims: sub, iat, exp, roles) . **Signature** (HMAC-SHA256 of header+payload)
 eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJlc2VyQGVTYWlsImNvbSIzImV4cCI6MTcwMDAwMH0.SflKxwRJSMeKKF2QT4fw

JWT Interview Key Points

- **Stateless:** no server-side session. Any server instance can validate any token- scales horizontally.
- **Signature validation:** server verifies signature on every request. No DB lookup needed for auth check.
- **Expiry:** tokens expire (exp claim). Use short-lived access tokens + refresh tokens for production.
- **Revocation:** JWT cannot be invalidated server-side without a blocklist- this is a known trade-off.
- **Storage:** localStorage is vulnerable to XSS. Memory or HttpOnly cookie is safer.

Interview Traps & Gotchas

- JWT payload is Base64-encoded, NOT encrypted- never store passwords, PII, or secrets in the payload.
- Token revocation before expiry requires a blocklist (e.g., Redis)- pure stateless JWT can't revoke tokens.
- Using symmetric key (HS256) means both signing and verification use the same secret- if compromised, all tokens are forged.

PART 6: THE GOLDEN RULES

Non-negotiable in real projects and interviews

1

Never expose entities directly.

Return DTOs. You control the API contract, not the DB schema. Prevents data leaks and tight coupling.

2

Always validate at the boundary.

@Valid + @NotBlank/@Email on every inbound DTO. Reject bad data before it reaches the service layer.

3

Use separate request and response DTOs.

Input and output shapes often differ. Merging them creates confusion and over-exposure.

4

Handle errors globally.

@ControllerAdvice + @ExceptionHandler. One place for all error responses- no try-catch sprawl in controllers.

5

LAZY load collections; fetch explicitly when needed.

Prevents accidental N+1 queries. Use JOIN FETCH or @EntityGraph in the repository when you need the association.

6

Keep controllers thin.

No business logic, no DB calls, no try-catch. Controllers receive, delegate, and return.

7

@Transactional belongs on the Service.

Ensures atomicity for write operations. RuntimeException triggers rollback. Add rollbackFor=Exception.class for checked exceptions.

8

Handle Optional defensively.

Never call .get() without checking. Use .orElseThrow(() → new ResourceNotFoundException(...))- returns 404, not 500.

9

Constructor inject- not field inject.

Makes dependencies visible, supports immutability, and makes unit testing trivial without Spring context.

10

Never swallow exceptions silently.

`catch (Exception e) {}` is a bug waiting to be discovered in production. Always log or rethrow.

You're Ready.

You've covered Spring's internals, layered architecture, validation, error handling, transactions, and JWT. That's more depth than most candidates walk in with.

Don't memorise annotations. Know the why behind each decision- why DTOs, why @Transactional on service, why global exception handling, why LAZY loading. If you can explain the reasoning, the syntax will follow.

The Spring Boot Playbook · Chapter 12 · Premium Edition

@JavaInterviewready

THE SPRING BOOT PLAYBOOK

From First API to Hired Developer

CHAPTER 13: TOP INTERVIEW QUESTIONS WITH REAL ANSWERS

Chapter 13: Top Interview Questions

These are not random questions scraped from some list. They're the ones that actually come up. The ones interviewers ask because they separate candidates who've read documentation from candidates who actually understand what they built.

Every answer here is written the way you should speak it out loud. Not a textbook definition. Not bullet points you memorised. The way a developer who knows their stuff would explain it to someone sitting across a table.

Read the answers. Then close the book and say them in your own words. That's the actual preparation.

SECTION 1: CORE SPRING BOOT

3 questions that open almost every interview

Q What is Spring Boot?

Comes up in literally every Spring Boot interview

Spring Boot is Spring Framework with the boring parts taken care of for you.

Normally with Spring, you spend time configuring every single thing: datasource, beans, server settings. Spring Boot handles all of that automatically based on what's in your classpath.

It also comes with an embedded server, Tomcat by default, so you don't need to deploy a WAR file anywhere. You just run the jar and your application is up.

"Convention over configuration" is the phrase to remember. Spring Boot makes sensible defaults and you override only what you need to change.

Q What is Auto Configuration?

Follow-up after "What is Spring Boot?"

When you add a dependency to your project, Spring Boot detects it and automatically sets up the related configuration.

Add `spring-boot-starter-data-jpa` and Spring Boot automatically configures a `DataSource`, `EntityManagerFactory`, and transaction management. You didn't write any of that. It just happened.

This works through `@EnableAutoConfiguration`, which scans for configuration classes in `spring.factories` and applies them conditionally based on what's available in your project.

Interview phrasing: Auto configuration means Spring Boot inspects your classpath and automatically configures beans based on what dependencies are present. You can override any of it.

Q What is IoC (Inversion of Control)?

Normally you write code like: `UserService service = new UserService()`. You create the object, you control it.

IoC flips that. You tell Spring what you need, and Spring creates and manages the object for you. You're not in control of the object lifecycle anymore. Spring is. That's the inversion.

Dependency Injection is how IoC actually gets implemented. Spring injects the objects your class needs instead of you creating them manually.

One-liner: **"IoC means Spring manages object creation and lifecycle instead of your code doing it manually."**

SECTION 2: ANNOTATIONS & DEPENDENCY INJECTION

Q What is a Bean?

A Bean is just an object that Spring creates and manages. That's it.

When Spring starts up, it reads your application, finds all the classes annotated with `@Component`, `@Service`, `@Repository`, or defined via `@Bean` methods, creates instances of them, and stores them in something called the Application Context.

Whenever another class needs one of those objects, Spring injects it from the context. You never call `new` yourself.

Q @Component vs @Service vs @Repository- what's the difference?

One of the most common fresher questions

Technically, all three do the same thing: they register the class as a Spring-managed bean. You could use `@Component` everywhere and your application would still work.

The difference is intent and one extra behavior. `@Service` signals that this class holds business logic. `@Repository` signals that it handles data access, and it also catches database-specific exceptions and translates them into Spring's `DataAccessException` hierarchy.

`@Component` is the generic one. Use it when neither `@Service` nor `@Repository` fits.

Interviewers ask this to see if you know they're more than just labels. Mention the exception translation for @Repository. That's the detail that lands.

Q Constructor Injection vs Field Injection- which is better and why?

Shows maturity- most freshers use field injection without thinking

Field injection looks simpler:

```
@Autowired
private UserService userService; // field injection
```

Constructor injection is the preferred approach:

```
private final UserService userService;

public UserController(UserService userService) {
    this.userService = userService;
}
```

Constructor injection has two concrete advantages. First, the dependency is declared final, so it cannot be accidentally replaced after creation. Second, it makes unit testing straightforward because you can inject mock objects through the constructor without needing Spring at all.

Field injection hides dependencies. You can't tell what a class needs just by looking at its constructor. And testing requires reflection-based frameworks to inject mocks.

Spring itself recommends constructor injection. Say that in the interview.

SECTION 3: REST APIs

Q What is REST?

REST stands for Representational State Transfer. It's an architectural style for designing APIs that communicate over HTTP.

A REST API treats everything as a resource, and you interact with resources using standard HTTP methods: GET to read, POST to create, PUT to update, DELETE to remove.

The key principles are: stateless (the server doesn't store client session data between requests), uniform interface (consistent URL structure), and using HTTP status codes correctly.

Keep this answer tight. You don't need to recite all six REST constraints. Three clear points is enough: **resources**, **HTTP verbs**, **stateless**.

Q What is the difference between GET and POST?

GET is for retrieving data. POST is for creating or submitting data.

GET requests don't have a body. The data goes in the URL as path variables or query parameters. GET requests are safe (they don't change server state) and idempotent (calling them multiple times gives the same result).

POST requests have a body. You send JSON, form data, or files in the request body. They're not idempotent, calling POST twice might create two records.

Simple rule: if you're reading data, use GET. If you're writing data or sending something sensitive (like a password), use POST.

Q @PathVariable vs @RequestParam

@PathVariable pulls a value from the URL path itself. @RequestParam pulls a value from the query string.

```
// @PathVariable: value is part of the path
GET /users/42
@GetMapping("/users/{id}")
public User getUser(@PathVariable Long id) { }

// @RequestParam: value is in the query string
```

```
GET /users?page=2&size=10
@GetMapping("/users")
public List<User> getUsers(@RequestParam int page, @RequestParam int size) { }
```

Use `@PathVariable` when the value identifies a specific resource. Use `@RequestParam` for optional filters, pagination, or search parameters.

SECTION 4: JPA & DATABASE

This section has the most variety- read carefully

Q What is JPA?

JPA stands for Java Persistence API. It's a specification, a set of rules, for how Java objects should be mapped to database tables.

JPA itself doesn't do anything. It's just a standard. Hibernate is the most common implementation of that standard. When you use Spring Data JPA, you're using JPA annotations with Hibernate doing the actual work underneath.

One-liner: "JPA is the specification, Hibernate is the implementation. Think of JPA as an interface and Hibernate as the class that implements it."

Q What is Hibernate?

Hibernate is an ORM (Object Relational Mapper). It maps Java classes to database tables and lets you work with objects instead of writing raw SQL for most operations.

You define a User class with `@Entity` and Hibernate creates the SQL to insert, select, update, and delete rows automatically. When you call `userRepository.save(user)`, Hibernate translates that into an INSERT or UPDATE statement.

It also manages the concept of a Hibernate Session, which tracks changes to your objects and flushes them to the database at the right time.

Q What is FetchType? What is the difference between LAZY and EAGER?

Very common, especially at product companies

FetchType controls when Hibernate loads related entities.

```
@OneToMany(fetch = FetchType.LAZY)    // default for collections
private List<Order> orders;

@ManyToOne(fetch = FetchType.EAGER)    // default for single objects
private User user;
```

LAZY means: load the related data only when you actually access it in code. EAGER means: load it immediately when the parent entity is fetched, even if you never use it.

LAZY is almost always what you want for collections. Using EAGER on a collection means every time you load a User, Hibernate also fetches all their Orders, even if you only needed the username.

Mention the N+1 problem when asked about LAZY. It shows you understand the trade-off.

Q What is Cascade in JPA?

Cascade controls what happens to child entities when you perform an operation on the parent. If you have a User with a list of Addresses and you set `cascade = CascadeType.ALL`, then when you save the User, Hibernate automatically saves all the Addresses too. When you delete the User, the Addresses get deleted too.

```
@OneToMany(cascade = CascadeType.ALL, mappedBy = "user")
private List<Address> addresses;
```

`CascadeType.ALL` means all operations propagate: PERSIST, MERGE, REMOVE, REFRESH, DETACH. You can be more specific, like `CascadeType.PERSIST` if you only want saves to cascade but not deletes.

A common mistake is using `CascadeType.ALL` without thinking about the REMOVE behavior. Deleting a parent will delete all children. Make sure that's what you actually want.

Q What is mappedBy in JPA?

`mappedBy` is used in bidirectional relationships to tell JPA which side owns the relationship, meaning which table holds the foreign key.

```

@Entity
public class User {
    @OneToMany(mappedBy = "user") // User is NOT the owner
    private List<Post> posts;
}

@Entity
public class Post {
    @ManyToOne // Post IS the owner (has the foreign key)
    @JoinColumn(name = "user_id")
    private User user;
}

```

mappedBy = "user" means: "the other side manages this relationship, look at the 'user' field in the Post class." The foreign key column user_id lives in the Post table.

If you don't use mappedBy correctly, Hibernate creates a join table it doesn't need, or duplicates data. This is one of those things where the mistake is invisible until you look at the generated SQL.

SECTION 5: ADVANCED QUESTIONS

The ones that separate prepared candidates from the rest

Q What is @Transactional and when do you use it?

Know the rollback behavior- most freshers don't

@Transactional wraps a method in a database transaction. All database operations inside that method either succeed together or roll back together.

You use it on service methods that perform multiple database operations where partial success is not acceptable. A money transfer is the classic example: debit one account, credit another. If the second operation fails, the first should roll back.

```

@Transactional
public void transfer(Long fromId, Long toId, Double amount) {
    Account from = accountRepo.findById(fromId).orElseThrow(...);
    Account to = accountRepo.findById(toId).orElseThrow(...);
    from.setBalance(from.getBalance() - amount);
    to.setBalance(to.getBalance() + amount);
    accountRepo.save(from);
    accountRepo.save(to); // if this fails, the first save rolls back
}

```

The gotcha: `@Transactional` only rolls back on unchecked exceptions (`RuntimeException`) by default. For checked exceptions, you need `@Transactional(rollbackFor = Exception.class)`.

Q What is a DTO and why do we use it?

DTO stands for Data Transfer Object. It's a plain Java class used to carry data between layers, specifically between your API and the client.

We use DTOs instead of exposing entities directly for two reasons. First, you control exactly what data the client sees. If your `User` entity has a `passwordHash` field, the DTO simply doesn't have that field. Second, your API contract stays stable even if your database schema changes internally.

There's also a practical reason: bidirectional JPA relationships can cause infinite recursion when Jackson tries to serialize an entity. DTOs break that cycle cleanly.

Interview answer in one line: "DTOs decouple the API layer from the database layer, give you control over what's exposed, and prevent serialization issues."

Q What is the N+1 Problem?

If you explain this clearly, you will stand out

It's a performance issue where fetching `N` parent records triggers `N` additional queries to load their related data, one per parent.

```
List<Order> orders = orderRepo.findAll(); // 1 query
for (Order o : orders) {
    o.getItems(); // 1 query per order  <- N queries
}
// 100 orders = 101 total queries
```

It happens because of LAZY loading. When you access the `items` collection inside the loop, Hibernate fires a separate query each time because it didn't load them upfront.

The fix is JOIN FETCH, which loads the parent and children in a single query:

```
@Query("SELECT o FROM Order o JOIN FETCH o.items")
List<Order> findAllWithItems();
```

Five orders in development means five extra queries. Fifty thousand orders in production means fifty thousand extra queries. That's when your app slows down and your DBA starts asking questions.

SECTION 6: SECURITY

Q What is JWT and how does it work?

Almost every interview with a security component asks this

JWT stands for JSON Web Token. It's a way to do stateless authentication, meaning the server doesn't store any session data.

When a user logs in, the server validates their credentials and generates a token. The token is a Base64-encoded string with three parts: a header (algorithm used), a payload (user data like ID and roles), and a signature (to verify it hasn't been tampered with).

The client stores this token and sends it in the Authorization header with every subsequent request. The server validates the signature on each request without hitting the database. If the signature is valid and the token hasn't expired, access is granted.

```
Authorization: Bearer  
eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJlc2VyQGVTYWlsLmNvbSJ9.SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5  
c
```

Why stateless matters: in a scaled application with multiple servers, any server can validate the token without needing shared session storage.

Q What is the difference between Authentication and Authorization?

Authentication is verifying who you are. Authorization is verifying what you're allowed to do.

Authentication happens first. When you send a username and password (or a token), the system checks if you are who you claim to be. If yes, you're authenticated.

Authorization happens after. Even if you're authenticated, you might not have permission to access everything. An authenticated user with role USER can't access an endpoint restricted to ADMIN. That check is authorization.

One-liner: "Authentication = are you who you say you are? Authorization = are you allowed to do what you're trying to do?"

In Spring Security: authentication is handled by AuthenticationManager and UserDetailsService. Authorization is handled by your security configuration, where you define which roles can access which endpoints.

◆ You've now read every question with the kind of answer that shows understanding, not memorisation. The difference in an interview is whether you can explain the "why." Why DTOs? Why constructor injection? Why JOIN FETCH? Practice saying these answers out loud once. That's the last step.

Chapter 13 Complete

Nineteen questions. Every major topic covered. If you can answer these clearly without hesitation, you're ready for most fresher and junior-level Spring Boot interviews.

The book is done. Go get that offer.

The Spring Boot Playbook | Chapter 13 | Premium Edition

THE SPRING BOOT PLAYBOOK

From First API to Hired Developer

CHAPTER 14: COMMON BEGINNER MISTAKES

Chapter 14: Common Beginner Mistakes

Nobody tells you about these in tutorials. You find out the hard way. Either your code review gets torn apart, or an interviewer asks a follow-up question that exposes a gap you didn't know you had.

Every mistake in this chapter is something real developers have shipped to production. Some caused subtle bugs. Some caused data loss. Some just made the codebase harder to maintain.

Go through each one and ask yourself honestly: have I done this? Most of us have. Knowing why it's wrong is what matters.

SECTION 1: DESIGN MISTAKES

The structural ones that haunt you later

✗ Returning the Entity directly from the Controller

This is where most beginners get caught in interviews

You build a User API. The getUser endpoint works. You return the User entity, everything looks fine. But here's what's actually happening:

If your entity has a passwordHash field, that just went to the client. If you have a bidirectional relationship between User and Order, Jackson starts serializing User, then each Order, then back to User again. StackOverflowError. Application crash.

And there's a less obvious problem too. The moment your database schema changes, your API response changes. Your frontend breaks. Your mobile app breaks. You've tied your API contract directly to your database structure, which will absolutely change over time.

✓ The fix is a DTO. A plain class with exactly the fields the client needs. You control the output. The database schema can change without the API response changing. Always return a DTO from your controllers.

✗ Putting Business Logic in the Controller

Looks harmless at first, painful to maintain

It starts small. One if statement in a controller method. Then a calculation. Then a validation check. Then a second database call. Before you know it, your controller method is 60 lines long and untestable.

Bad	Good
<pre>@GetMapping("/discount/{userId}") public double getDiscount(@PathVariable Long userId) { User u = userRepo.findById(userId) .orElseThrow(...); if (u.getOrderCount() > 10) { return 0.20; } else if (u.getOrderCount() > 5) { return 0.10; } return 0.0; }</pre>	<pre>@GetMapping("/discount/{userId}") public double getDiscount(@PathVariable Long userId) { return discountService .calculateDiscount(userId); } // Logic lives in DiscountService // Testable, reusable, clean</pre>

The controller has one job: receive a request, call the service, return a response. If you can't describe what your controller does in one sentence without saying "and then it also...", the logic is in the wrong place.

✗ No Separation Between Request and Response DTOs

A small decision that pays off as the project grows

Using a single UserDTO for both incoming requests and outgoing responses feels convenient. But the data you receive from a client is almost never the same shape as the data you send back.

A create request needs name, email, password. The response should have id, name, email, createdAt. Never password. When you use one DTO for both, you end up with nullable fields everywhere, confusing documentation, and awkward checks like "only set this field on the way out."

Use UserRequestDTO for what comes in. Use UserResponseDTO for what goes out. Two extra files. Worth it every time.

SECTION 2: JPA MISTAKES

These ones bite hardest in production

JPA mistakes are tricky because they're invisible in development. Five rows in a test database and everything is fast. Ten thousand rows in production and your application grinds to a halt. The mistakes in this section are the reason why.

✗ Using FetchType.EAGER on Collections

The most common JPA mistake, by a long way

A developer gets LazyInitializationException. Switches the collection to EAGER. Exception goes away. Problem solved?

Not even close. What actually happened is: every time you load a User, Hibernate now also loads all their Orders. All their OrderItems. All their associated data. Even when you only needed the user's name and email.

In a list endpoint that returns 200 users, you just fired 200 silent extra queries and loaded data you threw away immediately. Your endpoint gets slower, your database gets hammered, and there's no warning in the logs.

⚠ LazyInitializationException is not a signal to switch to EAGER. It means you accessed lazy-loaded data outside of an active Hibernate session. The real fix is to load what you need inside a @Transactional service method, or use JOIN FETCH for that specific query.

✗ Wrong Relationship Mapping (the mappedBy mistake)

Silent bug- no error, just duplicate data or extra tables

If you have a bidirectional `@OneToMany` relationship and you forget `mappedBy`, Hibernate assumes both sides own the relationship. It creates a join table that shouldn't exist, or inserts duplicate foreign key data.

Bad	Good
<code>// Missing mappedBy on User side</code>	<code>// Correct: mappedBy tells JPA</code>
<code>@OneToMany</code>	<code>// Post owns the relationship</code>
<code>private List<Post> posts;</code>	<code>@OneToMany(mappedBy = "user")</code>
	<code>private List<Post> posts;</code>
<code>// Hibernate creates an extra</code>	
<code>// user_posts join table you</code>	<code>// Post table has user_id column</code>
<code>// never asked for</code>	<code>// No extra join table created</code>

The foreign key lives on the "many" side. `mappedBy` on the "one" side tells JPA: "that field over there manages this relationship, I'm just referencing it." Get this wrong and Hibernate quietly creates the wrong schema.

✗ Using CascadeType.ALL Without Thinking About REMOVE

`CascadeType.ALL` is convenient. Save the parent and children save automatically. But `ALL` includes `REMOVE`.

Delete a User with `CascadeType.ALL` on their Orders? All orders deleted. Delete a Department with `CascadeType.ALL` on its Employees? You just deleted every employee in that department.

Be explicit. Use `CascadeType.PERSIST` if you just want saves to cascade. Use `CascadeType.MERGE` if you want updates to cascade. Only use `ALL` when you genuinely want every operation to propagate, and you've thought through what `DELETE` actually means in that context.

SECTION 3: API MISTAKES

× No Input Validation

Works fine in dev, breaks in 5 minutes on production

You test your API with good data. Name, email, age, all filled in, all valid. You ship it. A user submits an empty email. Now you have a null value in a NOT NULL database column. Exception. 500 response. Confused user.

Another user submits a 2000-character username because why not. Your database column was VARCHAR(50). Error. Another sends a string where you expected a number. ClassCastException somewhere deep in the stack.

Add @Valid to every @RequestBody parameter. Add @NotBlank, @Email, @Size, @Min, @Max to every DTO field that needs it. It takes five minutes and prevents an entire category of production bugs.

× Ignoring HTTP Status Codes

Makes your API unpredictable for anyone consuming it

Returning 200 OK for everything is one of those habits that feels harmless until someone else has to consume your API.

Bad	Good
<pre>// Created a user, returns 200 return ResponseEntity.ok(user);</pre>	<pre>// Created: 201 CREATED return ResponseEntity .status(HttpStatus.CREATED) .body(userDTO);</pre>
<pre>// User not found, still 200 return ResponseEntity.ok(null);</pre>	<pre>// Not found: 404 throw new UserNotFoundException(id);</pre>

```
// Validation failed, still 200
```

```
return ResponseEntity.ok("error");
```

```
// Validation fail: 400 (auto)
```

```
// via @Valid + @ControllerAdvice
```

HTTP status codes exist for a reason. 200 means success. 201 means created. 400 means the client sent bad data. 404 means the resource doesn't exist. 500 means something broke on the server. Use them correctly and your API becomes self-documenting.

× Inconsistent Response Structure

One endpoint returns just the object. Another wraps it in a success field. Another returns a string message on error but a JSON object on success. The client has to handle three different shapes.

Pick a response structure and use it everywhere. A simple wrapper with success, message, and data covers most cases. When every response looks the same, the client code is simpler and bugs are easier to find.

SECTION 4: CODE MISTAKES

× Field Injection Everywhere

Works until you need to write a test

Field injection is what most tutorials show because it's shorter to type. But it creates hidden dependencies. When you look at a class, you can't tell what it needs just by reading the constructor. And when you try to unit test it, you either need Spring to wire it up or you need reflection-based tools to inject mocks.

Bad	Good
<pre>@Service</pre>	<pre>@Service</pre>
<pre>public class OrderService {</pre>	<pre>public class OrderService {</pre>
<pre> @Autowired</pre>	<pre> private final UserRepository userRepo;</pre>
<pre> private UserRepository userRepo;</pre>	<pre> private final EmailService emailSvc;</pre>

<code>@Autowired</code>	<code>public OrderService(UserRepository r,</code>
<code>private EmailService emailService;</code>	<code>EmailService e) {</code>
	<code> this.userRepo = r;</code>
<code>}</code>	<code> this.emailSvc = e;</code>
	<code>}</code>
	<code>}</code>

Constructor injection makes dependencies visible, makes fields final (immutable after creation), and makes unit testing straightforward. You pass in mock objects through the constructor with no Spring context needed. This is what Spring itself recommends.

✗ Calling `.get()` on Optional Without Checking

A `NoSuchElementException` waiting to happen

Optional exists to make null handling explicit. It's the repository's way of saying "this might not exist, handle it properly."

Calling `.get()` directly says "I know this exists." When it doesn't, you get `NoSuchElementException` with a stack trace that doesn't tell you what resource was missing or why. The client gets a confusing 500.

Bad	Good
<code>// Will crash with</code>	<code>// Throws meaningful exception</code>
<code>// NoSuchElementException</code>	<code>// Handled by @ControllerAdvice</code>
<code>// if user doesn't exist</code>	<code>// Returns clean 404 to client</code>
<code>User user = userRepository</code>	<code>User user = userRepository</code>
<code> .findById(id)</code>	<code> .findById(id)</code>
<code> .get();</code>	<code> .orElseThrow(() -></code>
	<code> new UserNotFoundException(id)</code>
	<code>);</code>

`orElseThrow` with a custom exception gives you a meaningful error, the right HTTP status, and a message that actually helps. It takes one extra line. Use it every time.

SECTION 5: SECURITY MISTAKES

The ones with real consequences

Security mistakes are in a different category from the others. A bad response structure is inconvenient. Leaking a password hash is a breach.

✗ Sending Sensitive Data in API Responses

The most embarrassing mistake in a code review

You create a User entity. It has name, email, passwordHash, resetToken, internalNotes. You return the entity directly from the controller because you haven't added DTOs yet. Every field goes to the client.

The password hash is not a password, but it's still useful for offline cracking attacks. The reset token can be used to take over accounts. The internal notes were never meant for users.

A UserResponseDTO with only id, name, email, and createdAt takes ten minutes to write and closes this entire category of exposure. There's no good reason to skip it.

✗ Storing JWT Secrets as Plain Strings in Code

The JWT secret key is what protects your tokens. Anyone who has it can generate valid tokens for any user in your system, including admin accounts.

Hardcoding the secret in your application.properties file and committing it to a public GitHub repository is one of the most common security mistakes in student projects. Bots scan GitHub for exposed secrets within minutes of a commit.

Keep secrets in environment variables. Use application.properties only for non-sensitive configuration. Use Spring's @Value or a secrets manager in production.

Bad

Good

# application.properties	# application.properties
# DO NOT do this	jwt.secret=\${JWT_SECRET}
jwt.secret=mySecretKey123	
	# Set the env variable in
	# your deployment env
	# Never commit the value

✗ Not Validating JWT on Every Protected Endpoint

A partial security setup is the same as no security

A common mistake is adding Spring Security to a project, writing the JWT filter, and then forgetting to actually protect all the endpoints that need protection.

The registration and login endpoints should be open. Every other endpoint, unless explicitly public, should require a valid token. In your security configuration, deny by default and explicitly permit only what should be public. Never the other way around.

Also check that your filter validates the expiry timestamp. A token that's valid but expired should be rejected. This is a basic check that some implementations skip.

◆ Most of these mistakes share something in common: they work fine in the short term and cause problems later. The developer who thinks ahead to what happens in production, under load, with real users, is the one who gets hired and gets promoted.

Chapter 14 Complete

Fourteen mistakes. All of them real. All of them fixable. If you can explain why each one is wrong and what the right approach is, you're thinking like a developer who's been in production, not just in tutorials.

That's actually what interviewers are trying to figure out: can you think about what happens when things go wrong? This chapter is your answer to that question.

SPRING BOOT INTERVIEW SERIES

The Spring Boot Playbook

From First API to Hired Developer

CHAPTER 15

Mini Project User Management API

Practical Build · REST API · DTOs · Validation · Error Handling

CHAPTER 15 - MINI PROJECT: USER MANAGEMENT API

Everything we've covered in this book has been building toward this chapter.

Annotations, beans, dependency injection, REST controllers, JPA, validation, exception handling. All of it. This project pulls those pieces together into something a real interviewer might ask you to walk through on a whiteboard or build in a take-home assignment.

The goal here isn't to show you a fancy project. It's to show you a **clean one**. Something structured, easy to explain, and something you could actually build live in front of someone without panicking.

We're building a User Management API. Simple enough that a fresher can finish it in a day. Deep enough that it covers the topics every interviewer cares about.

What We're Building:

POST /api/users Create a new user
GET /api/users Fetch all users
GET /api/users/{id} Fetch a single user by ID

With: input validation, response DTOs, custom exception handling, clean layering

Step 1 - Project Structure

Before writing a single line of business logic, let's talk about structure. How you organize your code says a lot about how you think.

When you generate a Spring Boot project from start.spring.io, you get a flat structure with one main class. That's fine for a demo. But for anything real, you want a layered package structure that separates concerns clearly.

```
src/  
  main/  
    java/  
      com.example.usermanagement/  
        UserManagementApplication.java  
        controller/
```

```
UserController.java
service/
  UserService.java
repository/
  UserRepository.java
entity/
  User.java
dto/
  CreateUserRequest.java
  UserResponse.java
exception/
  UserNotFoundException.java
  GlobalExceptionHandler.java
resources/
  application.properties
```

Each folder has one job. Controllers handle HTTP. Services handle logic. Repositories handle data. DTOs are what the client sends and receives. Exceptions go in their own package.

In interviews, if you're asked to explain your project, starting with the folder structure shows maturity. It tells the interviewer you think about design before typing code.

Step 2 - Dependencies and Configuration

Add these starters in your pom.xml when generating from start.spring.io:

```
<dependencies>

  <!-- Web: REST controllers, embedded Tomcat -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>

  <!-- JPA: Hibernate, Spring Data JPA -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>

  <!-- Validation: @NotBlank, @NotNull, @Email etc. -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
  </dependency>
```

```
<!-- H2: in-memory database, no setup needed -->
<dependency>
  <groupId>com.h2database</groupId>
  <artifactId>h2</artifactId>
  <scope>runtime</scope>
</dependency>

<!-- Lombok: reduces boilerplate -->
<dependency>
  <groupId>org.projectlombok</groupId>
  <artifactId>lombok</artifactId>
  <optional>true</optional>
</dependency>

</dependencies>
```

We're using H2 (in-memory database) here because this is a demo project and we don't want to spend time setting up MySQL. H2 starts with your app, holds data while it's running, and disappears when you stop it. Perfect for interviews and learning.

Now the **application.properties** configuration:

```
# Server
server.port=8080

# H2 in-memory database
spring.datasource.url=jdbc:h2:mem:usersdb
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=

# JPA / Hibernate
spring.jpa.hibernate.ddl-auto=create-drop
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

# H2 browser console
spring.h2.console.enabled=true
```

Turn **show-sql=true** on during development. You can see every query Hibernate runs. It's how you catch N+1 problems, unintended updates, and missing indexes early. Don't forget to turn it off in production.

Step 3 - The Entity

The **User** entity is the Java class that maps to the database table. It's the center of everything in this project.

```
package com.example.usermanagement.entity;

import jakarta.persistence.*;
import lombok.*;

@Entity
@Table(name = "users")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String name;

    @Column(nullable = false, unique = true)
    private String email;

    @Column(nullable = false)
    private String password; // stored in the DB, hidden from API responses

    @Column(name = "phone_number")
    private String phoneNumber;
}
```

A few things worth calling out here. The `@Table(name = "users")` is there because some databases treat **user** as a reserved word. Table named **users** avoids that headache.

The **password** field is on the entity intentionally. It gets stored in the database. But notice it will **never appear in API responses** because we're going to use a **Response DTO** that doesn't include it. This is one of those things that separates a thoughtful API from a careless one.

The **@Builder** annotation from Lombok generates a builder pattern. Instead of calling a constructor with five arguments in a specific order, you can write

`User.builder().name("Ananya").email("a@a.com").build()`. Much cleaner, much more readable in tests and service code.

Step 4 - The DTOs

DTOs deserve their own section because this is where a lot of freshers take a shortcut that they later regret. The shortcut is: using the entity class directly in the controller for both receiving and sending data.

Don't do that. Here's why.

Your entity is a database object. It might have sensitive fields (`password`), internal fields (`createdAt`, `lastLoginIp`), or Hibernate-managed fields that the client has no business seeing or setting. When you expose the entity directly, you expose all of that. And when you accept the entity as request input, the client can potentially set fields they shouldn't.

The solution is two separate classes: one for what comes **in** (Request DTO) and one for what goes **out** (Response DTO).

Request DTO - What the Client Sends

```
package com.example.usermanagement.dto;

import jakarta.validation.constraints.*;
import lombok.*;

@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
public class CreateUserRequest {

    @NotBlank(message = "Name is required")
    @Size(min = 2, max = 100, message = "Name must be between 2 and 100 characters")
    private String name;

    @NotBlank(message = "Email is required")
    @Email(message = "Invalid email format")
    private String email;

    @NotBlank(message = "Password is required")
    @Size(min = 8, message = "Password must be at least 8 characters")
```

```
private String password;

    @Pattern(regexp = "[0-9]{10}$", message = "Phone must be a 10-digit
number")
    private String phoneNumber;    // optional, no @NotBlank
}
```

Validation annotations live on the DTO, not the entity. The entity doesn't need to care about HTTP validation rules. That's the controller layer's responsibility, and DTOs carry it.

Notice `phoneNumber` has no `@NotBlank` on it. It's optional. If the client doesn't send it, the field will be null and that's fine. `@Pattern` only fires if the field is non-null and non-blank. This is the right way to handle optional validated fields.

Response DTO - What the Client Receives

```
package com.example.usermanagement.dto;

import lombok.*;

@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class UserResponse {

    private Long id;
    private String name;
    private String email;
    private String phoneNumber;

    // No password field. Ever.
}
```

Simple. No password. No internal metadata that isn't the client's business. When an interviewer asks **"how do you prevent sensitive data from leaking in API responses?"** this is the answer: Response DTOs. Not `@JsonIgnore` on the entity field (that's fragile). Separate response classes that contain only what should be visible.

Step 5 - Repository

This one is short. That's the point. Spring Data JPA does the heavy lifting.

```
package com.example.usermanagement.repository;

import com.example.usermanagement.entity.User;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import java.util.Optional;

@Repository
public interface UserRepository extends JpaRepository<User, Long> {

    // Spring Data JPA generates the SQL from the method name
    Optional<User> findByEmail(String email);

    boolean existsByEmail(String email);
}
```

You extend `JpaRepository<User, Long>`. The first type is the entity, the second is the primary key type. You get `save()`, `findById()`, `findAll()`, `deleteById()` - all the standard CRUD operations- for free. No implementation code needed.

The two custom methods- `findByEmail` and `existsByEmail`- are query methods. Spring Data JPA reads the method name and generates the correct SQL automatically. `findByEmail(email)` becomes `SELECT * FROM users WHERE email = ?`. You write no SQL. You write no implementation. It just works.

Step 6 - Custom Exception

Before we write the service, we need our custom exception. When someone requests a user by ID that doesn't exist, we don't return null or throw a generic `RuntimeException`. We throw a named, meaningful exception that clearly describes what went wrong.

```
package com.example.usermanagement.exception;

public class UserNotFoundException extends RuntimeException {

    private final Long userId;

    public UserNotFoundException(Long userId) {
        super("User not found with ID: " + userId);
    }
}
```

```

        this.userId = userId;
    }

    public Long getUserId() {
        return userId;
    }
}

```

Extending `RuntimeException` means Spring won't require you to declare it in method signatures (it's unchecked). The message is set in the `super()` call, which means when this exception propagates, the message is already human-readable.

Interviewers sometimes ask: **"why extend RuntimeException and not Exception?"** The practical answer is that checked exceptions (extending `Exception`) force every calling method to either catch or re-declare them. In a layered architecture, that creates noise at every layer. For business exceptions like "user not found", `RuntimeException` is cleaner. The global exception handler catches it at the top.

Step 7 - Service Layer

The service is where the business logic lives. In this project it's simple, but the **structure** is what matters. Notice how the service converts between DTOs and entities, enforces business rules (no duplicate emails), and delegates data access entirely to the repository.

```

package com.example.usermanagement.service;

import com.example.usermanagement.dto.*;
import com.example.usermanagement.entity.User;
import com.example.usermanagement.exception.UserNotFoundException;
import com.example.usermanagement.repository.UserRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;
import java.util.List;
import java.util.stream.Collectors;

@Service
@RequiredArgsConstructor
public class UserService {

    private final UserRepository userRepository;

    // — Create User —————
    public UserResponse createUser(CreateUserRequest request) {

```

```

        // Business rule: email must be unique
        if (userRepository.existsByEmail(request.getEmail())) {
            throw new IllegalArgumentException(
                "Email already in use: " + request.getEmail());
        }

        // Convert DTO to Entity
        User user = User.builder()
            .name(request.getName())
            .email(request.getEmail())
            .password(request.getPassword()) // hash in real apps
            .phoneNumber(request.getPhoneNumber())
            .build();

        User saved = userRepository.save(user);

        // Convert Entity to Response DTO
        return toResponse(saved);
    }

    // --- Get All Users ---
    public List<UserResponse> getAllUsers() {
        return userRepository.findAll()
            .stream()
            .map(this::toResponse)
            .collect(Collectors.toList());
    }

    // --- Get User By ID ---
    public UserResponse getUserById(Long id) {
        User user = userRepository.findById(id)
            .orElseThrow(() -> new UserNotFoundException(id));
        return toResponse(user);
    }

    // --- Helper: Entity to Response DTO ---
    private UserResponse toResponse(User user) {
        return UserResponse.builder()
            .id(user.getId())
            .name(user.getName())
            .email(user.getEmail())
            .phoneNumber(user.getPhoneNumber())
            .build();
    }
}

```

The `toResponse()` helper method at the bottom is worth highlighting. It's a private method that maps a `User` entity to a `UserResponse` DTO. Every place in the service that needs to return user data calls this one method. If the `UserResponse` structure ever changes, you update it in one place.

In a real project you'd probably use MapStruct or ModelMapper for this conversion instead of writing it manually. But for an interview project, writing it manually is fine and actually shows the interviewer you understand what's happening under the hood.

One important comment in the code: `// hash in real apps`. Don't store passwords in plain text ever. In a production app you'd use BCrypt via Spring Security's `PasswordEncoder`. For this demo project, it's okay to skip it, but **mention it in the interview**. Say "In a real application I'd hash this with BCrypt before storing." That one line shows you know security basics.

Step 8 - Controller

The controller is the entry point for all HTTP requests. It's intentionally thin. No business logic lives here. It receives a request, hands it to the service, and returns the response.

```
package com.example.usermanagement.controller;

import com.example.usermanagement.dto.*;
import com.example.usermanagement.service.UserService;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/api/users")
@RequiredArgsConstructor
public class UserController {

    private final UserService userService;

    // POST /api/users
    @PostMapping
    public ResponseEntity<UserResponse> createUser(
        @Valid @RequestBody CreateUserRequest request) {

        UserResponse response = userService.createUser(request);
        return ResponseEntity.status(201).body(response);
    }

    // GET /api/users
    @GetMapping
    public ResponseEntity<List<UserResponse>> getAllUsers() {
```

```

        return ResponseEntity.ok(userService.getAllUsers());
    }

    // GET /api/users/{id}
    @GetMapping("/{id}")
    public ResponseEntity<UserResponse> getUserById(@PathVariable Long id) {
        return ResponseEntity.ok(userService.getUserById(id));
    }
}

```

That `@Valid` annotation on the `@RequestBody` parameter is what triggers Spring's validation. Without it, all those `@NotBlank`, `@Email`, `@Size` annotations on the DTO do nothing. `@Valid` is the switch that says "please actually validate this object".

When validation fails, Spring throws a `MethodArgumentNotValidException` with details about every field that failed. Our global exception handler will catch that and return a proper 400 response. We'll write that next.

Step 9 - Global Exception Handler

This is the part most beginners skip, and it's what separates a toy project from a production-quality one.

Right now, if you request a user that doesn't exist, Spring returns a 500 with a wall of Hibernate stacktrace. If validation fails, you get a 400 but with Spring's default error format which is verbose and inconsistent. We want clean, consistent error responses across all endpoints.

The `@ControllerAdvice` class catches exceptions from all controllers in one place:

```

package com.example.usermanagement.exception;

import org.springframework.http.*;
import org.springframework.validation.*;
import org.springframework.web.bind.*;
import org.springframework.web.bind.annotation.*;
import java.util.*;
import java.time.LocalDateTime;

@RestControllerAdvice
public class GlobalExceptionHandler {

    // Handles our custom UserNotFoundException
    @ExceptionHandler(UserNotFoundException.class)

```

```
public ResponseEntity<ErrorResponse> handleUserNotFound(
    UserNotFoundException ex) {

    ErrorResponse error = new ErrorResponse(
        HttpStatus.NOT_FOUND.value(),
        "User Not Found",
        ex.getMessage(),
        LocalDateTime.now()
    );
    return ResponseEntity.status(404).body(error);
}

// Handles @Valid validation failures
@ExceptionHandler(MethodArgumentNotValidException.class)
public ResponseEntity<Map<String, Object>> handleValidation(
    MethodArgumentNotValidException ex) {

    Map<String, String> fieldErrors = new LinkedHashMap<>();
    ex.getBindingResult().getFieldErrors().forEach(err ->
        fieldErrors.put(err.getField(), err.getDefaultMessage())
    );

    Map<String, Object> body = new LinkedHashMap<>();
    body.put("status", 400);
    body.put("error", "Validation Failed");
    body.put("fields", fieldErrors);
    body.put("timestamp", LocalDateTime.now());

    return ResponseEntity.status(400).body(body);
}

// Handles duplicate email
@ExceptionHandler(IllegalArgumentException.class)
public ResponseEntity<ErrorResponse> handleIllegalArgument(
    IllegalArgumentException ex) {

    ErrorResponse error = new ErrorResponse(
        HttpStatus.CONFLICT.value(),
        "Conflict",
        ex.getMessage(),
        LocalDateTime.now()
    );
    return ResponseEntity.status(409).body(error);
}

// Fallback for anything unexpected
@ExceptionHandler(Exception.class)
public ResponseEntity<ErrorResponse> handleGeneral(Exception ex) {
    ErrorResponse error = new ErrorResponse(
        500, "Internal Server Error",
        "Something went wrong. Please try again.",
```

```
        LocalDateTime.now()
    );
    return ResponseEntity.status(500).body(error);
}
}
```

And the **ErrorResponse** class that structures the error:

```
package com.example.usermanagement.exception;

import lombok.*;
import java.time.LocalDateTime;

@Getter
@AllArgsConstructor
public class ErrorResponse {
    private int status;
    private String error;
    private String message;
    private LocalDateTime timestamp;
}
```

With this in place, every error your API returns looks consistent and readable. No stacktraces. No Spring internals leaking to the client. Just a clean JSON body with status, error type, message, and timestamp.

Step 10 - API Design and Sample Requests

Let's see the full picture of what this API looks like from the outside. This is what an interviewer or technical reviewer would see when they test your project.

POST /api/users - Create a User

```
POST http://localhost:8080/api/users
Content-Type: application/json
```

```
{
  "name": "Ananya Sharma",
  "email": "ananya@example.com",
  "password": "securepass123",
  "phoneNumber": "9876543210"
}
```

Success Response 201 Created

```
{
  "id": 1,
  "name": "Ananya Sharma",
  "email": "ananya@example.com",
  "phoneNumber": "9876543210"
}
```

Notice: no **password** in the response. The Response DTO doesn't have that field. The client gets back exactly what they need to confirm creation- id, name, email, phone.

Validation Failure Response 400 Bad Request

```
{
  "status": 400,
  "error": "Validation Failed",
  "fields": {
    "email": "Invalid email format",
    "password": "Password must be at least 8 characters"
  },
  "timestamp": "2024-03-15T14:32:10"
}
```

GET /api/users - Fetch All Users

```
GET http://localhost:8080/api/users
```

Response 200 OK

```
[
  {
    "id": 1,
    "name": "Ananya Sharma",
    "email": "ananya@example.com",
    "phoneNumber": "9876543210"
  },
  {
    "id": 2,
    "name": "Raj Patel",
    "email": "raj@example.com",
    "phoneNumber": null
  }
]
```

GET /api/users/{id} - Fetch by ID

```
GET http://localhost:8080/api/users/1
```

Success Response 200 OK

```
{
  "id": 1,
  "name": "Ananya Sharma",
  "email": "ananya@example.com",
  "phoneNumber": "9876543210"
}
```

Not Found Response 404 Not Found

```
{
  "status": 404,
  "error": "User Not Found",
  "message": "User not found with ID: 99",
  "timestamp": "2024-03-15T14:35:22"
}
```

The Complete Flow in One View

This diagram is worth memorising. Draw it on a whiteboard in an interview and explain each layer as you go.

```
Client sends: POST /api/users {name, email, password, phoneNumber }
```

```
|
v
```

```
UserController.createUser(@Valid @RequestBody CreateUserRequest)
  @Valid triggers validation on CreateUserRequest
  If validation fails --> GlobalExceptionHandler --> 400 response
  If valid, delegates to service
```

```
|
v
```

```
UserService.createUser(request)
  Checks if email already exists --> 409 if duplicate
  Converts CreateUserRequest --> User entity
  Calls userRepository.save(user)
  Converts User entity --> UserResponse DTO
  Returns UserResponse
```

```
|
v
```

```

userRepository.save(user)
    Spring Data JPA generates INSERT SQL
    Hibernate executes against H2 database
    Returns saved User with generated ID

    |
    v (back up the chain)

UserController returns ResponseEntity.status(201).body(userResponse)

Client receives: 201 Created { id, name, email, phoneNumber }

```

What This Project Tests in an Interview

Every piece of this project maps to an interview concept. Here's the map:

What You Built	Concept Being Tested
Layered package structure	Separation of concerns, architecture thinking
@Entity with @Id @GeneratedValue	JPA fundamentals, primary key strategies
CreateUserRequest with @NotBlank @Email	Input validation, Bean Validation API
UserResponse DTO (no password field)	Data hiding, DTO pattern, API security awareness
UserRepository extends JpaRepository	Spring Data JPA, derived query methods
UserService with DTO-Entity conversion	Service layer pattern, builder pattern
@Valid on @RequestBody in controller	How validation is triggered in Spring MVC
UserNotFoundException extends RuntimeException	Custom exceptions, unchecked vs checked
@RestControllerAdvice with @ExceptionHandler	Global exception handling, consistent errors
ResponseEntity with correct status codes	HTTP semantics, 201 vs 200 vs 404 vs 409

Common Questions About This Project

Interviewers don't just look at your code. They ask follow-up questions. Here are the ones that come up most, with honest answers.

"Why use a DTO instead of the entity directly?"

Because the entity is a database object. The DTO is an API contract. They have different concerns and different lifecycles. The entity might change for database reasons; the DTO might change for client compatibility reasons. Mixing them creates fragile code. And practically, you never want to accidentally expose a password or an internal audit field through an API.

"Why does @Valid need to be on the method parameter?"

Because Spring doesn't validate request objects automatically. The `@Valid` annotation is explicit instruction to Spring's data binding layer: "before passing this object to the method, run the Bean Validation constraints on it." Without it, your constraints are declared but never enforced.

"Why @RestControllerAdvice instead of try-catch in each controller?"

Because you'd be writing the same catch blocks in every controller method. `@RestControllerAdvice` centralises exception handling across your entire application. One class handles it all. When the error response format changes, you change one class. When you add a new exception, you add one handler. Cleaner, more maintainable, easier to test.

"You're storing passwords in plain text. That's a security issue."

Yes, and you should bring this up **before** they do. Say something like: "For this demo project I've skipped hashing to keep it focused, but in a production app I'd use BCryptPasswordEncoder from Spring Security. Passwords should never be stored as plain text." That answer shows awareness. Waiting for them to catch it and then scrambling to explain looks bad.

"How would you add pagination to GET all users?"

Spring Data JPA has built-in pagination. Change the return type of `findAll()` to accept a `Pageable` object and return a `Page<User>`. The controller accepts `@RequestParam int page` and `@RequestParam int size` and passes them to `PageRequest.of(page, size)`. It's a few lines of change, not a redesign.

FINAL INSIGHT

If you can build this project from scratch, explain each layer without reading from notes, and answer the follow-up questions without freezing, you are ready for most fresher-level Java interviews in India.

Seriously. Most interviews at service companies, mid-size product companies, and even some early-stage startups are testing **exactly** this level of depth. Can you structure code properly? Do you understand why DTOs exist? Can you handle errors gracefully? Can you talk about what you'd do differently in production?

Those questions don't require five years of experience. They require **understanding**. And if you've built this project and thought through every decision, you have that.

Things to Know Cold Before Any Interview:

Be able to draw the Controller > Service > Repository flow on paper in 30 seconds

Know why you use a Request DTO and a Response DTO separately

Know what @Valid does and why it must be on the method parameter

Be able to explain what @RestControllerAdvice does in one clear sentence

Know your HTTP status codes: 200, 201, 204, 400, 404, 409, 500

Have a rehearsed answer for 'why not store passwords in plain text'

Know how to add pagination with Pageable if asked as a follow-up

One last thing. When you show this project in an interview, don't just paste the code and say "here it is". **Walk through it.** Say "I structured it in layers because...", "I used a Response DTO because...", "The exception handler sits here because...". Narrating your decisions is what turns code on a screen into proof that you understand what you built.

That is the difference between someone who copied a tutorial and someone who actually learned.

Build it. Break it. Fix it. Explain it. That is how you get hired.

End of Chapter 15 - The Spring Boot Playbook